

3



Understanding Quality Attributes

Quality is never an accident; it is always the result of high intention, sincere effort, intelligent direction and skillful execution.

—William A. Foster

Many factors determine the qualities that must be provided for in a system's architecture. These qualities go beyond functionality, which is the basic statement of the system's capabilities, services, and behavior. Although functionality and other qualities are closely related, as you will see, functionality often takes the front seat in the development scheme. This preference is shortsighted, however. Systems are frequently redesigned not because they are functionally deficient—the replacements are often functionally identical—but because they are difficult to maintain, port, or scale; or they are too slow; or they have been compromised by hackers. In Chapter 2, we said that architecture was the first place in software creation in which the achievement of quality requirements could be addressed. It is the mapping of a system's functionality onto software structures that determines the architecture's support for qualities. In Chapters 4–14, we discuss how various qualities are supported by architectural design decisions. In Chapter 20, we show how to integrate all of your drivers, including quality attribute decisions, into a coherent design.

We have been using the term “quality attribute” loosely, but now it is time to define it more carefully. A quality attribute (QA) is a measurable or testable property of a system that is used to indicate how well the system satisfies the needs of its stakeholders beyond the basic function of the system. You can think of a quality attribute as measuring the “utility” of a product along some dimension of interest to a stakeholder.

In this chapter our focus is on understanding the following:

- How to express the qualities we want our architecture to exhibit
- How to achieve the qualities through architectural means
- How to determine the design decisions we might make with respect to the qualities

This chapter provides the context for the discussions of individual quality attributes in Chapters 4–14.

3.1 Functionality

Functionality is the ability of the system to do the work for which it was intended. Of all of the requirements, functionality has the strangest relationship to architecture.

First of all, functionality does not determine architecture. That is, given a set of required functionality, there is no end to the architectures you could create to satisfy that functionality. At the very least, you could divide up the functionality in any number of ways and assign the sub-pieces to different architectural elements.

In fact, if functionality were the only thing that mattered, you wouldn't have to divide the system into pieces at all: A single monolithic blob with no internal structure would do just fine. Instead, we design our systems as structured sets of cooperating architectural elements—modules, layers, classes, services, databases, apps, threads, peers, tiers, and on and on—to make them understandable and to support a variety of other purposes. Those “other purposes” are the other quality attributes that we'll examine in the remaining sections of this chapter, and in the subsequent quality attribute chapters in Part II.

Although functionality is independent of any particular structure, it is achieved by assigning responsibilities to architectural elements. This process results in one of the most basic architectural structures—module decomposition.

Although responsibilities can be allocated arbitrarily to any module, software architecture constrains this allocation when other quality attributes are important. For example, systems are frequently (or perhaps always) divided so that several people can cooperatively build them. The architect's interest in functionality is how it interacts with and constrains other qualities.

Functional Requirements

After more than 30 years of writing about and discussing the distinction between functional requirements and quality requirements, the definition of functional requirements still eludes me. Quality attribute requirements are well defined: Performance has to do with the system's timing behavior, modifiability has to do with the system's ability to support changes in its behavior or other qualities after initial deployment, availability has to do with the system's ability to survive failures, and so forth.

Function, however, is a much more slippery concept. An international standard (ISO 25010) defines functional suitability as “the capability of the software product to provide functions which meet stated and implied needs when the software is used under specified conditions.” That is, functionality is the ability to provide functions. One interpretation of this definition is that functionality describes what the system does and quality describes how well the system does its function. That is, qualities are attributes of the system and function is the purpose of the system.

This distinction breaks down, however, when you consider the nature of some of the “function.” If the function of the software is to control engine behavior, how can the function be correctly implemented without considering timing behavior? Is the ability to control access by requiring a user name/password combination not a function, even though it is not the purpose of any system?

I much prefer using the word “responsibility” to describe computations that a system must perform. Questions such as “What are the timing constraints on that set of responsibilities?”, “What modifications are anticipated with respect to that set of responsibilities?”, and “What class of users is allowed to execute that set of responsibilities?” make sense and are actionable.

The achievement of qualities induces responsibility; think of the user name/password example just mentioned. Further, one can identify responsibilities as being associated with a particular set of requirements.

So does this mean that the term “functional requirement” shouldn’t be used? People have an understanding of the term, but when precision is desired, we should talk about sets of specific responsibilities instead.

Paul Clements has long ranted against the careless use of the term “nonfunctional,” and now it’s my turn to rant against the careless use of the term “functional”—which is probably equally ineffectually.

—LB

3.2 Quality Attribute Considerations

Just as a system’s functions do not stand on their own without due consideration of quality attributes, neither do quality attributes stand on their own; they pertain to the functions of the system. If a functional requirement is “When the user presses the green button, the Options dialog appears,” a performance QA annotation might describe how quickly the dialog will appear; an availability QA annotation might describe how often this function is allowed to fail, and how quickly it will be repaired; a usability QA annotation might describe how easy it is to learn this function.

Quality attributes as a distinct topic have been studied by the software community at least since the 1970s. A variety of taxonomies and definitions have been published (we discuss some of these in Chapter 14), many of which have their own research and practitioner communities. However, there are three problems with most discussions of system quality attributes:

1. The definitions provided for an attribute are not testable. It is meaningless to say that a system will be “modifiable.” Every system will be modifiable with respect to one set of changes and not modifiable with respect to another. The other quality attributes are similar in this regard: A system may be robust with respect to some faults and brittle with respect to others, and so forth.
2. Discussion often focuses on which quality a particular issue belongs to. Is a denial-of-service attack on a system an aspect of availability, an aspect of performance, an aspect of security, or an aspect of usability? All four attribute communities would claim “ownership” of the denial-of-service attack. All are, to some extent, correct. But this debate over categorization doesn’t help us, as architects, understand and create architectural solutions to actually manage the attributes of concern.

3. Each attribute community has developed its own vocabulary. The performance community has “events” arriving at a system, the security community has “attacks” arriving at a system, the availability community has “faults” arriving, and the usability community has “user input.” All of these may actually refer to the same occurrence, but they are described using different terms.

A solution to the first two problems (untestable definitions and overlapping issues) is to use *quality attribute scenarios* as a means of characterizing quality attributes (see Section 3.3). A solution to the third problem is to illustrate the concepts that are fundamental to that attribute community in a common form, which we do in Chapters 4–14.

We will focus on two categories of quality attributes. The first category includes those attributes that describe some property of the system at runtime, such as availability, performance, or usability. The second category includes those that describe some property of the development of the system, such as modifiability, testability, or deployability.

Quality attributes can never be achieved in isolation. The achievement of any one will have an effect—sometimes positive and sometimes negative—on the achievement of others. For example, almost every quality attribute negatively affects performance. Take portability: The main technique for achieving portable software is to isolate system dependencies, which introduces overhead into the system’s execution, typically as process or procedure boundaries, which then hurts performance. Determining a design that may satisfy quality attribute requirements is partially a matter of making the appropriate tradeoffs; we discuss design in Chapter 21.

In the next three sections, we focus on how quality attributes can be specified, what architectural decisions will enable the achievement of particular quality attributes, and what questions about quality attributes will enable the architect to make the correct design decisions.

3.3 Specifying Quality Attribute Requirements: Quality Attribute Scenarios

We use a common form to specify all QA requirements as scenarios. This addresses the vocabulary problems we identified previously. The common form is testable and unambiguous; it is not sensitive to whims of categorization. Thus it provides regularity in how we treat all quality attributes.

Quality attribute scenarios have six parts:

- *Stimulus*. We use the term “stimulus” to describe an event arriving at the system or the project. The stimulus can be an *event* to the performance community, a *user operation* to the usability community, or an *attack* to the security community, and so forth. We use the same term to describe a motivating action for developmental qualities. Thus a stimulus for modifiability is a request for a modification; a stimulus for testability is the completion of a unit of development.
- *Stimulus source*. A stimulus must have a source—it must come from somewhere. Some entity (a human, a computer system, or any other actor) must have generated the

stimulus. The source of the stimulus may affect how it is treated by the system. A request from a trusted user will not undergo the same scrutiny as a request by an untrusted user.

- *Response.* The response is the activity that occurs as the result of the arrival of the stimulus. The response is something the architect undertakes to satisfy. It consists of the responsibilities that the system (for runtime qualities) or the developers (for development-time qualities) should perform in response to the stimulus. For example, in a performance scenario, an event arrives (the stimulus) and the system should process that event and generate a response. In a modifiability scenario, a request for a modification arrives (the stimulus) and the developers should implement the modification—without side effects—and then test and deploy the modification.
- *Response measure.* When the response occurs, it should be measurable in some fashion so that the scenario can be tested—that is, so that we can determine if the architect achieved it. For performance, this could be a measure of latency or throughput; for modifiability, it could be the labor or wall clock time required to make, test, and deploy the modification.

These four characteristics of a scenario are the heart of our quality attribute specifications. But two more characteristics are important, yet often overlooked: environment and artifact.

- *Environment.* The environment is the set of circumstances in which the scenario takes place. Often this refers to a runtime state: The system may be in an overload condition or in normal operation, or some other relevant state. For many systems, “normal” operation can refer to one of a number of modes. For these kinds of systems, the environment should specify in which mode the system is executing. But the environment can also refer to states in which the system is not running at all: when it is in development, or testing, or refreshing its data, or recharging its battery between runs. The environment sets the context for the rest of the scenario. For example, a request for a modification that arrives after the code has been frozen for a release may be treated differently than one that arrives before the freeze. The fifth successive failure of a component may be treated differently than the first failure of that component.
- *Artifact.* The stimulus arrives at some target. This is often captured as just the system or project itself, but it’s helpful to be more precise if possible. The artifact may be a collection of systems, the whole system, or one or more pieces of the system. A failure or a change request may affect just a small portion of the system. A failure in a data store may be treated differently than a failure in the metadata store. Modifications to the user interface may have faster response times than modifications to the middleware.

To summarize, we capture quality attribute requirements as six-part scenarios. While it is common to omit one or more of these six parts, particularly in the early stages of thinking about quality attributes, knowing that all of the parts are there forces the architect to consider whether each part is relevant.

We have created a general scenario for each of the quality attributes presented in Chapters 4–13 to facilitate brainstorming and elicitation of concrete scenarios. We distinguish

general quality attribute scenarios—general scenarios—which are system independent and can pertain to any system, from *concrete* quality attribute scenarios—concrete scenarios—which are specific to the particular system under consideration.

To translate these generic attribute characterizations into requirements for a particular system, the general scenarios need to be made system specific. But, as we have found, it is much easier for a stakeholder to tailor a general scenario into one that fits their system than it is for them to generate a scenario from thin air.

Figure 3.1 shows the parts of a quality attribute scenario just discussed. Figure 3.2 shows an example of a general scenario, in this instance for availability.

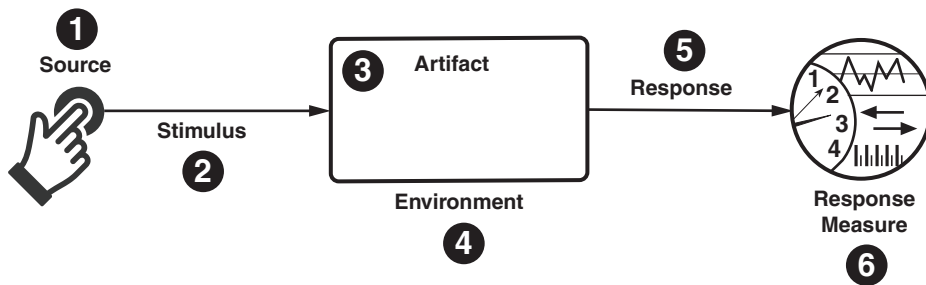


FIGURE 3.1 The parts of a quality attribute scenario

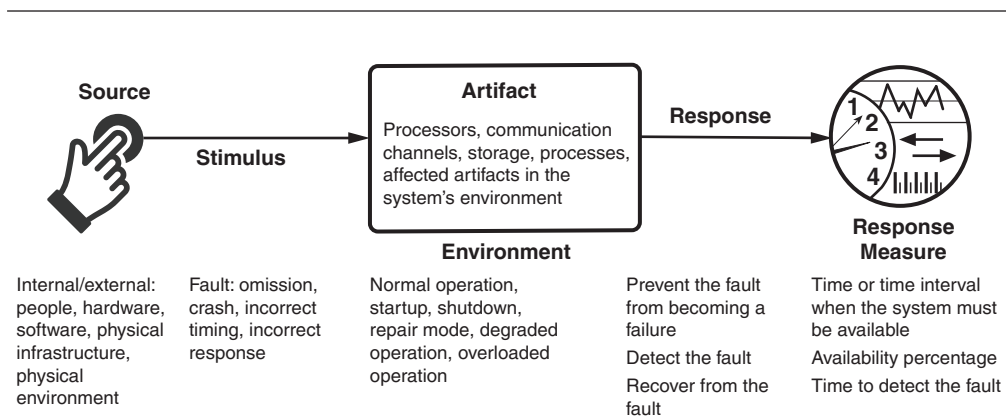


FIGURE 3.2 A general scenario for availability

Not My Problem

Some time ago I was doing an architecture analysis on a complex system created by and for Lawrence Livermore National Laboratory. If you visit this organization's website (llnl.gov) and try to figure out what Livermore Labs does, you will see the word "security" mentioned over and over. The lab focuses on nuclear security, international and domestic security, and environmental and energy security. Serious stuff . . .

Keeping this emphasis in mind, I asked my clients to describe the quality attributes of concern for the system that I was analyzing. I'm sure you can imagine my surprise when security wasn't mentioned once! The system stakeholders mentioned performance, modifiability, evolvability, interoperability, configurability, and portability, and one or two more, but the word "security" never passed their lips.

Being a good analyst, I questioned this seemingly shocking and obvious omission. Their answer was simple and, in retrospect, straightforward: "We don't care about it. Our systems are not connected to any external network, and we have barbed-wire fences and guards with machine guns."

Of course, *someone* at Livermore Labs was very interested in security. But not the software architects. The lesson here is that the software architect may not bear the responsibility for every QA requirement.

—RK

3.4 Achieving Quality Attributes through Architectural Patterns and Tactics

We now turn to the techniques an architect can use to *achieve* the required quality attributes: architectural patterns and tactics.

A tactic is a design decision that influences the achievement of a quality attribute response—it directly affects the system's response to some stimulus. Tactics may impart portability to one design, high performance to another, and integrability to a third.

An architectural pattern describes a particular recurring design problem that arises in specific design contexts and presents a well-proven architectural solution for the problem. The solution is specified by describing the roles of its constituent elements, their responsibilities and relationships, and the ways in which they collaborate. Like the choice of tactics, the choice of an architectural pattern has a profound effect on quality attributes—usually more than one.

Patterns typically comprise multiple design decisions and, in fact, often comprise multiple quality attribute tactics. We say that patterns often bundle tactics and, consequently, frequently make tradeoffs among quality attributes.

We will look at example relationships between tactics and patterns in each of our quality attribute-specific chapters. Chapter 14 explains how a set of tactics for any quality attribute can be constructed; those tactics are, in fact, the steps we used to produce the sets found in this book.

While we discuss patterns and tactics as though they were foundational design decisions, the reality is that architectures often emerge and evolve as a result of many small decisions and business forces. For example, a system that was once tolerably modifiable may deteriorate over time, through the actions of developers adding features and fixing bugs. Similarly, a system's performance, availability, security, and any other quality may (and typically does) deteriorate over time, again through the well-intentioned actions of programmers who are focused on their immediate tasks and not on preserving architectural integrity.

This “death by a thousand cuts” is common on software projects. Developers may make suboptimal decisions due to a lack of understanding of the structures of the system, schedule pressures, or perhaps a lack of clarity in the architecture from the start. This kind of deterioration is a form of technical debt known as architecture debt. We discuss architecture debt in Chapter 23. To reverse this debt, we typically refactor.

Refactoring may be done for many reasons. For example, you might refactor a system to improve its security, placing different modules into different subsystems based on their security properties. Or you might refactor a system to improve its performance, removing bottlenecks and rewriting slow portions of the code. Or you might refactor to improve the system's modifiability. For example, when two modules are affected by the same kinds of changes over and over because they are (at least partial) duplicates of each other, the common functionality could be factored out into its own module, thereby improving cohesion and reducing the number of places that need to be changed when the next (similar) change request arrives.

Code refactoring is a mainstay practice of agile development projects, as a cleanup step to make sure that teams have not produced duplicative or overly complex code. However, the concept applies to architectural elements as well.

Successfully achieving quality attributes often involves process-related decisions, in addition to architecture-related decisions. For example, a great security architecture is worthless if your employees are susceptible to phishing attacks or do not choose strong passwords. We are not dealing with the process aspects in this book, but be aware that they are important.

3.5 Designing with Tactics

A system design consists of a collection of decisions. Some of these decisions help control the quality attribute responses; others ensure achievement of system functionality. We depict this relationship in Figure 3.3. Tactics, like patterns, are design techniques that architects have been using for years. In this book, we isolate, catalog, and describe them. We are not inventing tactics here, but rather just capturing what good architects do in practice.

Why do we focus on tactics? There are three reasons:

1. Patterns are foundational for many architectures, but sometimes there may be no pattern that solves your problem completely. For example, you might need the high-availability high-security broker pattern, not the textbook broker pattern. Architects frequently need to modify and adapt patterns to their particular context, and tactics provide a systematic means for augmenting an existing pattern to fill the gaps.

2. If no pattern exists to realize the architect's design goal, tactics allow the architect to construct a design fragment from "first principles." Tactics give the architect insight into the properties of the resulting design fragment.
3. Tactics provide a way of making design and analysis more systematic within some limitations. We'll explore this idea in the next section.

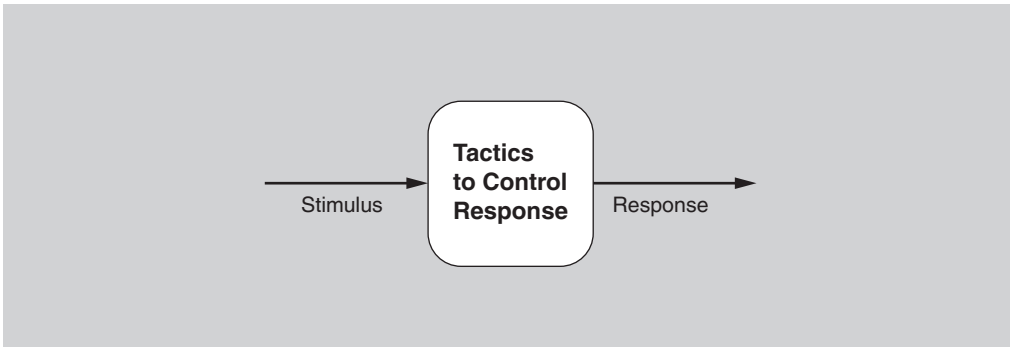


FIGURE 3.3 Tactics are intended to control responses to stimuli.

Like any design concept, the tactics that we present here can and should be refined as they are applied to design a system. Consider performance: *Schedule resources* is a common performance tactic. But this tactic needs to be refined into a specific scheduling strategy, such as shortest-job-first, round-robin, and so forth, for specific purposes. *Use an intermediary* is a modifiability tactic. But there are multiple types of intermediaries (layers, brokers, proxies, and tiers, to name just a few), which are realized in different ways. Thus a designer will employ refinements to make each tactic concrete.

In addition, the application of a tactic depends on the context. Again, consider performance: *Manage sampling rate* is relevant in some real-time systems but not in all real-time systems, and certainly not in database systems or stock-trading systems where losing a single event is highly problematic.

Note that there are some “super-tactics”—tactics that are so fundamental and so pervasive that they deserve special mention. For example, the modifiability tactics of encapsulation, restricting dependencies, using an intermediary, and abstracting common services are found in the realization of almost every pattern ever! But other tactics, such as the scheduling tactic from performance, also appear in many places. For example, a load balancer is an intermediary that does scheduling. We see monitoring appearing in many quality attributes: We monitor aspects of a system to achieve energy efficiency, performance, availability, and safety. Thus we should not expect a tactic to live in only one place, for just a single quality attribute. Tactics are design primitives and, as such, are found over and over in different aspects of design. This is actually an argument for why tactics are so powerful and deserving of our attention—and yours. Get to know them; they’ll be your friends.

3.6 Analyzing Quality Attribute Design Decisions: Tactics-Based Questionnaires

In this section, we introduce a tool the analyst can use to understand potential quality attribute behavior at various stages through the architecture's design: tactics-based questionnaires.

Analyzing how well quality attributes have been achieved is a critical part of the task of designing an architecture. And (no surprise) you shouldn't wait until your design is complete before you begin to do it. Opportunities for quality attribute analysis crop up at many different points in the software development life cycle, even very early ones.

At any point, the analyst (who might be the architect) needs to respond appropriately to whatever artifacts have been made available for analysis. The accuracy of the analysis and expected degree of confidence in the analysis results will vary according to the maturity of the available artifacts. But no matter the state of the design, we have found tactics-based questionnaires to be helpful in gaining insights into the architecture's ability (or likely ability, as it is refined) to provide the needed quality attributes.

In Chapters 4–13, we include a tactics-based questionnaire for each quality attribute covered in the chapters. For each question in the questionnaire, the analyst records the following information:

- Whether each tactic is supported by the system's architecture.
- Whether there are any obvious risks in the use (or nonuse) of this tactic. If the tactic has been used, record how it is realized in the system, or how it is intended to be realized (e.g., via custom code, generic frameworks, or externally produced components).
- The specific design decisions made to realize the tactic and where in the code base the implementation (realization) may be found. This is useful for auditing and architecture reconstruction purposes.
- Any rationale or assumptions made in the realization of this tactic.

To use these questionnaires, simply follow these four steps:

1. For each tactics question, fill the "Supported" column with "Y" if the tactic is supported in the architecture and with "N" otherwise.
2. If the answer in the "Supported" column is "Y," then in the "Design Decisions and Location" column describe the specific design decisions made to support the tactic and enumerate where these decisions are, or will be, manifested (located) in the architecture. For example, indicate which code modules, frameworks, or packages implement this tactic.
3. In the "Risk" column indicate the risk of implementing the tactic using a (H = High, M = Medium, L = Low) scale.
4. In the "Rationale" column, describe the rationale for the design decisions made (including a decision to *not* use this tactic). Briefly explain the implications of this decision. For example, explain the rationale and implications of the decision in terms of the effort on cost, schedule, evolution, and so forth.

While this questionnaire-based approach might sound simplistic, it can actually be very powerful and insightful. Addressing the set of questions forces the architect to take a step back and consider the bigger picture. This process can also be quite efficient: A typical questionnaire for a single quality attribute takes between 30 and 90 minutes to complete.

3.7 Summary

Functional requirements are satisfied by including an appropriate set of responsibilities within the design. *Quality attribute* requirements are satisfied by the structures and behaviors of the architecture.

One challenge in architectural design is that these requirements are often captured poorly, if at all. To capture and express a quality attribute requirement, we recommend the use of a quality attribute scenario. Each scenario consists of six parts:

1. Source of stimulus
2. Stimulus
3. Environment
4. Artifact
5. Response
6. Response measure

An architectural tactic is a design decision that affects a quality attribute response. The focus of a tactic is on a single quality attribute response. An architectural pattern describes a particular recurring design problem that arises in specific design contexts and presents a well-proven architectural solution for the problem. Architectural patterns can be seen as “bundles” of tactics.

An analyst can understand the decisions made in an architecture through the use of a tactics-based checklist. This lightweight architecture analysis technique can provide insights into the strengths and weaknesses of the architecture in a very short amount of time.

3.8 For Further Reading

Some extended case studies showing how tactics and patterns are used in design can be found in [Cervantes 16].

A substantial catalog of architectural patterns can be found in the five-volume set *Pattern-Oriented Software Architecture*, by Frank Buschmann et al.

Arguments showing that many different architectures can provide the same functionality—that is, that architecture and functionality are largely orthogonal—can be found in [Shaw 95].

3.9 Discussion Questions

1. What is the relationship between a use case and a quality attribute scenario? If you wanted to add quality attribute information to a use case, how would you do it?
2. Do you suppose that the set of tactics for a quality attribute is finite or infinite? Why?
3. Enumerate the set of responsibilities that an automatic teller machine should support and propose a design to accommodate that set of responsibilities. Justify your proposal.
4. Choose an architecture that you are familiar with (or choose the ATM architecture you defined in question 3) and walk through the performance tactics questionnaire (found in Chapter 9). What insight did these questions provide into the design decisions made (or not made)?

4



Availability

*Technology does not always rhyme
with perfection and reliability.
Far from it in reality!*
—Jean-Michel Jarre

Availability refers to a property of software—namely, that it is there and ready to carry out its task when you need it to be. This is a broad perspective and encompasses what is normally called reliability (although it may encompass additional considerations such as downtime due to periodic maintenance). Availability builds on the concept of reliability by adding the notion of recovery—that is, when the system breaks, it repairs itself. Repair may be accomplished by various means, as we’ll see in this chapter.

Availability also encompasses the ability of a system to mask or repair faults such that they do not become failures, thereby ensuring that the cumulative service outage period does not exceed a required value over a specified time interval. This definition subsumes concepts of reliability, robustness, and any other quality attribute that involves a concept of unacceptable failure.

A failure is the deviation of the system from its specification, where that deviation is externally visible. Determining that a failure has occurred requires some external observer in the environment.

A failure’s cause is called a *fault*. A fault can be either internal or external to the system under consideration. Intermediate states between the occurrence of a fault and the occurrence of a failure are called errors. Faults can be prevented, tolerated, removed, or forecast. Through these actions, a system becomes “resilient” to faults. Among the areas with which we are concerned are how system faults are detected, how frequently system faults may occur, what happens when a fault occurs, how long a system is allowed to be out of operation, when faults or failures may occur safely, how faults or failures can be prevented, and what kinds of notifications are required when a failure occurs.

Availability is closely related to, but clearly distinct from, security. A denial-of-service attack is explicitly designed to make a system fail—that is, to make it unavailable. Availability is also closely related to performance, since it may be difficult to tell when a system has failed

and when it is simply being egregiously slow to respond. Finally, availability is closely allied with safety, which is concerned with keeping the system from entering a hazardous state and recovering or limiting the damage when it does.

One of the most demanding tasks in building a high-availability fault-tolerant system is to understand the nature of the failures that can arise during operation. Once those are understood, mitigation strategies can be designed into the system.

Since a system failure is observable by users, the time to repair is the time until the failure is no longer observable. This may be an imperceptible delay in a user's response time or it may be the time it takes someone to fly to a remote location in the Andes to repair a piece of mining machinery (as was recounted to us by a person responsible for repairing the software in a mining machine engine). The notion of "observability" is critical here: If a failure *could have been observed*, then it is a failure, whether or not it was actually observed.

In addition, we are often concerned with the level of capability that remains when a failure has occurred—a degraded operating mode.

Distinguishing between faults and failures allows us to discuss repair strategies. If code containing a fault is executed but the system is able to recover from the fault without any observable deviation from the otherwise specified behavior, we say that no failure has occurred.

The availability of a system can be measured as the probability that it will provide the specified services within the required bounds over a specified time interval. A well-known expression is used to derive steady-state availability (which came from the world of hardware):

$$MTBF/(MTBF + MTTR)$$

where *MTBF* refers to the mean time between failures and *MTTR* refers to the mean time to repair. In the software world, this formula should be interpreted to mean that when thinking about availability, you should think about what will make your system fail, how likely it is that such an event will occur, and how much time will be required to repair it.

From this formula, it is possible to calculate probabilities and make claims like "the system exhibits 99.999 percent availability" or "there is a 0.001 percent probability that the system will not be operational when needed." Scheduled downtimes (when the system is intentionally taken out of service) should not be considered when calculating availability, since the system is deemed "not needed" then; of course, this is dependent on the specific requirements for the system, which are often encoded in a service level agreement (SLA). This may lead to seemingly odd situations where the system is down and users are waiting for it, but the downtime is scheduled and so is not counted against any availability requirements.

Detected faults can be categorized prior to being reported and repaired. This categorization is commonly based on the fault's severity (critical, major, or minor) and service impact (service-affecting or non-service-affecting). It provides the system operator with a timely and accurate system status and allows for an appropriate repair strategy to be employed. The repair strategy may be automated or may require manual intervention.

As just mentioned, the availability expected of a system or service is frequently expressed as an SLA. The SLA specifies the availability level that is guaranteed and, usually, the

penalties that the provider will suffer if the SLA is violated. For example, Amazon provides the following SLA for its EC2 cloud service:

AWS will use commercially reasonable efforts to make the Included Services each available for each AWS region with a Monthly Uptime Percentage of at least 99.99%, in each case during any monthly billing cycle (the “Service Commitment”). In the event any of the Included Services do not meet the Service Commitment, you will be eligible to receive a Service Credit as described below.

Table 4.1 provides examples of system availability requirements and associated threshold values for acceptable system downtime, measured over observation periods of 90 days and one year. The term *high availability* typically refers to designs targeting availability of 99.999 percent (“5 nines”) or greater. As mentioned earlier, only unscheduled outages contribute to system downtime.

TABLE 4.1 System Availability Requirements

Availability	Downtime/90 Days	Downtime/Year
99.0%	21 hr, 36 min	3 days, 15.6 hr
99.9%	2 hr, 10 min	8 hr, 0 min, 46 sec
99.99%	12 min, 58 sec	52 min, 34 sec
99.999%	1 min, 18 sec	5 min, 15 sec
99.9999%	8 sec	32 sec

4.1 Availability General Scenario

We can now describe the individual portions of an availability general scenario as summarized in Table 4.2.

TABLE 4.2 Availability General Scenario

Portion of Scenario	Description	Possible Values
Source	This specifies where the fault comes from.	Internal/external: people, hardware, software, physical infrastructure, physical environment
Stimulus	The stimulus to an availability scenario is a fault.	Fault: omission, crash, incorrect timing, incorrect response
Artifact	This specifies which portions of the system are responsible for and affected by the fault.	Processors, communication channels, storage, processes, affected artifacts in the system’s environment

continues

TABLE 4.2 Availability General Scenario *continued*

Portion of Scenario	Description	Possible Values
Environment	We may be interested in not only how a system behaves in its “normal” environment, but also how it behaves in situations such as when it is already recovering from a fault.	Normal operation, startup, shutdown, repair mode, degraded operation, overloaded operation
Response	The most commonly desired response is to prevent the fault from becoming a failure, but other responses may also be important, such as notifying people or logging the fault for later analysis. This section specifies the desired system response.	Prevent the fault from becoming a failure Detect the fault: ▪ Log the fault ▪ Notify the appropriate entities (people or systems) ▪ Recover from the fault ▪ Disable the source of events causing the fault ▪ Be temporarily unavailable while a repair is being effected ▪ Fix or mask the fault/failure or contain the damage it causes ▪ Operate in a degraded mode while a repair is being effected
Response measure	We may focus on a number of measures of availability, depending on the criticality of the service being provided.	▪ Time or time interval when the system must be available ▪ Availability percentage (e.g., 99.999 percent) ▪ Time to detect the fault ▪ Time to repair the fault ▪ Time or time interval in which system can be in degraded mode ▪ Proportion (e.g., 99 percent) or rate (e.g., up to 100 per second) of a certain class of faults that the system prevents, or handles without failing

An example concrete availability scenario derived from the general scenario in Table 4.2 is shown in Figure 4.1. The scenario is this: *A server in a server farm fails during normal operation, and the system informs the operator and continues to operate with no downtime.*

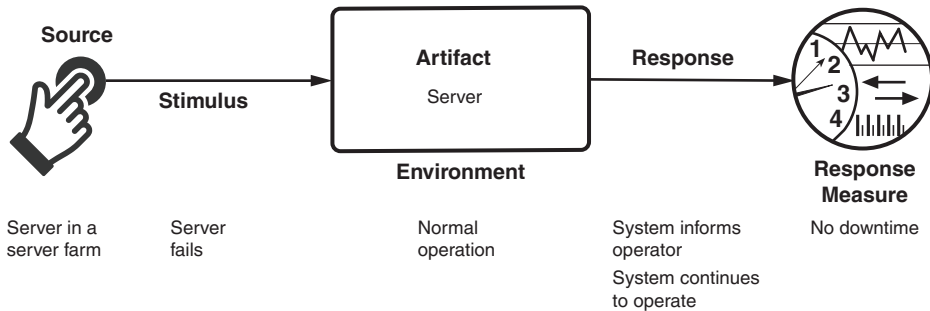


FIGURE 4.1 Sample concrete availability scenario

4.2 Tactics for Availability

A failure occurs when the system no longer delivers a service that is consistent with its specification and this failure is observable by the system's actors. A fault (or combination of faults) has the potential to cause a failure. Availability tactics, in turn, are designed to enable a system to prevent or endure system faults so that a service being delivered by the system remains compliant with its specification. The tactics we discuss in this section will keep faults from becoming failures or at least bound the effects of the fault and make repair possible, as illustrated in Figure 4.2.

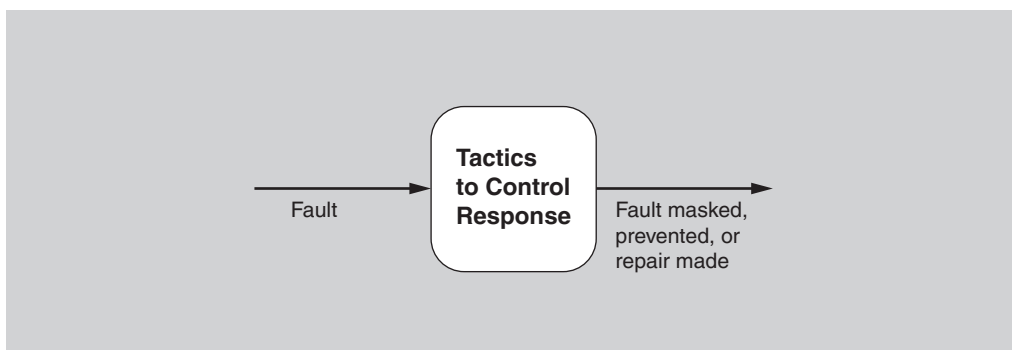


FIGURE 4.2 Goal of availability tactics

Availability tactics have one of three purposes: fault detection, fault recovery, or fault prevention. The tactics for availability are shown in Figure 4.3. These tactics will often be provided by a software infrastructure, such as a middleware package, so your job as an architect may be choosing and assessing (rather than implementing) the right availability tactics and the right combination of tactics.

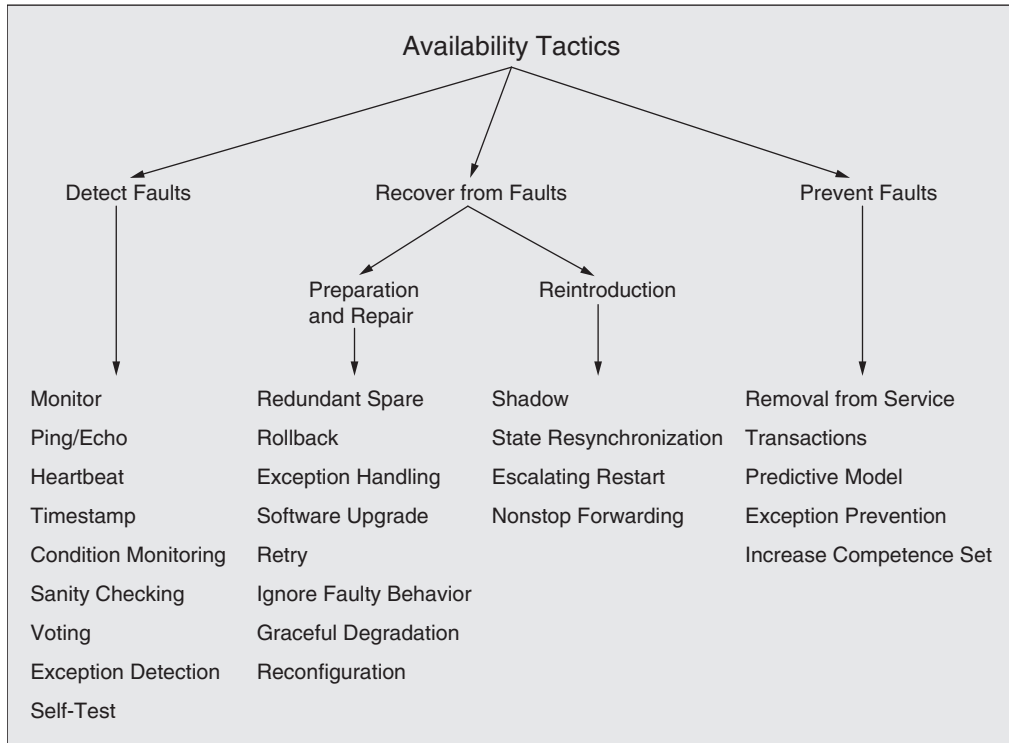


FIGURE 4.3 Availability tactics

Detect Faults

Before any system can take action regarding a fault, the presence of the fault must be detected or anticipated. Tactics in this category include:

- *Monitor.* This component is used to monitor the state of health of various other parts of the system: processors, processes, I/O, memory, and so forth. A system monitor can

detect failure or congestion in the network or other shared resources, such as from a denial-of-service attack. It orchestrates software using other tactics in this category to detect malfunctioning components. For example, the system monitor can initiate *self-tests*, or be the component that detects faulty *timestamps* or missed *heartbeats*.¹

- *Ping/echo*. In this tactic, an asynchronous request/response message pair is exchanged between nodes; it is used to determine reachability and the round-trip delay through the associated network path. In addition, the echo indicates that the pinged component is alive. The ping is often sent by a system monitor. Ping/echo requires a time threshold to be set; this threshold tells the pinging component how long to wait for the echo before considering the pinged component to have failed (“timed out”). Standard implementations of ping/echo are available for nodes interconnected via Internet Protocol (IP).
- *Heartbeat*. This fault detection mechanism employs a periodic message exchange between a system monitor and a process being monitored. A special case of heartbeat is when the process being monitored periodically resets the watchdog timer in its monitor to prevent it from expiring and thus signaling a fault. For systems where scalability is a concern, transport and processing overhead can be reduced by piggybacking heartbeat messages onto other control messages being exchanged. The difference between heartbeat and ping/echo lies in who holds the responsibility for initiating the health check—the monitor or the component itself.
- *Timestamp*. This tactic is used to detect incorrect sequences of events, primarily in distributed message-passing systems. A timestamp of an event can be established by assigning the state of a local clock to the event immediately after the event occurs. Sequence numbers can also be used for this purpose, since timestamps in a distributed system may be inconsistent across different processors. See Chapter 17 for a fuller discussion of the topic of time in a distributed system.
- *Condition monitoring*. This tactic involves checking conditions in a process or device, or validating assumptions made during the design. By monitoring conditions, this tactic prevents a system from producing faulty behavior. The computation of checksums is a common example of this tactic. However, the monitor must itself be simple (and, ideally, provably correct) to ensure that it does not introduce new software errors.
- *Sanity checking*. This tactic checks the validity or reasonableness of specific operations or outputs of a component. It is typically based on a knowledge of the internal design, the state of the system, or the nature of the information under scrutiny. It is most often employed at interfaces, to examine a specific information flow.
- *Voting*. Voting involves comparing computational results from multiple sources that should be producing the same results and, if they are not, deciding which results to use. This tactic depends critically on the voting logic, which is usually realized as a simple, rigorously reviewed, and tested singleton so that the probability of error is low. Voting

1. When the detection mechanism is implemented using a counter or timer that is periodically reset, this specialization of the system monitor is referred to as a *watchdog*. During nominal operation, the process being monitored will periodically reset the watchdog counter/timer as part of its signal that it’s working correctly; this is sometimes referred to as “petting the watchdog.”

also depends critically on having multiple sources to evaluate. Typical schemes include the following:

- *Replication* is the simplest form of voting; here, the components are exact clones of each other. Having multiple copies of identical components can be effective in protecting against random failures of hardware but cannot protect against design or implementation errors, in hardware or software, since there is no form of diversity embedded in this tactic.
- *Functional redundancy*, in contrast, is intended to address the issue of common-mode failures (where replicas exhibit the same fault at the same time because they share the same implementation) in hardware or software components, by implementing design diversity. This tactic attempts to deal with the systematic nature of design faults by adding diversity to redundancy. The outputs of functionally redundant components should be the same given the same input. The functional redundancy tactic is still vulnerable to specification errors—and, of course, functional replicas will be more expensive to develop and verify.
- *Analytic redundancy* permits not only diversity among components' private sides, but also diversity among the components' inputs and outputs. This tactic is intended to tolerate specification errors by using separate requirement specifications. In embedded systems, analytic redundancy helps when some input sources are likely to be unavailable at times. For example, avionics programs have multiple ways to compute aircraft altitude, such as using barometric pressure, with the radar altimeter, and geometrically using the straight-line distance and look-down angle of a point ahead on the ground. The voter mechanism used with analytic redundancy needs to be more sophisticated than just letting majority rule or computing a simple average. It may have to understand which sensors are currently reliable (or not), and it may be asked to produce a higher-fidelity value than any individual component can, by blending and smoothing individual values over time.
- *Exception detection*. This tactic focuses on the detection of a system condition that alters the normal flow of execution. It can be further refined as follows:
 - *System exceptions* will vary according to the processor hardware architecture employed. They include faults such as divide by zero, bus and address faults, illegal program instructions, and so forth.
 - The *parameter fence* tactic incorporates a known data pattern (such as 0xDEADBEEF) placed immediately after any variable-length parameters of an object. This allows for runtime detection of overwriting the memory allocated for the object's variable-length parameters.
 - *Parameter typing* employs a base class that defines functions that add, find, and iterate over type-length-value (TLV) formatted message parameters. Derived classes use the base class functions to provide functions to build and parse messages. Use of parameter typing ensures that the sender and the receiver of messages agree on the type of the content, and detects cases where they don't.
 - *Timeout* is a tactic that raises an exception when a component detects that it or another component has failed to meet its timing constraints. For example, a component

awaiting a response from another component can raise an exception if the wait time exceeds a certain value.

- *Self-test.* Components (or, more likely, whole subsystems) can run procedures to test themselves for correct operation. Self-test procedures can be initiated by the component itself or invoked from time to time by a system monitor. These may involve employing some of the techniques found in condition monitoring, such as checksums.

Recover from Faults

Recover from faults tactics are refined into preparation and repair tactics and reintroduction tactics. The latter are concerned with reintroducing a failed (but rehabilitated) component back into normal operation.

Preparation and repair tactics are based on a variety of combinations of retrying a computation or introducing redundancy:

- *Redundant spare.* This tactic refers to a configuration in which one or more duplicate components can step in and take over the work if the primary component fails. This tactic is at the heart of the hot spare, warm spare, and cold spare patterns, which differ primarily in how up-to-date the backup component is at the time of its takeover.
- *Rollback.* A rollback permits the system to revert to a previous known good state (referred to as the “rollback line”)—rolling back time—upon the detection of a failure. Once the good state is reached, then execution can continue. This tactic is often combined with the transactions tactic and the redundant spare tactic so that after a rollback has occurred, a standby version of the failed component is promoted to active status. Rollback depends on a copy of a previous good state (a checkpoint) being available to the components that are rolling back. Checkpoints can be stored in a fixed location and updated at regular intervals, or at convenient or significant times in the processing, such as at the completion of a complex operation.
- *Exception handling.* Once an exception has been detected, the system will handle it in some fashion. The easiest thing it can do is simply to crash—but, of course, that’s a terrible idea from the point of availability, usability, testability, and plain good sense. There are much more productive possibilities. The mechanism employed for exception handling depends largely on the programming environment employed, ranging from simple function return codes (error codes) to the use of exception classes that contain information helpful in fault correlation, such as the name of the exception, the origin of the exception, and the cause of the exception. Software can then use this information to mask or repair the fault.
- *Software upgrade.* The goal of this tactic is to achieve in-service upgrades to executable code images in a non-service-affecting manner. Strategies include the following:
 - *Function patch.* This kind of patch, which is used in procedural programming, employs an incremental linker/loader to store an updated software function into a pre-allocated segment of target memory. The new version of the software function will employ the entry and exit points of the deprecated function.

- *Class patch.* This kind of upgrade is applicable for targets executing object-oriented code, where the class definitions include a backdoor mechanism that enables the run-time addition of member data and functions.
- *Hitless in-service software upgrade (ISSU).* This leverages the redundant spare tactic to achieve non-service-affecting upgrades to software and associated schema.

In practice, the function patch and class patch are used to deliver bug fixes, while the hitless ISSU is used to deliver new features and capabilities.

- *Retry.* The retry tactic assumes that the fault that caused a failure is transient, and that retrying the operation may lead to success. It is used in networks and in server farms where failures are expected and common. A limit should be placed on the number of retries that are attempted before a permanent failure is declared.
- *Ignore faulty behavior.* This tactic calls for ignoring messages sent from a particular source when we determine that those messages are spurious. For example, we would like to ignore the messages emanating from the live failure of a sensor.
- *Graceful degradation.* This tactic maintains the most critical system functions in the presence of component failures, while dropping less critical functions. This is done in circumstances where individual component failures gracefully reduce system functionality, rather than causing a complete system failure.
- *Reconfiguration.* Reconfiguration attempts to recover from failures by reassigning responsibilities to the (potentially restricted) resources or components left functioning, while maintaining as much functionality as possible.

Reintroduction occurs when a failed component is reintroduced after it has been repaired. Reintroduction tactics include the following:

- *Shadow.* This tactic refers to operating a previously failed or in-service upgraded component in a “shadow mode” for a predefined duration of time prior to reverting the component back to an active role. During this duration, its behavior can be monitored for correctness and it can repopulate its state incrementally.
- *State resynchronization.* This reintroduction tactic is a partner to the redundant spare tactic. When used with active redundancy—a version of the redundant spare tactic—the state resynchronization occurs organically, since the active and standby components each receive and process identical inputs in parallel. In practice, the states of the active and standby components are periodically compared to ensure synchronization. This comparison may be based on a cyclic redundancy check calculation (checksum) or, for systems providing safety-critical services, a message digest calculation (a one-way hash function). When used alongside the passive redundancy version of the redundant spare tactic, state resynchronization is based solely on periodic state information transmitted from the active component(s) to the standby component(s), typically via checkpointing.
- *Escalating restart.* This reintroduction tactic allows the system to recover from faults by varying the granularity of the component(s) restarted and minimizing the level of service affectation. For example, consider a system that supports four levels of restart, numbered 0–3. The lowest level of restart (Level 0) has the least impact on services and

employs passive redundancy (warm spare), where all child threads of the faulty component are killed and recreated. In this way, only data associated with the child threads is freed and reinitialized. The next level of restart (Level 1) frees and reinitializes all unprotected memory; protected memory is untouched. The next level of restart (Level 2) frees and reinitializes all memory, both protected and unprotected, forcing all applications to reload and reinitialize. The final level of restart (Level 3) involves completely reloading and reinitializing the executable image and associated data segments. Support for the escalating restart tactic is particularly useful for the concept of graceful degradation, where a system is able to degrade the services it provides while maintaining support for mission-critical or safety-critical applications.

- *Nonstop forwarding.* This concept originated in router design, and assumes that functionality is split into two parts: the supervisory or control plane (which manages connectivity and routing information) and the data plane (which does the actual work of routing packets from sender to receiver). If a router experiences the failure of an active supervisor, it can continue forwarding packets along known routes—with neighboring routers—while the routing protocol information is recovered and validated. When the control plane is restarted, it implements a “graceful restart,” incrementally rebuilding its routing protocol database even as the data plane continues to operate.

Prevent Faults

Instead of detecting faults and then trying to recover from them, what if your system could prevent them from occurring in the first place? Although it might sound as if some measure of clairvoyance would be required, it turns out that in many cases it is possible to do just that.²

- *Removal from service.* This tactic refers to temporarily placing a system component in an out-of-service state for the purpose of mitigating potential system failures. For example, a component of a system might be taken out of service and reset to scrub latent faults (such as memory leaks, fragmentation, or soft errors in an unprotected cache) before the accumulation of faults reaches the service-affecting level, resulting in system failure. Other terms for this tactic are *software rejuvenation* and *therapeutic reboot*. If you reboot your computer every night, you are practicing removal from service.
- *Transactions.* Systems targeting high-availability services leverage transactional semantics to ensure that asynchronous messages exchanged between distributed components are atomic, consistent, isolated, and durable—properties collectively referred to as the “ACID properties.” The most common realization of the transactions tactic is the “two-phase commit” (2PC) protocol. This tactic prevents race conditions caused by two processes attempting to update the same data item at the same time.

2. These tactics deal with runtime means to prevent faults from occurring. Of course, an excellent way to prevent faults—at least in the system you’re building, if not in systems that your system must interact with—is to produce high-quality code. This can be done by means of code inspections, pair programming, solid requirements reviews, and a host of other good engineering practices.

- *Predictive model.* A predictive model, when combined with a monitor, is employed to monitor the state of health of a system process to ensure that the system is operating within its nominal operating parameters, and to take corrective action when the system nears a critical threshold. The operational performance metrics monitored are used to predict the onset of faults; examples include the session establishment rate (in an HTTP server), threshold crossing (monitoring high and low watermarks for some constrained, shared resource), statistics on the process state (e.g., in-service, out-of-service, under maintenance, idle), and message queue length statistics.
- *Exception prevention.* This tactic refers to techniques employed for the purpose of preventing system exceptions from occurring. The use of exception classes, which allows a system to transparently recover from system exceptions, was discussed earlier. Other examples of exception prevention include error-correcting code (used in telecommunications), abstract data types such as smart pointers, and the use of wrappers to prevent faults such as dangling pointers or semaphore access violations. Smart pointers prevent exceptions by doing bounds checking on pointers, and by ensuring that resources are automatically de-allocated when no data refers to them, thereby avoiding resource leaks.
- *Increase competence set.* A program's competence set is the set of states in which it is "competent" to operate. For example, the state when the denominator is zero is outside the competence set of most divide programs. When a component raises an exception, it is signaling that it has discovered itself to be outside its competence set; in essence, it doesn't know what to do and is throwing in the towel. Increasing a component's competence set means designing it to handle more cases—faults—as part of its normal operation. For example, a component that assumes it has access to a shared resource might throw an exception if it discovers that access is blocked. Another component might simply wait for access or return immediately with an indication that it will complete its operation on its own the next time it does have access. In this example, the second component has a larger competence set than the first.

4.3 Tactics-Based Questionnaire for Availability

Based on the tactics described in Section 4.2, we can create a set of availability tactics—inspired questions, as presented in Table 4.3. To gain an overview of the architectural choices made to support availability, the analyst asks each question and records the answers in the table. The answers to these questions can then be made the focus of further activities: investigation of documentation, analysis of code or other artifacts, reverse engineering of code, and so forth.

TABLE 4.3 Tactics-Based Questionnaire for Availability

Tactics Group	Tactics Question	Support? (Y/N)	Risk	Design Decisions and Location	Rationale and Assumptions
Detect Faults	Does the system use ping/echo to detect failure of a component or connection, or network congestion? Does the system use a component to monitor the state of health of other parts of the system? A system monitor can detect failure or congestion in the network or other shared resources, such as from a denial-of-service attack. Does the system use a heartbeat—a periodic message exchange between a system monitor and a process—to detect failure of a component or connection, or network congestion? Does the system use a timestamp to detect incorrect sequences of events in distributed systems? Does the system use voting to check that replicated components are producing the same results? The replicated components may be identical replicas, functionally redundant, or analytically redundant. Does the system use exception detection to detect a system condition that alters the normal flow of execution (e.g., system exception, parameter fence, parameter typing, timeout)? Can the system do a self-test to test itself for correct operation?				

continues

TABLE 4.3 Tactics-Based Questionnaire for Availability *continued*

Tactics Group	Tactics Question	Support? (Y/N)	Risk	Design Decisions and Location	Rationale and Assumptions
Recover from Faults (Preparation and Repair)	Does the system employ redundant spares? Is a component's role as active versus spare fixed, or does it change in the presence of a fault? What is the switchover mechanism? What is the trigger for a switchover? How long does it take for a spare to assume its duties? Does the system employ exception handling to deal with faults? Typically the handling involves either reporting, correcting, or masking the fault. Does the system employ rollback, so that it can revert to a previously saved good state (the "rollback line") in the event of a fault? Can the system perform in-service software upgrades to executable code images in a non-service-affecting manner? Does the system systematically retry in cases where the component or connection failure may be transient? Can the system simply ignore faulty behavior (e.g., ignore messages when it is determined that those messages are spurious)? Does the system have a policy of degradation when resources are compromised, maintaining the most critical system functions in the presence of component failures, and dropping less critical functions? Does the system have consistent policies and mechanisms for reconfiguration after failures, reassigning responsibilities to the resources left functioning, while maintaining as much functionality as possible?				

Tactics Group	Tactics Question	Support? (Y/N)	Risk	Design Decisions and Location	Rationale and Assumptions
Recover from Faults (Reintroduction)	Can the system operate a previously failed or in-service upgraded component in a “shadow mode” for a predefined time prior to reverting the component back to an active role? If the system uses active or passive redundancy, does it also employ state resynchronization to send state information from active components to standby components? Does the system employ escalating restart to recover from faults by varying the granularity of the component(s) restarted and minimizing the level of service affected? Can message processing and routing portions of the system employ nonstop forwarding, where functionality is split into supervisory and data planes?				
Prevent Faults	Can the system remove components from service, temporarily placing a system component in an out-of-service state for the purpose of preempting potential system failures? Does the system employ transactions—bundling state updates so that asynchronous messages exchanged between distributed components are <i>atomic, consistent, isolated, and durable</i>? Does the system use a predictive model to monitor the state of health of a component to ensure that the system is operating within nominal parameters? When conditions are detected that are predictive of likely future faults, the model initiates corrective action.				

4.4 Patterns for Availability

This section presents a few of the most important architectural patterns for availability.

The first three patterns are all centered on the redundant spare tactic, and will be described as a group. They differ primarily in the degree to which the backup components' state matches that of the active component. (A special case occurs when the components are stateless, in which case the first two patterns become identical.)

- *Active redundancy (hot spare)*. For stateful components, this refers to a configuration in which all of the nodes (active or redundant spare) in a protection group³ receive and process identical inputs in parallel, allowing the redundant spare(s) to maintain a synchronous state with the active node(s). Because the redundant spare possesses an identical state to the active processor, it can take over from a failed component in a matter of milliseconds. The simple case of one active node and one redundant spare node is commonly referred to as one-plus-one redundancy. Active redundancy can also be used for facilities protection, where active and standby network links are used to ensure highly available network connectivity.
- *Passive redundancy (warm spare)*. For stateful components, this refers to a configuration in which only the active members of the protection group process input traffic. One of their duties is to provide the redundant spare(s) with periodic state updates. Because the state maintained by the redundant spares is only loosely coupled with that of the active node(s) in the protection group (with the looseness of the coupling being a function of the period of the state updates), the redundant nodes are referred to as warm spares. Passive redundancy provides a solution that achieves a balance between the more highly available but more compute-intensive (and expensive) active redundancy pattern and the less available but significantly less complex cold spare pattern (which is also significantly cheaper).
- *Spare (cold spare)*. Cold sparing refers to a configuration in which redundant spares remain out of service until a failover occurs, at which point a power-on-reset⁴ procedure is initiated on the redundant spare prior to its being placed in service. Due to its poor recovery performance, and hence its high mean time to repair, this pattern is poorly suited to systems having high-availability requirements.

Benefits:

- The benefit of a redundant spare is a system that continues to function correctly after only a brief delay in the presence of a failure. The alternative is a system that stops functioning correctly, or stops functioning altogether, until the failed component is repaired. This repair could take hours or days.

3. A protection group is a group of processing nodes in which one or more nodes are "active," with the remaining nodes serving as redundant spares.

4. A power-on-reset ensures that a device starts operating in a known state.

Tradeoffs:

- The tradeoff with any of these patterns is the additional cost and complexity incurred in providing a spare.
- The tradeoff among the three alternatives is the time to recover from a failure versus the runtime cost incurred to keep a spare up-to-date. A hot spare carries the highest cost but leads to the fastest recovery time, for example.

Other patterns for availability include the following.

- *Triple modular redundancy (TMR)*. This widely used implementation of the voting tactic employs three components that do the same thing. Each component receives identical inputs and forwards its output to the voting logic, which detects any inconsistency among the three output states. Faced with an inconsistency, the voter reports a fault. It must also decide which output to use, and different instantiations of this pattern use different decision rules. Typical choices are letting the majority rule or choosing some computed average of the disparate outputs.

Of course, other versions of this pattern that employ 5 or 19 or 53 redundant components are also possible. However, in most cases, 3 components are sufficient to ensure a reliable result.

Benefits:

- TMR is simple to understand and to implement. It is blissfully independent of what might be causing disparate results, and is only concerned about making a reasonable choice so that the system can continue to function.

Tradeoffs:

- There is a tradeoff between increasing the level of replication, which raises the cost, and the resulting availability. In systems employing TMR, the statistical likelihood of two or more components failing is vanishingly small, and three components represents a sweet spot between availability and cost.
- *Circuit breaker*. A commonly used availability tactic is retry. In the event of a timeout or fault when invoking a service, the invoker simply tries again—and again, and again. A circuit breaker keeps the invoker from trying countless times, waiting for a response that never comes. In this way, it breaks the endless retry cycle when it deems that the system is dealing with a fault. That’s the signal for the system to begin handling the fault. Until the circuit break is “reset,” subsequent invocations will return immediately without passing along the service request.

Benefits:

- This pattern can remove from individual components the policy about how many retries to allow before declaring a failure.
- At worst, endless fruitless retries would make the invoking component as useless as the invoked component that has failed. This problem is especially acute in distributed systems, where you could have many callers calling an unresponsive component and effectively going out of service themselves, causing the failure to cascade across the

whole system. The circuit breaker, *in conjunction with software that listens to it and begins recovery procedures*, prevents that problem.

Tradeoffs:

- Care must be taken in choosing timeout (or retry) values. If the timeout is too long, then unnecessary latency is added. But if the timeout is too short, then the circuit breaker will be tripping when it does not need to—a kind of “false positive”—which can lower the availability and performance of these services.

Other availability patterns that are commonly used include the following:

- *Process pairs*. This pattern employs checkpointing and rollback. In case of failure, the backup has been checkpointing and (if necessary) rolling back to a safe state, so is ready to take over when a failure occurs.
- *Forward error recovery*. This pattern provides a way to get out of an undesirable state by *moving forward* to a desirable state. This often relies upon built-in error-correction capabilities, such as data redundancy, so that errors may be corrected without the need to fall back to a previous state or to retry. Forward error recovery finds a safe, possibly degraded state from which the operation can move forward.

4.5 For Further Reading

Patterns for availability:

- You can read about patterns for fault tolerance in [Hanmer 13].

General tactics for availability:

- A more detailed discussion of some of the availability tactics in this chapter is given in [Scott 09]. This is the source of much of the material in this chapter.
- The Internet Engineering Task Force has promulgated a number of standards supporting availability tactics. These standards include *Non-Stop Forwarding* [IETF 2004], *Ping/Echo (ICMP)* [IETF 1981] or *ICMPv6* [RFC 2006b] *Echo Request/Response*, and MPLS (LSP Ping) networks [IETF 2006a].

Tactics for availability—fault detection:

- Triple modular redundancy (TMR) was developed in the early 1960s by Lyons [Lyons 62].
- The fault detection in the voting tactic is based on the fundamental contributions to automata theory by Von Neumann, who demonstrated how systems having a prescribed reliability could be built from unreliable components [Von Neumann 56].

Tactics for availability—fault recovery:

- Standards-based realizations of active redundancy exist for protecting network links (i.e., facilities) at both the physical layer of the seven-layer OSI (Open Systems

Interconnection) model [Bellcore 98, 99; Telcordia 00] and the network/link layer [IETF 2005].

- Some examples of how a system can degrade through use (degradation) are given in [Nygard 18].
- Mountains of papers have been written about parameter typing, but [Utas 05] writes about it in the context of availability (as opposed to bug prevention, its usual context). [Utas 05] has also written about escalating restart.
- Hardware engineers often use preparation and repair tactics. Examples include error detection and correction (EDAC) coding, forward error correction (FEC), and temporal redundancy. EDAC coding is typically used to protect control memory structures in high-availability distributed real-time embedded systems [Hamming 80]. Conversely, FEC coding is typically employed to recover from physical layer errors occurring in external network links [Morelos-Zaragoza 06]. Temporal redundancy involves sampling spatially redundant clock or data lines at time intervals that exceed the pulse width of any transient pulse to be tolerated, and then voting out any defects detected [Mavis 02].

Tactics for availability—fault prevention:

- Parnas and Madey have written about increasing an element's competence set [Parnas 95].
- The ACID properties, important in the transactions tactic, were introduced by Gray in the 1970s and discussed in depth in [Gray 93].

Disaster recovery:

- A disaster is an event such as an earthquake, flood, or hurricane that destroys an entire data center. The U.S. National Institute of Standards and Technology (NIST) identifies eight different types of plans that should be considered in the event of a disaster. See Section 2.2 of NIST Special Publication 800-34, *Contingency Planning Guide for Federal Information Systems*, <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-34r1.pdf>.

4.6 Discussion Questions

1. Write a set of concrete scenarios for availability using each of the possible responses in the general scenario.
2. Write a concrete availability scenario for the software for a (hypothetical) driverless car.
3. Write a concrete availability scenario for a program like Microsoft Word.
4. Redundancy is a key strategy for achieving high availability. Look at the patterns and tactics presented in this chapter and decide how many of them exploit some form of redundancy and how many do not.
5. How does availability trade off against modifiability and deployability? How would you make a change to a system that is required to have 24/7 availability (i.e., no scheduled or unscheduled down time, ever)?

6. Consider the fault detection tactics (ping/echo, heartbeat, system monitor, voting, and exception detection). What are the performance implications of using these tactics?
7. Which tactics are used by a load balancer (see Chapter 17) when it detects a failure of an instance?
8. Look up recovery point objective (RPO) and recovery time objective (RTO), and explain how these can be used to set a checkpoint interval when using the rollback tactic.

5



Deployability

*From the day we arrive on the planet
And blinking, step into the sun
There's more to be seen than can ever be seen
More to do than can ever be done
—The Lion King*

There comes a day when software, like the rest of us, must leave home and venture out into the world and experience real life. Unlike the rest of us, software typically makes the trip many times, as changes and updates are made. This chapter is about making that transition as orderly and as effective and—most of all—as *rapid* as possible. That is the realm of continuous deployment, which is most enabled by the quality attribute of deployability.

Why has deployability come to take a front-row seat in the world of quality attributes?

In the “bad old days,” releases were infrequent—large numbers of changes were bundled into releases and scheduled. A release would contain new features and bug fixes. One release per month, per quarter, or even per year was common. Competitive pressures in many domains—with the charge being led by e-commerce—resulted in a need for much shorter release cycles. In these contexts, releases can occur at any time—possibly hundreds of releases per day—and each can be instigated by a different team within an organization. Being able to release frequently means that bug fixes in particular do not have to wait until the next scheduled release, but rather can be made and released as soon as a bug is discovered and fixed. It also means that new features do not need to be bundled into a release, but can be put into production at any time.

This is not desirable, or even possible, in all domains. If your software exists in a complex ecosystem with many dependencies, it may not be possible to release just one part of it without coordinating that release with the other parts. In addition, many embedded systems, systems in hard-to-access locations, and systems that are not networked would be poor candidates for a continuous deployment mindset.

This chapter focuses on the large and growing numbers of systems for which just-in-time feature releases are a significant competitive advantage, and just-in-time bug fixes are essential to safety or security or continuous operation. Often these systems are microservice and cloud-based, although the techniques here are not limited to those technologies.

5.1 Continuous Deployment

Deployment is a process that starts with coding and ends with real users interacting with the system in a production environment. If this process is fully automated—that is, if there is no human intervention—then it is called *continuous deployment*. If the process is automated up to the point of placing (portions of) the system into production and human intervention is required (perhaps due to regulations or policies) for this final step, the process is called *continuous delivery*.

To speed up releases, we need to introduce the concept of a *deployment pipeline*: the sequence of tools and activities that begin when you check your code into a version control system and end when your application has been deployed for users to send it requests. In between those points, a series of tools integrate and automatically test the newly committed code, test the integrated code for functionality, and test the application for concerns such as performance under load, security, and license compliance.

Each stage in the deployment pipeline takes place in an environment established to support isolation of the stage and perform the actions appropriate to that stage. The major environments are as follows:

- Code is developed in a *development environment* for a single module where it is subject to standalone unit tests. Once it passes the tests, and after appropriate review, the code is committed to a version control system that triggers the build activities in the integration environment.
- An *integration environment* builds an executable version of your service. A continuous integration server compiles¹ your new or changed code, along with the latest compatible versions of code for other portions of your service and constructs an executable image for your service.² Tests in the integration environment include the unit tests from the various modules (now run against the built system), as well as integration tests designed specifically for the whole system. When the various tests are passed, the built service is promoted to the staging environment.
- A *staging environment* tests for various qualities of the total system. These include performance testing, security testing, license conformance checks, and possibly user testing. For embedded systems, this is where simulators of the physical environment (feeding synthetic inputs to the system) are brought to bear. An application that passes all staging environment tests—which may include field testing—is deployed to the production environment, using either a blue/green model or a rolling upgrade (see Section 5.6). In some cases, partial deployments are used for quality control or to test the market response to a proposed change or offering.
- Once in the *production environment*, the service is monitored closely until all parties have some level of confidence in its quality. At that point, it is considered a normal part of the system and receives the same amount of attention as the other parts of the system.

1. If you are developing software using an interpreted language such as Python or JavaScript, there is no compilation step.

2. In this chapter, we use the term “service” to denote any independently deployable unit.

You perform a different set of tests in each environment, expanding the testing scope from unit testing of a single module in the development environment, to functional testing of all the components that make up your service in the integration environment, and ending with broad quality testing in the staging environment and usage monitoring in the production environment.

But not everything always goes according to plan. If you find problems after the software is in its production environment, it is often necessary to roll back to a previous version while the defect is being addressed.

Architectural choices affect deployability. For example, by employing the microservice architecture pattern (see Section 5.6), each team responsible for a microservice can make its own technology choices; this removes incompatibility problems that would previously have been discovered at integration time (e.g., incompatible choices of which version of a library to use). Since microservices are independent services, such choices do not cause problems.

Similarly, a continuous deployment mindset forces you to think about the testing infrastructure earlier in the development process. This is necessary because designing for continuous deployment requires continuous automated testing. In addition, the need to be able to roll back or disable features leads to architectural decisions about mechanisms such as feature toggles and backward compatibility of interfaces. These decisions are best taken early on.

The Effect of Virtualization on the Different Environments

Before the widespread use of virtualization technology, the environments that we describe here were physical facilities. In most organizations, the development, integration, and staging environments comprised hardware and software procured and operated by different groups. The development environment might consist of a few desktop computers that the development team repurposed as servers. The integration environment was operated by the test or quality-assurance team, and might consist of some racks, populated with previous-generation equipment from the data center. The staging environment was operated by the operations team and might have hardware similar to that used in production.

A lot of time was spent trying to figure out why a test that passed in one environment failed in another environment. One benefit of environments that employ virtualization is the ability to have *environment parity*, where environments may differ in scale but not in type of hardware or fundamental structure. A variety of provisioning tools support environment parity by allowing every team to easily build a common environment and by ensuring that this common environment mimics the production environment as closely as possible.

Three important ways to measure the quality of the pipeline are as follows:

- *Cycle time* is the pace of progress through the pipeline. Many organizations will deploy to production several or even hundreds of times a day. Such rapid deployment is not possible if human intervention is required. It is also not possible if one team must coordinate

with other teams before placing its service in production. Later in this chapter, we will see architectural techniques that allow teams to perform continuous deployment without consulting other teams.

- *Traceability* is the ability to recover all of the artifacts that led to an element having a problem. That includes all the code and dependencies that are included in that element. It also includes the test cases that were run on that element and the tools that were used to produce the element. Errors in tools used in the deployment pipeline can cause problems in production. Typically, traceability information is kept in an *artifact database*. This database will contain code version numbers, version numbers of elements the system depends on (such as libraries), test version numbers, and tool version numbers.
- *Repeatability* is getting the same result when you perform the same action with the same artifacts. This is not as easy as it sounds. For example, suppose your build process fetches the latest version of a library. The next time you execute the build process, a new version of the library may have been released. As another example, suppose one test modifies some values in the database. If the original values are not restored, subsequent tests may not produce the same results.

DevOps

DevOps—a portmanteau of “development” and “operations”—is a concept closely associated with continuous deployment. It is a movement (much like the Agile movement), a description of a set of practices and tools (again, much like the Agile movement), and a marketing formula touted by vendors selling those tools. The goal of DevOps is to shorten time to market (or time to release). The goal is to dramatically shorten the time between a developer making a change to an existing system—implementing a feature or fixing a bug—and the system reaching the hands of end users, as compared with traditional software development practices.

A formal definition of DevOps captures both the frequency of releases and the ability to perform bug fixes on demand:

DevOps is a set of practices intended to reduce the time between committing a change to a system and the change being placed into normal production, while ensuring high quality. [Bass 15]

Implementing DevOps is a process improvement effort. DevOps encompasses not only the cultural and organizational elements of any process improvement effort, but also a strong reliance on tools and architectural design. All environments are different, of course, but the tools and automation we describe are found in the typical tool chains built to support DevOps.

The continuous deployment strategy we describe here is the conceptual heart of DevOps. Automated testing is, in turn, a critically important ingredient of continuous deployment, and the tooling for that often represents the highest technological hurdle for DevOps. Some forms of DevOps include logging and post-deployment monitoring of those logs, for automatic detection of errors back at the “home office,” or even monitoring to understand the user experience. This, of course, requires a “phone home” or log delivery capability in the system, which may or may not be possible or allowable in some systems.

DevSecOps is a flavor of DevOps that incorporates approaches for security (for the infrastructure and for the applications it produces) into the entire process. DevSecOps is increasingly popular in aerospace and defense applications, but is also valid in any application area where DevOps is useful and a security breach would be particularly costly. Many IT applications fall in this category.

5.2 Deployability

Deployability refers to a property of software indicating that it may be deployed—that is, allocated to an environment for execution—within a predictable and acceptable amount of time and effort. Moreover, if the new deployment is not meeting its specifications, it may be rolled back, again within a predictable and acceptable amount of time and effort. As the world moves increasingly toward virtualization and cloud infrastructures, and as the scale of deployed software-intensive systems inevitably increases, it is one of the architect’s responsibilities to ensure that deployment is done in an efficient and predictable way, minimizing overall system risk.³

To achieve these goals, an architect needs to consider how an executable is updated on a host platform, and how it is subsequently invoked, measured, monitored, and controlled. Mobile systems in particular present a challenge for deployability in terms of how they are updated because of concerns about bandwidth. Some of the issues involved in deploying software are as follows:

- How does it arrive at its host (i.e., push, where updates deployed are unbidden, or pull, where users or administrators must explicitly request updates)?
- How is it integrated into an existing system? Can this be done while the existing system is executing?
- What is the medium, such as DVD, USB drive, or Internet delivery?
- What is the packaging (e.g., executable, app, plug-in)?
- What is the resulting integration into an existing system?
- What is the efficiency of executing the process?
- What is the controllability of the process?

With all of these concerns, the architect must be able to assess the associated risks. Architects are primarily concerned with the degree to which the architecture supports deployments that are:

- *Granular.* Deployments can be of the whole system or of elements within a system. If the architecture provides options for finer granularity of deployment, then certain risks can be reduced.

3. The quality attribute of testability (see Chapter 12) certainly plays a critical role in continuous deployment, and the architect can provide critical support for continuous deployment by ensuring that the system is testable, in all the ways just mentioned. However, our concern here is the quality attribute directly related to continuous deployment over and above testability: deployability.

- *Controllable.* The architecture should provide the capability to deploy at varying levels of granularity, monitor the operation of the deployed units, and roll back unsuccessful deployments.
- *Efficient.* The architecture should support rapid deployment (and, if needed, rollback) with a reasonable level of effort.

These characteristics will be reflected in the response measures of the general scenario for deployability.

5.3 Deployability General Scenario

Table 5.1 enumerates the elements of the general scenario that characterize deployability.

TABLE 5.1 General Scenario for Deployability

Portion of Scenario	Description	Possible Values
Source	The trigger for the deployment	End user, developer, system administrator, operations personnel, component marketplace, product owner.
Stimulus	What causes the trigger	A new element is available to be deployed. This is typically a request to replace a software element with a new version (e.g., fix a defect, apply a security patch, upgrade to the latest release of a component or framework, upgrade to the latest version of an internally produced element). New element is approved for incorporation. An existing element/set of elements needs to be rolled back.
Artifacts	What is to be changed	Specific components or modules, the system's platform, its user interface, its environment, or another system with which it interoperates. Thus the artifact might be a single software element, multiple software elements, or the entire system.
Environment	Staging, production (or a specific subset of either)	Full deployment. Subset deployment to a specified portion of users, VMs, containers, servers, platforms.
Response	What should happen	Incorporate the new components. Deploy the new components. Monitor the new components. Roll back a previous deployment.

Portion of Scenario	Description	Possible Values
Response measure	A measure of cost, time, or process effectiveness for a deployment, or for a series of deployments over time	Cost in terms of: ▪ Number, size, and complexity of affected artifacts ▪ Average/worst-case effort ▪ Elapsed clock or calendar time ▪ Money (direct outlay or opportunity cost) ▪ New defects introduced Extent to which this deployment/rollback affects other functions or quality attributes. Number of failed deployments. Repeatability of the process. Traceability of the process. Cycle time of the process.

Figure 5.1 illustrates a concrete deployability scenario: “A new release of an authentication/authorization service (which our product uses) is made available in the component marketplace and the product owner decides to incorporate this version into the release. The new service is tested and deployed to the production environment within 40 hours of elapsed time and no more than 120 person-hours of effort. The deployment introduces no defects and no SLA is violated.”

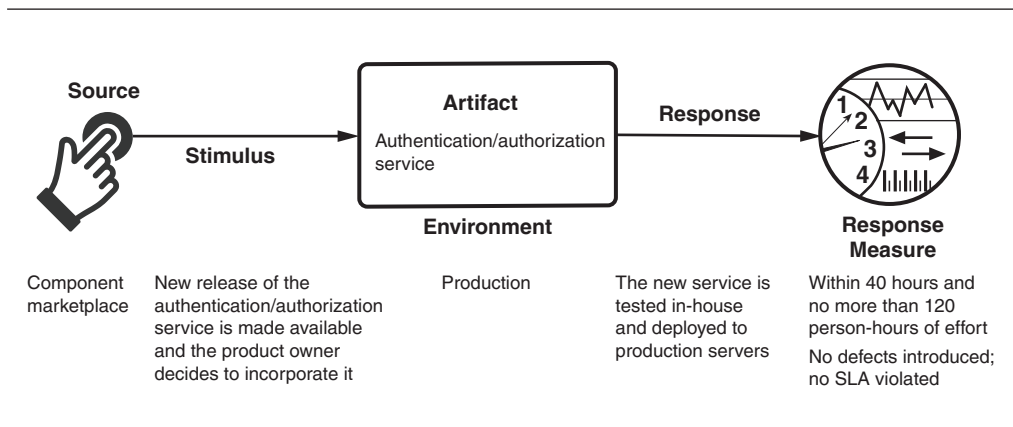


FIGURE 5.1 Sample concrete deployability scenario

5.4 Tactics for Deployability

A deployment is catalyzed by the release of a new software or hardware element. The deployment is successful if these new elements are deployed within acceptable time, cost, and quality constraints. We illustrate this relationship—and hence the goal of deployability tactics—in Figure 5.2.

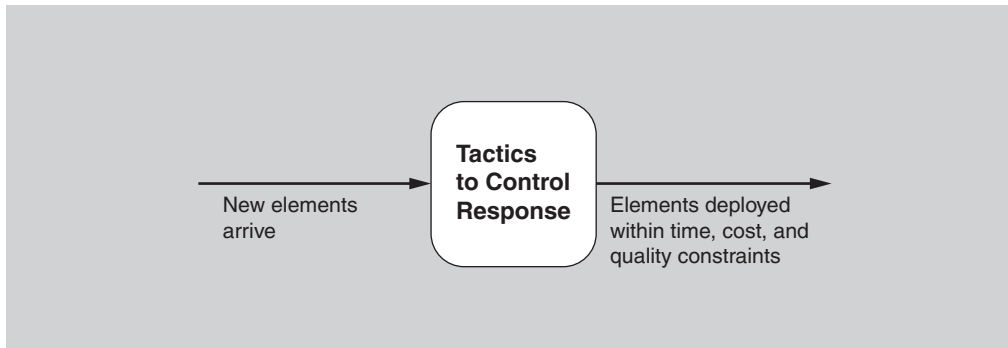


FIGURE 5.2 Goal of deployability tactics

The tactics for deployability are shown in Figure 5.3. In many cases, these tactics will be provided, at least in part, by a CI/CD (continuous integration/continuous deployment) infrastructure that you buy rather than build. In such a case, your job as an architect is often one of choosing and assessing (rather than implementing) the right deployability tactics and the right combination of tactics.

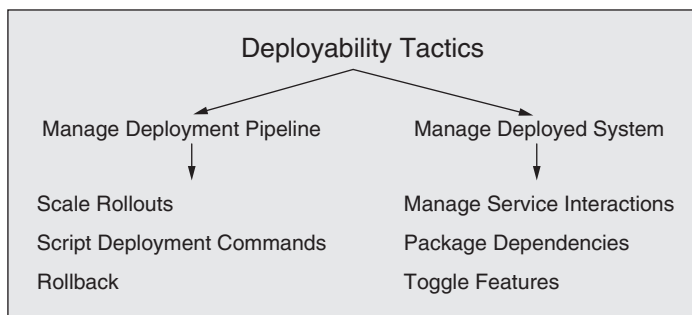


FIGURE 5.3 Deployability tactics

Next, we describe these six deployability tactics in more detail. The first category of deployability tactics focuses on strategies for managing the deployment pipeline, and the second category deals with managing the system as it is being deployed and once it has been deployed.

Manage Deployment Pipeline

- *Scale rollouts.* Rather than deploying to the entire user base, scaled rollouts deploy a new version of a service gradually, to controlled subsets of the user population, often with no explicit notification to those users. (The remainder of the user base continues to use the previous version of the service.) By gradually releasing, the effects of new deployments can be monitored and measured and, if necessary, rolled back. This tactic minimizes the potential negative impact of deploying a flawed service. It requires an architectural mechanism (not part of the service being deployed) to route a request from a user to either the new or old service, depending on that user's identity.
- *Roll back.* If it is discovered that a deployment has defects or does not meet user expectations, then it can be “rolled back” to its prior state. Since deployments may involve multiple coordinated updates of multiple services and their data, the rollback mechanism must be able to keep track of all of these, or must be able to reverse the consequences of any update made by a deployment, ideally in a fully automated fashion.
- *Script deployment commands.* Deployments are often complex and require many steps to be carried out and orchestrated precisely. For this reason, deployment is often scripted. These deployment scripts should be treated like code—documented, reviewed, tested, and version controlled. A scripting engine executes the deployment script automatically, saving time and minimizing opportunities for human error.

Manage Deployed System

- *Manage service interactions.* This tactic accommodates simultaneous deployment and execution of multiple versions of system services. Multiple requests from a client could be directed to either version in any sequence. Having multiple versions of the same service in operation, however, may introduce version incompatibilities. In such cases, the interactions between services need to be mediated so that version incompatibilities are proactively avoided. This tactic is a resource management strategy, obviating the need to completely replicate the resources so as to separately deploy the old and new versions.
- *Package dependencies.* This tactic packages an element together with its dependencies so that they get deployed together and so that the versions of the dependencies are consistent as the element moves from development into production. The dependencies may include libraries, OS versions, and utility containers (e.g., sidecar, service mesh), which we will discuss in Chapter 9. Three means of packaging dependencies are using containers, pods, or virtual machines; these are discussed in more detail in Chapter 16.

- *Feature toggle.* Even when your code is fully tested, you might encounter issues after deploying new features. For that reason, it is convenient to be able to integrate a “kill switch” (or feature toggle) for new features. The kill switch automatically disables a feature in your system at runtime, without forcing you to initiate a new deployment. This provides the ability to control deployed features without the cost and risk of actually redeploying services.

5.5 Tactics-Based Questionnaire for Deployability

Based on the tactics described in Section 5.4, we can create a set of deployability tactics—inspired questions, as presented in Table 5.2. To gain an overview of the architectural choices made to support deployability, the analyst asks each question and records the answers in the table. The answers to these questions can then be made the focus of subsequent activities: investigation of documentation, analysis of code or other artifacts, reverse engineering of code, and so forth.

TABLE 5.2 Tactics-Based Questionnaire for Deployability

Tactics Groups	Tactics Question	Supported? (Y/N)	Risk	Design Decisions and Location	Rationale and Assumptions
Manage deployment pipeline	Do you scale rollouts , rolling out new releases gradually (in contrast to releasing in an all-or-nothing fashion)?				
	Are you able to automatically roll back deployed services if you determine that they are not operating in a satisfactory fashion?				
	Do you script deployment commands to automatically execute complex sequences of deployment instructions?				
Manage deployed system	Do you manage service interactions so that multiple versions of services can be safely deployed simultaneously?				
	Do you package dependencies so that services are deployed along with all of the libraries, OS versions, and utility containers that they depend on?				

Tactics Groups	Tactics Question	Supported? (Y/N)	Risk	Design Decisions and Location	Rationale and Assumptions
	Do you employ feature toggles to automatically disable a newly released feature (rather than rolling back the newly deployed service) if the feature is determined to be problematic?				

5.6 Patterns for Deployability

Patterns for deployability can be organized into two categories. The first category contains patterns for structuring services to be deployed. The second category contains patterns for how to deploy services, which can be parsed into two broad subcategories: all-or-nothing or partial deployment. The two main categories for deployability are not completely independent of each other, because certain deployment patterns depend on certain structural properties of the services.

Patterns for Structuring Services

Microservice Architecture

The microservice architecture pattern structures the system as a collection of independently deployable services that communicate only via messages through service interfaces. There is no other form of interprocess communication allowed: no direct linking, no direct reads of another team's data store, no shared-memory model, no back-doors whatsoever. Services are usually stateless, and (because they are developed by a single relatively small team⁴) are relatively small—hence the term *microservice*. Service dependencies are acyclic. An integral part of this pattern is a discovery service so that messages can be appropriately routed.

Benefits:

- Time to market is reduced. Since each service is small and independently deployable, a modification to a service can be deployed without coordinating with teams that own other services. Thus, once a team completes its work on a new version of a service and that version has been tested, it can be deployed immediately.

4. At Amazon, service teams are constrained in size by the “two pizza rule”: The team must be no larger than can be fed by two pizzas.

- Each team can make its own technology choices for its service, as long as the technology choices support message passing. No coordination is needed with respect to library versions or programming languages. This reduces errors due to incompatibilities that arise during integration—and which are a major source of integration errors.
- Services are more easily scaled than coarser-grained applications. Since each service is independent, dynamically adding instances of the service is straightforward. In this way, the supply of services can be more easily matched to the demand.

Tradeoffs:

- Overhead is increased, compared to in-memory communication, because all communication among services occurs via messages across a network. This can be mitigated somewhat by using the service mesh pattern (see Chapter 9), which constrains the deployment of some services to the same host to reduce network traffic. Furthermore, because of the dynamic nature of microservice deployments, discovery services are heavily used, adding to the overhead. Ultimately, those discovery services may become a performance bottleneck.
- Microservices are less suitable for complex transactions because of the difficulty of synchronizing activities across distributed systems.
- The freedom for every team to choose its own technology comes at a cost—the organization must maintain those technologies and the required experience base.
- Intellectual control of the total system may be difficult because of the large number of microservices. This introduces a requirement for catalogs and databases of interfaces to assist in maintaining intellectual control. In addition, the process of properly combining services to achieve a desired outcome may be complex and subtle.
- Designing the services to have appropriate responsibilities and an appropriate level of granularity is a formidable design task.
- To achieve the ability to deploy versions independently, the architecture of the services must be designed to allow for that deployment strategy. Using the manage service interactions tactic described in Section 5.4 can help achieve this goal.

Organizations that have heavily employed the microservice architecture pattern include Google, Netflix, PayPal, Twitter, Facebook, and Amazon. Many other organizations have adopted the microservice architecture pattern as well; books and conferences exist that focus on how an organization can adopt the microservice architecture pattern for its own needs.

Patterns for Complete Replacement of Services

Suppose there are N instances of Service A and you wish to replace them with N instances of a new version of Service A, leaving no instances of the original version. You wish to do this with no reduction in quality of service to the clients of the service, so there must always be N instances of the service running.

Two different patterns for the complete replacement strategy are possible, both of which are realizations of the scale rollouts tactic. We'll cover them both together:

1. *Blue/green.* In a blue/green deployment, N new instances of the service would be created and each populated with new Service A (let's call these the green instances). After the N instances of new Service A are installed, the DNS server or discovery service would be changed to point to the new version of Service A. Once it is determined that the new instances are working satisfactorily, then and only then are the N instances of the original Service A removed. Before this cutoff point, if a problem is found in the new version, it is a simple matter of switching back to the original (the blue services) with little or no interruption.
2. *Rolling upgrade.* A rolling upgrade replaces the instances of Service A with instances of the new version of Service A one at a time. (In practice, you can replace more than one instance at a time, but only a small fraction are replaced in any single step.) The steps of the rolling upgrade are as follows:
 - a. Allocate resources for a new instance of Service A (e.g., a virtual machine).
 - b. Install and register the new version of Service A.
 - c. Begin to direct requests to the new version of Service A.
 - d. Choose an instance of the old Service A, allow it to complete any active processing, and then destroy that instance.
 - e. Repeat the preceding steps until all instances of the old version have been replaced.

Figure 5.4 shows a rolling upgrade process as implemented by Netflix's Asgard tool on Amazon's EC2 cloud platform.

Benefits:

- The benefit of these patterns is the ability to completely replace deployed versions of services without having to take the system out of service, thus increasing the system's availability.

Tradeoffs:

- The peak resource utilization for a blue/green approach is $2N$ instances, whereas the peak utilization for a rolling upgrade is $N + 1$ instances. In either case, resources to host these instances must be procured. Before the widespread adoption of cloud computing, procurement meant purchase: An organization had to purchase physical computers to perform the upgrade. Most of the time there was no upgrade in progress, so these additional computers largely sat idle. This made the financial tradeoff clear, and rolling upgrade was the standard approach. Now that computing resources can be rented on an as-needed basis, rather than purchased, the financial tradeoff is less compelling but still present.
- Suppose you detect an error in the new Service A when you deploy it. Despite all the testing you did in the development, integration, and staging environments, when your service is deployed to production, there may still be latent errors. If you are using blue/green deployment, by the time you discover an error in the new Service A, all of the original instances may have been deleted and rolling back to the old version could take considerable time. In contrast, a rolling upgrade may allow you to discover an error in the new version of the service while instances of the old version are still available.

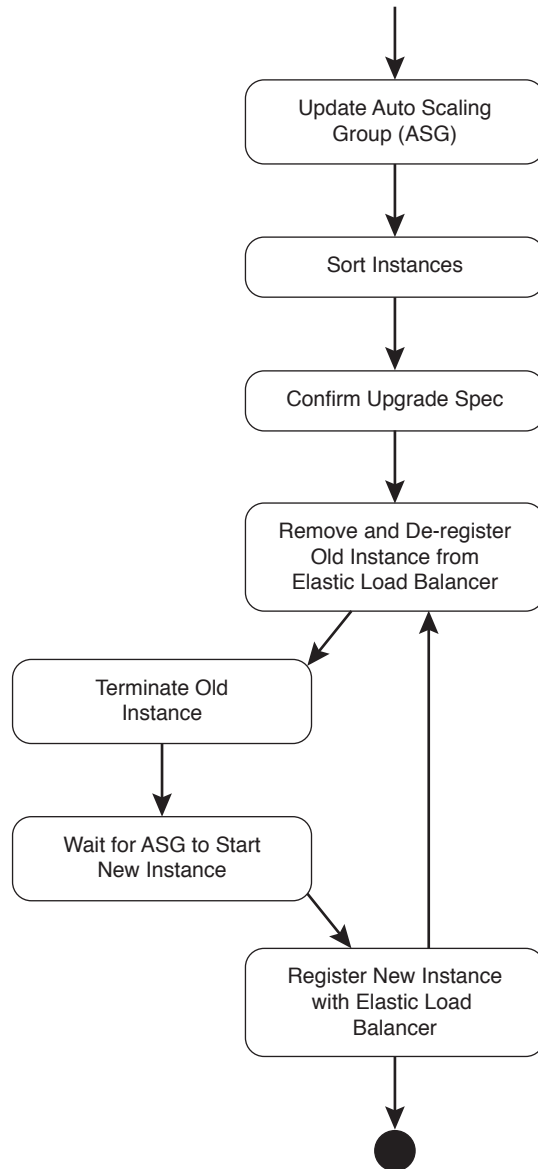


FIGURE 5.4 A flowchart of the rolling upgrade pattern as implemented by Netflix's Asgard tool

- From a client's perspective, if you are using the blue/green deployment model, then at any point in time either the new version or the old version is active, but not both. If you are using the rolling upgrade pattern, both versions are simultaneously active. This introduces the possibility of two types of problems: *temporal inconsistency* and *interface mismatch*.
 - *Temporal inconsistency*. In a sequence of requests by Client C to Service A, some may be served by the old version of the service and some may be served by the new version. If the versions behave differently, this may cause Client C to produce erroneous, or at least inconsistent, results. (This can be prevented by using the manage service interactions tactic.)
 - *Interface mismatch*. If the interface to the new version of Service A is different from the interface to the old version of Service A, then invocations by clients of Service A that have not been updated to reflect the new interface will produce unpredictable results. This can be prevented by extending the interface but not modifying the existing interface, and using the mediator pattern (see Chapter 7) to translate from the extended interface to an internal interface that produces correct behavior. See Chapter 15 for a fuller discussion.

Patterns for Partial Replacement of Services

Sometimes changing all instances of a service is undesirable. Partial-deployment patterns aim at providing multiple versions of a service simultaneously for different user groups; they are used for purposes such as quality control (canary testing) and marketing tests (A/B testing).

Canary Testing

Before rolling out a new release, it is prudent to test it in the production environment, but with a limited set of users. *Canary testing* is the continuous deployment analog of beta testing.⁵ Canary testing designates a small set of users who will test the new release. Sometimes, these testers are so-called power users or preview-stream users from outside your organization who are more likely to exercise code paths and edge cases that typical users may use less frequently. Users may or may not know that they are being used as guinea pigs—er, that is, canaries. Another approach is to use testers from within the organization that is developing the software. For example, Google employees almost never use the release that external users would be using, but instead act as testers for upcoming releases. When the focus of the testing is on determining how well new features are accepted, a variant of canary testing called dark launch is used.

In both cases, the users are designated as canaries and routed to the appropriate version of a service through DNS settings or through discovery-service configuration. After testing is

5. Canary testing is named after the 19th-century practice of bringing canaries into coal mines. Coal mining releases gases that are explosive and poisonous. Because canaries are more sensitive to these gases than humans, coal miners brought canaries into the mines and watched them for signs of reaction to the gases. The canaries acted as early warning devices for the miners, indicating an unsafe environment.

complete, users are all directed to either the new version or the old version, and instances of the deprecated version are destroyed. Rolling upgrade or blue/green deployment could be used to deploy the new version.

Benefits:

- Canary testing allows real users to “bang on” the software in ways that simulated testing cannot. This allows the organization deploying the service to collect “in use” data and perform controlled experiments with relatively low risk.
- Canary testing incurs minimal additional development costs, because the system being tested is on a path to production anyway.
- Canary testing minimizes the number of users who may be exposed to a serious defect in the new system.

Tradeoffs:

- Canary testing requires additional up-front planning and resources, and a strategy for evaluating the results of the tests needs to be formulated.
- If canary testing is aimed at power users, those users have to be identified and the new version routed to them.

A/B Testing

A/B testing is used by marketers to perform an experiment with real users to determine which of several alternatives yields the best business results. A small but meaningful number of users receive a different treatment from the remainder of the users. The difference can be minor, such as a change to the font size or form layout, or it can be more significant. For example, HomeAway (now Vrbo) has used A/B testing to vary the format, content, and look-and-feel of its worldwide websites, tracking which editions produced the most rentals. The “winner” would be kept, the “loser” discarded, and another contender designed and deployed. Another example is a bank offering different promotions to open new accounts. An oft-repeated story is that Google tested 41 different shades of blue to decide which shade to use to report search results.

As in canary testing, DNS servers and discovery-service configurations are set to send client requests to different versions. In A/B testing, the different versions are monitored to see which one provides the best response from a business perspective.

Benefits:

- A/B testing allows marketing and product development teams to run experiments on, and collect data from, real users.
- A/B testing can allow for targeting of users based on an arbitrary set of characteristics.

Tradeoffs:

- A/B testing requires the implementation of alternatives, one of which will be discarded.
- Different classes of users, and their characteristics, need to be identified up front.

5.7 For Further Reading

Much of the material in this chapter is adapted from *Deployment and Operations for Software Engineers* by Len Bass and John Klein [Bass 19] and from [Kazman 20b].

A general discussion of deployability and architecture in the context of DevOps can be found in [Bass 15].

The tactics for deployability owe much to the work of Martin Fowler and his colleagues, which can be found in [Fowler 10], [Lewis 14], and [Sato 14].

Deployment pipelines are described in much more detail in [Humble 10]

Microservices and the process of migrating to microservices are described in [Newman 15].

5.8 Discussion Questions

1. Write a set of concrete scenarios for deployability using each of the possible responses in the general scenario.
2. Write a concrete deployability scenario for the software for a car (such as a Tesla).
3. Write a concrete deployability scenario for a smartphone app. Now write one for the server-side infrastructure that communicates with this app.
4. If you needed to display the results of a search operation, would you perform A/B testing or simply use the color that Google has chosen? Why?
5. Referring to the structures described in Chapter 1, which structures would be involved in implementing the package dependencies tactic? Would you use the uses structure? Why or why not? Are there other structures you would need to consider?
6. Referring to the structures described in Chapter 1, which structures would be involved in implementing the manage service interactions tactic? Would you use the uses structure? Why or why not? Are there other structures you would need to consider?
7. Under what circumstances would you prefer to roll forward to a new version of service, rather than to roll back to a prior version? When is roll forward a poor choice?

This page intentionally left blank

6



Energy Efficiency

Energy is a bit like money: If you have a positive balance, you can distribute it in various ways, but according to the classical laws that were believed at the beginning of the century, you weren't allowed to be overdrawn.

—Stephen Hawking

Energy used by computers used to be free and unlimited—or at least that's how we behaved. Architects rarely gave much consideration to the energy consumption of software in the past. But those days are now gone. With the dominance of mobile devices as the primary form of computing for most people, with the increasing adoption of the Internet of Things (IoT) in industry and government, and with the ubiquity of cloud services as the backbone of our computing infrastructure, energy has become an issue that architects can no longer ignore. Power is no longer “free” and unlimited. The energy efficiency of mobile devices affects us all. Likewise, cloud providers are increasingly concerned with the energy efficiency of their server farms. In 2016, it was reported that data centers globally accounted for more energy consumption (by 40 percent) than the entire United Kingdom—about 3 percent of all energy consumed worldwide. More recent estimates put that share up as high as 10 percent. The energy costs associated with running and, more importantly, cooling large data centers have led people to calculate the cost of putting whole data centers in space, where cooling is free and the sun provides unlimited power. At today's launch prices, the economics are actually beginning to look favorable. Notably, server farms located underwater and in arctic climates are already a reality.

At both the low end and the high end, energy consumption of computational devices has become an issue that we should consider. This means that we, as architects, now need to add *energy efficiency* to the long list of competing qualities that we consider when designing a system. And, as with every other quality attribute, there are nontrivial tradeoffs to consider: energy usage versus performance or availability or modifiability or time to market. Thus considering energy efficiency as a first-class quality attribute is important for the following reasons:

1. An architectural approach is necessary to gain control over *any* important system quality attribute, and energy efficiency is no different. If system-wide techniques for monitoring

and managing energy are lacking, then developers are left to invent them on their own. This will, in the best case, result in an ad hoc approach to energy efficiency that produces a system that is hard to maintain, measure, and evolve. In the worst case, it will yield an approach that simply does not predictably achieve the desired energy efficiency goals.

2. Most architects and developers are unaware of energy efficiency as a quality attribute of concern, and hence do not know how to go about engineering and coding for it. More fundamentally, they lack an understanding of energy efficiency requirements—how to gather them and analyze them for completeness. Energy efficiency is not taught, or typically even mentioned, as a programmer’s concern in today’s educational curricula. In consequence, students may graduate with degrees in engineering or computer science without ever having been exposed to these issues.
3. Most architects and developers lack suitable design concepts—models, patterns, tactics, and so forth—for designing for energy efficiency, as well as managing and monitoring it at runtime. But since energy efficiency is a relatively recent concern for the software engineering community, these design concepts are still in their infancy and no catalog yet exists.

Cloud platforms typically do not have to be concerned with running out of energy (except in disaster scenarios), whereas this is a daily concern for users of mobile devices and some IoT devices. In cloud environments, scaling up and scaling down are core competencies, so decisions must be made on a regular basis about optimal resource allocation. With IoT devices, their size, form factors, and heat output all constrain their design space—there is no room for bulky batteries. In addition, the sheer number of IoT devices projected to be deployed in the next decade makes their energy usage a concern.

In all of these contexts, energy efficiency must be balanced with performance and availability, requiring engineers to consciously reason about such tradeoffs. In the cloud context, greater allocation of resources—more servers, more storage, and so on—creates improved performance capabilities as well as improved robustness against failures of individual devices, but at the cost of energy and capital outlays. In the mobile and IoT contexts, greater allocation of resources is typically not an option (although shifting the computational burden from a mobile device to a cloud back-end is possible), so the tradeoffs tend to center on energy efficiency versus performance and usability. Finally, in all contexts, there are tradeoffs between energy efficiency, on the one hand, and buildability and modifiability, on the other hand.

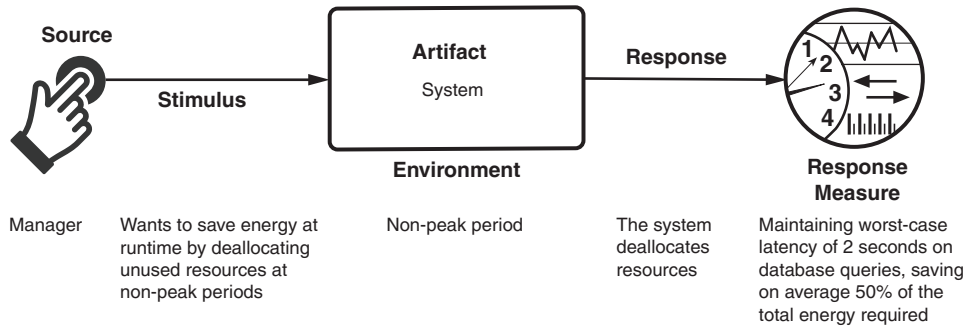
6.1 Energy Efficiency General Scenario

From these considerations, we can now determine the various portions of the energy efficiency general scenario, as presented in Table 6.1.

TABLE 6.1 Energy Efficiency General Scenario

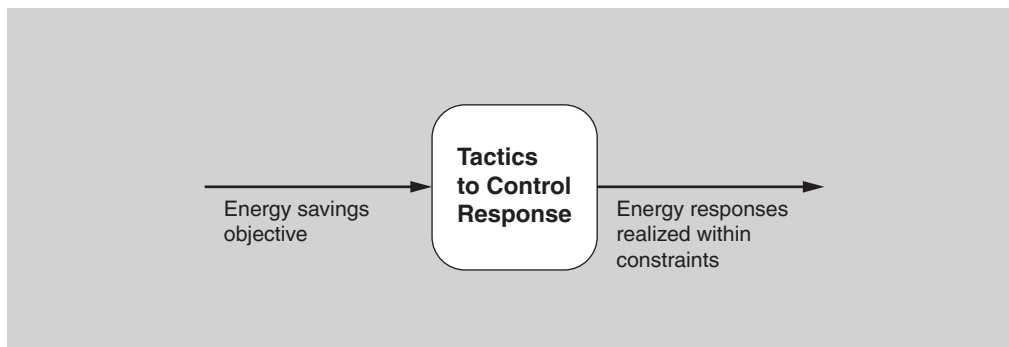
Portion of Scenario	Description	Possible Values
Source	This specifies who or what requests or initiates a request to conserve or manage energy.	End user, manager, system administrator, automated agent
Stimulus	A request to conserve energy.	Total usage, maximum instantaneous usage, average usage, etc.
Artifacts	This specifies what is to be managed.	Specific devices, servers, VMs, clusters, etc.
Environment	Energy is typically managed at runtime, but many interesting special cases exist, based on system characteristics.	Runtime, connected, battery-powered, low-battery mode, power-conservation mode
Response	What actions the system takes to conserve or manage energy usage.	One or more of the following: ▪ Disable services ▪ Deallocate runtime services ▪ Change allocation of services to servers ▪ Run services at a lower consumption mode ▪ Allocate/deallocate servers ▪ Change levels of service ▪ Change scheduling
Response measure	The measures revolve around the amount of energy saved or consumed and the effects on other functions or quality attributes.	Energy managed or saved in terms of: ▪ Maximum/average kilowatt load on the system ▪ Average/total amount of energy saved ▪ Total kilowatt hours used ▪ Time period during which the system must stay powered on ... while still maintaining a required level of functionality and acceptable levels of other quality attributes

Figure 6.1 illustrates a concrete energy efficiency scenario: *A manager wants to save energy at runtime by deallocating unused resources at non-peak periods. The system deallocates resources while maintaining worst-case latency of 2 seconds on database queries, saving on average 50 percent of the total energy required.*

**FIGURE 6.1** Sample energy efficiency scenario

6.2 Tactics for Energy Efficiency

An energy efficiency scenario is catalyzed by the desire to conserve or manage energy while still providing the required (albeit not necessarily full) functionality. This scenario is successful if the energy responses are achieved within acceptable time, cost, and quality constraints. We illustrate this simple relationship—and hence the goal of energy efficiency tactics—in Figure 6.2.

**FIGURE 6.2** Goal of energy efficiency tactics

Energy efficiency is, at its heart, about effectively utilizing resources. We group the tactics into three broad categories: resource monitoring, resource allocation, and resource adaptation (Figure 6.3). By “resource,” we mean a computational device that consumes energy while providing its functionality. This is analogous to the definition of a *hardware* resource in Chapter 9, which includes CPUs, data stores, network communications, and memory.

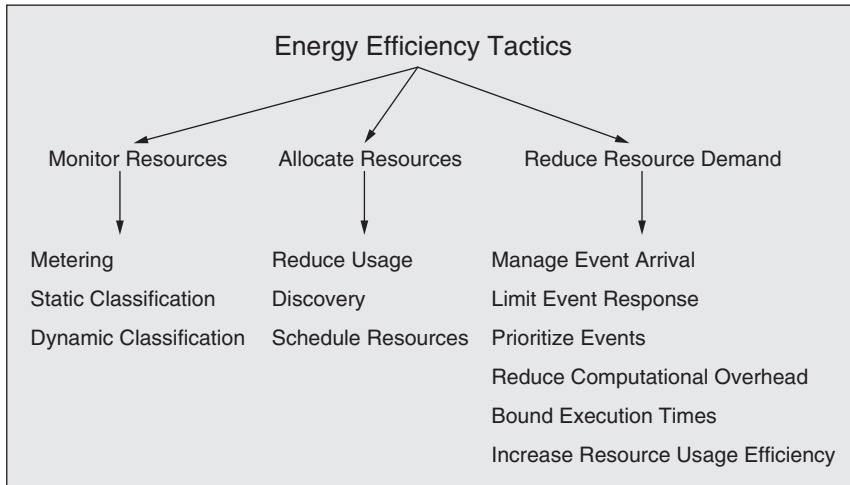


FIGURE 6.3 Energy efficiency tactics

Monitor Resources

You can’t manage what you can’t measure, and so we begin with resource monitoring. The tactics for resource monitoring are metering, static classification, and dynamic classification.

- *Metering.* The metering tactic involves collecting data about the energy consumption of computational resources via a sensor infrastructure, in near real time. At the coarsest level, the energy consumption of an entire data center can be measured from its power meter. Individual servers or hard drives can be measured using external tools such as amp meters or watt-hour meters, or using built-in tools such as those provided with metered rack PDUs (power distribution units), ASICs (application-specific integrated circuits), and so forth. In battery-operated systems, the energy remaining in a battery can be determined through a battery management system, which is a component of modern batteries.
- *Static classification.* Sometimes real-time data collection is infeasible. For example, if an organization is using an off-premises cloud, it might not have direct access to real-time energy data. Static classification allows us to estimate energy consumption by

cataloging the computing resources used and their known energy characteristics—the amount of energy used by a memory device per fetch, for example. These characteristics are available as benchmarks, or from manufacturers’ specifications.

- *Dynamic classification.* In cases where a static model of a computational resource is inadequate, a dynamic model might be required. Unlike static models, dynamic models estimate energy consumption based on knowledge of transient conditions such as workload. The model could be a simple table lookup, a regression model based on data collected during prior executions, or a simulation.

Allocate Resources

Resource allocation means assigning resources to do work in a way that is mindful of energy consumption. The tactics for resource allocation are to reduce usage, discovery, and scheduling.

- *Reduce usage.* Usage can be reduced at the device level by device-specific activities such as reducing the refresh rate of a display or darkening the background. Removing or deactivating resources when demands no longer require them is another method for decreasing energy consumption. This may involve spinning down hard drives, turning off CPUs or servers, running CPUs at a slower clock rate, or shutting down current to blocks of the processor that are not in use. It might also take the form of moving VMs onto the minimum number of physical servers (consolidation), combined with shutting down idle computational resources. In mobile applications, energy savings may be realized by sending part of the computation to the cloud, assuming that the energy consumption of communication is lower than the energy consumption of computation.
- *Discovery.* As we will see in Chapter 7, a discovery service matches service requests (from clients) with service providers, supporting the identification and remote invocation of those services. Traditionally discovery services have made these matches based on a description of the service request (typically an API). In the context of energy efficiency, this request could be annotated with energy information, allowing the requestor to choose a service provider (resource) based on its (possibly dynamic) energy characteristics. For the cloud, this energy information can be stored in a “green service directory” populated by information from metering, static classification, or dynamic classification (the resource monitoring tactics). For a smartphone, the information could be obtained from an app store. Currently such information is ad hoc at best, and typically nonexistent in service APIs.
- *Schedule resources.* Scheduling is the allocation of tasks to computational resources. As we will see in Chapter 9, the schedule resources tactic can increase performance. In the energy context, it can be used to effectively manage energy usage, given task constraints and respecting task priorities. Scheduling can be based on data collected using one or more resource monitoring tactics. Using an energy discovery service in a cloud context, or a controller in a multi-core context, a computational task can dynamically switch among computational resources, such as service providers, selecting the ones that offer better energy efficiency or lower energy costs. For example, one provider may be more lightly loaded than another, allowing it to adapt its energy usage, perhaps using some of the tactics described earlier, and consume less energy, on average, per unit of work.

Reduce Resource Demand

This category of tactics is detailed in Chapter 9. Tactics in this category—manage event arrival, limit event response, prioritize events (perhaps letting low-priority events go unserved), reduce computational overhead, bound execution times, and increase resource usage efficiency—all directly increase energy efficiency by doing less work. This is a complementary tactic to reduce usage, in that the reduce usage tactic assumes that the demand stays the same, whereas the reduce resource demand tactics are a means of explicitly managing (and reducing) the demand.

6.3 Tactics-Based Questionnaire for Energy Efficiency

As described in Chapter 3, this tactics-based questionnaire is intended to very quickly understand the degree to which an architecture employs specific tactics to manage energy efficiency.

Based on the tactics described in Section 6.2, we can create a set of tactics-inspired questions, as presented in Table 6.2. To gain an overview of the architectural choices made to support energy efficiency, the analyst asks each question and records the answers in the table. The answers to these questions can then be made the focus of further activities: investigation of documentation, analysis of code or other artifacts, reverse engineering of code, and so forth.

TABLE 6.2 Tactics-Based Questionnaire for Energy Efficiency

Tactics Group	Tactics Question	Supported? (Y/N)	Risk	Design Decisions and Location	Rationale and Assumptions
Resource Monitoring	Does your system meter the use of energy? That is, does the system collect data about the actual energy consumption of computational devices via a sensor infrastructure, in near real time? Does the system statically classify devices and computational resources? That is, does the system have reference values to estimate the energy consumption of a device or resource (in cases where real-time metering is infeasible or too computationally expensive)?				

continues

TABLE 6.2 Tactics-Based Questionnaire for Energy Efficiency *continued*

Tactics Group	Tactics Question	Supported? (Y/N)	Risk	Design Decisions and Location	Rationale and Assumptions
Resource Monitoring	Does the system dynamically classify devices and computational resources? In cases where static classification is not accurate due to varying load or environmental conditions, does the system use dynamic models, based on prior data collected, to estimate the varying energy consumption of a device or resource at runtime?				
Resource Allocation	Does the system reduce usage to scale down resource usage? That is, can the system deactivate resources when demands no longer require them, in an effort to save energy? This may involve spinning down hard drives, darkening displays, turning off CPUs or servers, running CPUs at a slower clock rate, or shutting down memory blocks of the processor that are not being used. Does the system schedule resources to more effectively utilize energy, given task constraints and respecting task priorities, by switching computational resources, such as service providers, to the ones that offer better energy efficiency or lower energy costs? Is scheduling based on data collected (using one or more resource monitoring tactics) about the state of the system? Does the system make use of a discovery service to match service requests to service providers? In the context of energy efficiency, a service request could be annotated with energy requirement information, allowing the requestor to choose a service provider based on its (possibly dynamic) energy characteristics.				

Tactics Group	Tactics Question	Supported? (Y/N)	Risk	Design Decisions and Location	Rationale and Assumptions
Reduce Resource Demand	Do you consistently attempt to reduce resource demand ? Here, you may insert the questions in this category from the Tactics-Based Questionnaire for Performance from Chapter 9.				

6.4 Patterns

Some examples of patterns used for energy efficiency include sensor fusion, kill abnormal tasks, and power monitor.

Sensor Fusion

Mobile apps and IoT systems often collect data from their environment using multiple sensors. In this pattern, data from low-power sensors can be used to infer whether data needs to be collected from higher-power sensors. A common example in the mobile phone context is using accelerometer data to assess if the user has moved and, if so, to update the GPS location. This pattern assumes that accessing the low-power sensor is much cheaper, in terms of energy consumption, than accessing the higher-power sensor.

Benefits:

- The obvious benefit of this pattern is the ability to minimize the usage of more energy-intensive devices in an intelligent way rather than, for example, just reducing the frequency of consulting the more energy-intensive sensor.

Tradeoffs:

- Consulting and comparing multiple sensors adds up-front complexity.
- The higher-energy-consuming sensor will provide higher-quality data, albeit at the cost of increased power consumption. And it will provide this data more quickly, since using the more energy-intensive sensor alone takes less time than first consulting a secondary sensor.
- In cases where the inference frequently results in accessing the higher-power sensor, this pattern could result in overall higher energy usage.

Kill Abnormal Tasks

Mobile systems, because they are often executing apps of unknown provenance, may end up unknowingly running some exceptionally power-hungry apps. This pattern provides a way

to monitor the energy usage of such apps and to interrupt or kill energy-greedy operations. For example, if an app is issuing an audible alert and vibrating the phone and the user is not responding to these alerts, then after a predetermined timeout period the task is killed.

Benefits:

- This pattern provides a “fail-safe” option for managing the energy consumption of apps with unknown energy properties.

Tradeoffs:

- Any monitoring process adds a small amount of overhead to system operations, which may affect performance and, to a small extent, energy usage.
- The usability of this pattern needs to be considered. Killing energy-hungry tasks may be counter to the user’s intention.

Power Monitor

The power monitor pattern monitors and manages system devices, minimizing the time during which they are active. This pattern attempts to automatically disable devices and interfaces that are not being actively used by the application. It has long been used within integrated circuits, where blocks of the circuit are shut down when they are not being used, in an effort to save energy.

Benefits:

- This pattern can allow for intelligent savings of power at little to no impact to the end user, assuming that the devices being shut down are truly not needed.

Tradeoffs:

- Once a device has been switched off, switching it on adds some latency before it can respond, as compared with keeping it continually running. And, in some cases, the startup may be more energy expensive than a certain period of steady-state operation.
- The power monitor needs to have knowledge of each device and its energy consumption characteristics, which adds up-front complexity to the system design.

6.5 For Further Reading

The first published set of energy tactics appeared in [Procaccianti 14]. These were, in part, the inspiration for the tactics presented here. The 2014 paper subsequently inspired [Paradis 21]. Many of the tactics presented in this chapter owe a debt to these two papers.

For a good general introduction to energy usage in software development—and what developers do not know—you should read [Pang 16].

Several research papers have investigated the consequences of design choices on energy consumption, such as [Kazman 18] and [Chowdhury 19].

A general discussion of the importance of creating “energy-aware” software can be found in [Fonseca 19].

Energy patterns for mobile devices have been catalogued by [Cruz 19] and [Schaarschmidt 20].

6.6 Discussion Questions

1. Write a set of concrete scenarios for energy efficiency using each of the possible responses in the general scenario.
2. Create a concrete energy efficiency scenario for a smartphone app (for example, a health monitoring app).
3. Create a concrete energy efficiency scenario for a cluster of data servers in a data center. What are the important distinctions between this scenario and the one you created for question 2?
4. Enumerate the energy efficiency techniques that are currently employed by your laptop or smartphone.
5. What are the energy tradeoffs in your smartphone between using Wi-Fi and the cellular network?
6. Calculate the amount of greenhouse gases in the form of carbon dioxide that you, over an average lifetime, will exhale into the atmosphere. How many Google searches does this equate to?
7. Suppose Google reduced its energy usage per search by 1 percent. How much energy would that save per year?
8. How much energy did you use to answer question 7?

This page intentionally left blank

7



Integrability

Integration is a basic law of life; when we resist it, disintegration is the natural result, both inside and outside of us. Thus we come to the concept of harmony through integration.

—Norman Cousins

According to the Merriam-Webster dictionary, the adjective *integrable* means “capable of being integrated.” We’ll give you a moment to catch your breath and absorb that profound insight. But for practical software systems, software architects need to be concerned about more than just making separately developed components cooperate; they are also concerned with the *costs* and *technical risks* of anticipated and (to varying degrees) unanticipated future integration tasks. These risks may be related to schedule, performance, or technology.

A general, abstract representation of the integration problem is that a project needs to integrate a unit of software C , or a set of units $C_1, C_2, \dots, C_n$, into a system S . S might be a platform, into which we integrate $\{C_i\}$, or it might be an existing system that already contains $\{C_1, C_2, \dots, C_n\}$ and our task is to design for, and analyze the costs and technical risks of, integrating $\{C_{n+1}, \dots, C_m\}$.

We assume we have control over S , but the $\{C_i\}$ may be outside our control—supplied by external vendors, for example, so our level of understanding of each C_i may vary. The clearer our understanding of C_i , the more capable the design and accurate the analysis will be.

Of course, S is not static but will evolve, and this evolution may require reanalysis. Integrability (like other quality attributes such as modifiability) is challenging because it is about planning for a future when we have incomplete information at our disposal. Simply put, some integrations will be simpler than others because they have been anticipated and accommodated in the architecture, whereas others will be more complex because they have not been.

Consider a simple analogy: To plug a North American plug (an example of a C_i) into a North American socket (an interface provided by the electrical system S), the “integration” is trivial. However, integrating a North American plug into a British socket will require an adapter. And the device with the North American plug may only run on 110-volt power, requiring further adaptation before it will work in a British 220-volt socket. Furthermore, if the component was designed to run at 60 Hz and the system provides 70 Hz, the component may not operate as intended even though it plugs in just fine. The architectural decisions made

by the creators of S and C_i —for example, to provide plug adapters or voltage adapters, or to make the component operate identically at different frequencies—will affect the cost and risk of the integration.

7.1 Evaluating the Integrability of an Architecture

Integration difficulty—the costs and the technical risks—can be thought of as a function of the size of and the “distance” between the interfaces of $\{C_i\}$ and S :

Size is the number of potential dependencies between $\{C_i\}$ and S .

Distance is the difficulty of resolving differences at each of the dependencies.

Dependencies are often measured syntactically. For example, we say that module A is dependent on component B if A calls B , if A inherits from B , or if A uses B . But while syntactic dependency is important, and will continue to be important in the future, dependency can occur in forms that are not detectable by any syntactic relation. Two components might be coupled *temporally* or through *resources* because they share and compete for a finite resource at runtime (e.g., memory, bandwidth, CPU), share control of an external device, or have a timing dependency. Or they might be coupled *semantically* because they share knowledge of the same protocol, file format, unit of measure, metadata, or some other aspect. The reason that these distinctions are important is that temporal and semantic dependencies are not often well understood, explicitly acknowledged, or properly documented. Missing or implicit knowledge is always a risk for a large, long-lived project, and such knowledge gaps will inevitably increase the costs and risks of integration and integration testing.

Consider the trend toward services and microservices in computation today. This approach is fundamentally about decoupling components to reduce the number and distance of their dependencies. Services only “know” each other via their published interfaces and, if that interface is an appropriate abstraction, changes to one service have less chance to ripple to other services in the system. The ever-increasing decoupling of components is an industry-wide trend that has been going on for decades. Service orientation, by itself, addresses (that is, reduces) only the syntactic aspects of dependency; it does not address the temporal or semantic aspects. Supposedly decoupled components that have detailed knowledge of each other and make assumptions about each other are in fact tightly coupled, and changing them in the future may well be costly.

For integrability purposes, “interfaces” must be understood as much more than simply APIs. They must characterize all of the relevant dependencies between the elements. When trying to understand dependencies between components, the concept of “distance” is helpful. As components interact, how aligned are they with respect to how they cooperate to successfully carry out an interaction? Distance may mean:

- *Syntactic distance.* The cooperating elements must agree on the number and type of the data elements being shared. For example, if one element sends an integer and the other

expects a floating point, or perhaps the bits within a data field are interpreted differently, this discrepancy presents a syntactic distance that must be bridged. Differences in data types are typically easy to observe and predict. For example, such type mismatches could be caught by a compiler. Differences in bit masks, while similar in nature, are often more difficult to detect, and the analyst may need to rely on documentation or scrutiny of the code to identify them.

- *Data semantic distance.* The cooperating elements must agree on the data semantics; that is, even if two elements share the same data type, their values are interpreted differently. For example, if one data value represents altitude in meters and the other represents altitude in feet, this presents a data semantic distance that must be bridged. This kind of mismatch is typically difficult to observe and predict, although the analyst's life is improved somewhat if the elements involved employ metadata. Mismatches in data semantics may be discovered by comparing interface documentation or metadata descriptions, if available, or by checking the code, if available.
- *Behavioral semantic distance.* The cooperating elements must agree on behavior, particularly with respect to the states and modes of the system. For example, a data element may be interpreted differently in system startup, shutdown, or recovery mode. Such states and modes may, in some cases, be explicitly captured in protocols. As another example, C_i and C_j may make different assumptions regarding control, such as each expecting the other to initiate interactions.
- *Temporal distance.* The cooperating elements must agree on assumptions about time. Examples of temporal distance include operating at different rates (e.g., one element emits values at a rate of 10 Hz and the other expects values at 60 Hz) or making different timing assumptions (e.g., one element expects event A to follow event B and the other element expects event A to follow event B with no more than 50 ms latency). While this might be considered to be a subcase of behavioral semantics, it is so important (and often subtle) that we call it out explicitly.
- *Resource distance.* The cooperating elements must agree on assumptions about shared resources. Examples of resource distance may involve devices (e.g., one element requires exclusive access to a device, whereas another expects shared access) or computational resources (e.g., one element needs 12 GB of memory to run optimally and the other needs 10 GB, but the target CPU has only 16 GB of physical memory; or three elements are simultaneously producing data at 3 Mbps each, but the communication channel offers a peak capacity of just 5 Mbps). Again, this distance may be seen as related to behavioral distance, but it should be consciously analyzed.

Such details are not typically mentioned in a programming language interface description. In the organizational context, however, these unstated, implicit interfaces often add time and complexity to integration tasks (and modification and debugging tasks). This is why interfaces are architectural concerns, as we will discuss further in Chapter 15.

In essence, integrability is about discerning and bridging the distance between the elements of each potential dependency. This is a form of planning for modifiability. We will revisit this topic in Chapter 8.

7.2 General Scenario for Integrability

Table 7.1 presents the general scenario for integrability.

TABLE 7.1 General Scenario for Integrability

Portion of Scenario	Description	Possible Values
Source	Where does the stimulus come from?	One or more of the following: ▪ Mission/system stakeholder ▪ Component marketplace ▪ Component vendor
Stimulus	What is the stimulus? That is, what kind of integration is being described?	One of the following: ▪ Add new component ▪ Integrate new version of existing component ▪ Integrate existing components together in a new way
Artifact	What parts of the system are involved in the integration?	One of the following: ▪ Entire system ▪ Specific set of components ▪ Component metadata ▪ Component configuration
Environment	What state is the system in when the stimulus occurs?	One of the following: ▪ Development ▪ Integration ▪ Deployment ▪ Runtime
Response	How will an “integrable” system respond to the stimulus?	One or more of the following: ▪ Changes are {completed, integrated, tested, deployed} ▪ Components in the new configuration are successfully and correctly (syntactically and semantically) exchanging information ▪ Components in the new configuration are successfully collaborating ▪ Components in the new configuration do not violate any resource limits
Response measure	How is the response measured?	One or more of the following: ▪ Cost, in terms of one or more of: ▪ Number of components changed ▪ Percentage of code changed ▪ Lines of code changed ▪ Effort ▪ Money ▪ Calendar time ▪ Effects on other quality attribute response measures (to capture allowable tradeoffs)

Figure 7.1 illustrates a sample integrability scenario constructed from the general scenario: *A new data filtering component has become available in the component marketplace. The new component is integrated into the system and deployed in 1 month, with no more than 1 person-month of effort.*

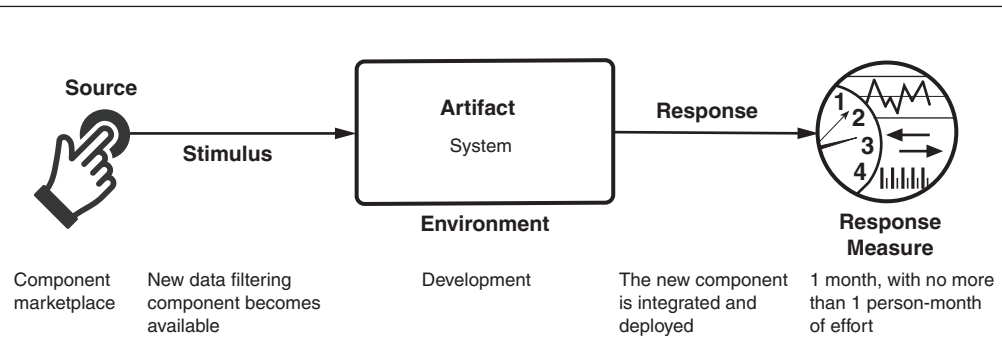


FIGURE 7.1 Sample integrability scenario

7.3 Integrability Tactics

The goals for the integrability tactics are to reduce the costs and risks of adding new components, reintegrating changed components, and integrating sets of components together to fulfill evolutionary requirements, as illustrated in Figure 7.2.

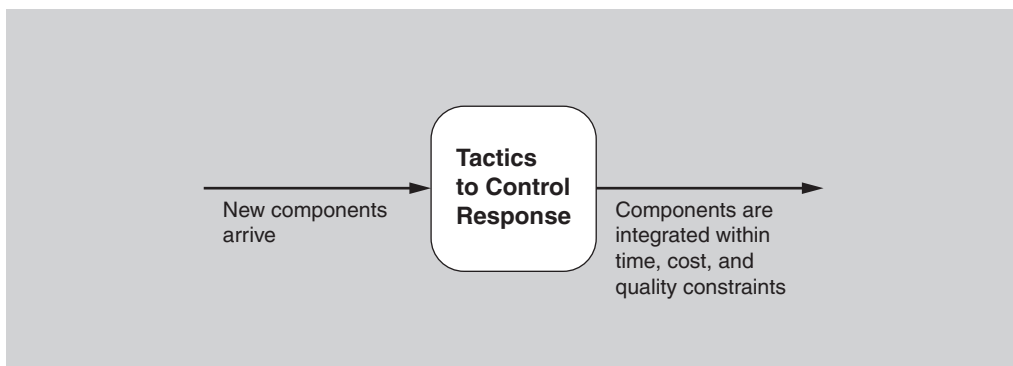


FIGURE 7.2 Goal of integrability tactics

The tactics achieve these goals either by reducing the number of potential dependencies between components or by reducing the expected distance between components. Figure 7.3 shows an overview of the integrability tactics.

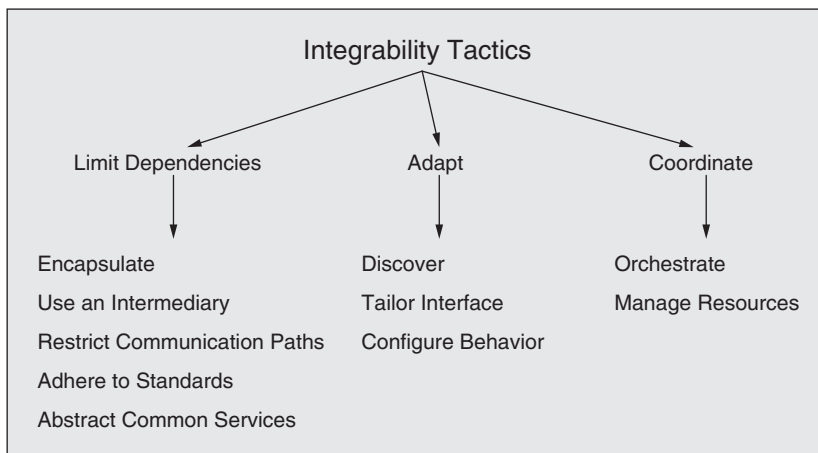


FIGURE 7.3 Integrability tactics

Limit Dependencies

Encapsulate

Encapsulation is the foundation upon which all other integrability tactics are built. It is therefore seldom seen on its own, but its use is implicit in the other tactics described here.

Encapsulation introduces an explicit interface to an element and ensures that all access to the element passes through this interface. Dependencies on the element internals are eliminated, because all dependencies must flow through the interface. Encapsulation reduces the probability that a change to one element will propagate to other elements, by reducing either the number of dependencies or their distances. These strengths are, however, reduced because the interface limits the ways in which external responsibilities can interact with the element (perhaps through a wrapper). In consequence, the external responsibilities can only directly interact with the element through the exposed interface (indirect interactions, such as dependence on quality of service, will likely remain unchanged).

Encapsulation may also hide interfaces that are not relevant for a particular integration task. An example is a library used by a service that can be completely hidden from all consumers and changed without these changes propagating to the consumers.

Encapsulation, then, can reduce the number of dependencies as well as the syntactic, data, and behavior semantic distances between *C* and *S*.

Use an Intermediary

Intermediaries are used for breaking dependencies between a set of components C_i or between C_i and the system S . Intermediaries can be used to resolve different types of dependencies. For example, intermediaries such as a publish–subscribe bus, shared data repository, or dynamic service discovery all reduce dependencies between data producers and consumers by removing any need for either to know the identity of the other party. Other intermediaries, such as data transformers and protocol translators, resolve forms of syntactic and data semantic distance.

Determining the specific benefits of a particular intermediary requires knowledge of what the intermediary actually does. An analyst needs to determine whether the intermediary reduces the number of dependencies between a component and the system and which dimensions of distance, if any, it addresses.

Intermediaries are often introduced during integration to resolve specific dependencies, but they can also be included in an architecture to promote integrability with respect to anticipated scenarios. Including a communication intermediary such as a publish–subscribe bus in an architecture, and then restricting communication paths to and from sensors to this bus, is an example of using an intermediary with the goal of promoting integrability of sensors.

Restrict Communication Paths

This tactic restricts the set of elements with which a given element can communicate. In practice, this tactic is implemented by restricting an element’s visibility (when developers cannot see an interface, they cannot employ it) and by authorization (i.e., restricting access to only authorized elements). The restrict communication paths tactic is seen in service-oriented architectures (SOAs), in which point-to-point requests are discouraged in favor of forcing all requests to go through an enterprise service bus so that routing and preprocessing can be done consistently.

Adhere to Standards

Standardization in system implementations is a primary enabler of integrability and interoperability, across both platforms and vendors. Standards vary considerably in terms of the scope of what they prescribe. Some focus on defining syntax and data semantics. Others include richer descriptions, such as those describing protocols that include behavioral and temporal semantics.

Standards similarly vary in their scope of applicability or adoption. For example, standards published by widely recognized standards-setting organizations such as the Institute of Electrical and Electronics Engineers (IEEE), the International Organization for Standardization (ISO), and the Object Management Group (OMG) are more likely to be broadly adopted. Conventions that are local to an organization, particularly if well documented and enforced, can provide similar benefits as “local standards,” though with less expectation of benefits when integrating components from outside the local standard’s sphere of adoption.

Adopting a standard can be an effective integrability tactic, although its effectiveness is limited to benefits based on the dimensions of difference addressed in the standard and how likely it is that future component suppliers will conform to the standard. Restricting communication with a system S to require use of the standard often reduces the number of potential

dependencies. Depending on what is defined in a standard, it may also address syntactic, data semantic, behavioral semantic, and temporal dimensions of distance.

Abstract Common Services

Where two elements provide services that are similar but not quite the same, it may be useful to hide both specific elements behind a common abstraction for a more general service. This abstraction might be realized as a common interface implemented by both, or it might involve an intermediary that translates requests for the abstract service to more specific requests for the elements hidden behind the abstraction. The resulting encapsulation hides the details of the elements from other components in the system. In terms of integrability, this means that future components can be integrated with a single abstraction rather than separately integrated with each of the specific elements.

When the abstract common services tactic is combined with an intermediary (such as a wrapper or adapter), it can also normalize syntactic and semantic variations among the specific elements. For example, we see this when systems use many sensors of the same type from different manufacturers, each with its own device drivers, accuracy, or timing properties, but the architecture provides a common interface to them. As another example, your browser may accommodate various kinds of ad-blocking plug-ins, yet because of the plug-in interface the browser itself can remain blissfully unaware of your choice.

Abstracting common services allows for consistency when handling common infrastructure concerns (e.g., translations, security mechanisms, and logging). When these features change, or when new versions of the components implementing these features change, the changes can be made in a smaller number of places. An abstract service is often paired with an intermediary that may perform processing to hide syntactic and data semantic differences among specific elements.

Adapt

Discover

A discovery service is a catalog of relevant addresses, which comes in handy whenever there is a need to translate from one form of address to another, whenever the target address may have been dynamically bound, or when there are multiple targets. It is the mechanism by which applications and services locate each other. A discovery service may be used to enumerate variants of particular elements that are used in different products.

Entries in a discovery service are there because they were registered. This registration can happen statically, or it can happen dynamically when a service is instantiated. Entries in the discovery service should be de-registered when they are no longer relevant. Again, this can be done statically, such as with a DNS server, or dynamically. Dynamic de-registration can be handled by the discovery service itself performing health checks on its entries, or it can be carried out by an external piece of software that knows when a particular entry in the catalog is no longer relevant.

A discovery service may include entries that are themselves discovery services. Likewise, entries in a discovery service may have additional attributes, which a query may reference. For

example, a weather discovery service may have an attribute of “cost of forecast”; you can then ask a weather discovery service for a service that provides free forecasts.

The discover tactic works by reducing the dependencies between cooperating services, which should be written without knowledge of each other. This enables flexibility in the binding between services, as well as when that binding occurs.

Tailor Interface

Tailoring an interface is a tactic that adds capabilities to, or hides capabilities in, an existing interface without changing the API or implementation. Capabilities such as translation, buffering, and data smoothing can be added to an interface without changing it. An example of removing capabilities is hiding particular functions or parameters from untrusted users. A common dynamic application of this tactic is intercepting filters that add functionality such as data validation to help prevent SQL injections or other attacks, or to translate between data formats. Another example is using techniques from aspect-oriented programming that weave in preprocessing and postprocessing functionality at compile time.

The tailor interface tactic allows functionality that is needed by many services to be added or hidden based on context and managed independently. It also enables services with syntactic differences to interoperate without modification to either service.

This tactic is typically applied during integration; however, designing an architecture so that it facilitates interface tailoring can support integrability. Interface tailoring is commonly used to resolve syntactic and data semantic distance during integration. It can also be applied to resolve some forms of behavioral semantic distance, though it can be more complex to do (e.g., maintaining a complex state to accommodate protocol differences) and is perhaps more accurately categorized as introducing an intermediary.

Configure Behavior

The tactic of configuring behavior is used by software components that are implemented to be configurable in prescribed ways that allow them to more easily interact with a range of components. The behavior of a component can be configured during the build phase (recompile with a different flag), during system initialization (read a configuration file or fetch data from a database), or during runtime (specify a protocol version as part of your requests). A simple example is configuring a component to support different versions of a standard on its interfaces. Ensuring that multiple options are available increases the chances that the assumptions of S and a future C will match.

Building configurable behavior into portions of S is an integrability tactic that allows S to support a wider range of potential C s. This tactic can potentially address syntactic, data semantic, behavioral semantic, and temporal dimensions of distance.

Coordinate

Orchestrate

Orchestrate is a tactic that uses a control mechanism to coordinate and manage the invocation of particular services so that they can remain unaware of each other.

Orchestration helps with the integration of a set of loosely coupled reusable services to create a system that meets a new need. Integration costs are reduced when orchestration is included in an architecture in a way that supports the services that are likely to be integrated in the future. This tactic allows future integration activities to focus on integration with the orchestration mechanism instead of point-to-point integration with multiple components.

Workflow engines commonly make use of the orchestrate tactic. A workflow is a set of organized activities that order and coordinate software components to complete a business process. It may consist of other workflows, each of which may itself consist of aggregated services. The workflow model encourages reuse and agility, leading to more flexible business processes. Business processes can be managed under a philosophy of business process management (BPM) that views processes as a set of competitive assets to be managed. Complex orchestration can be specified in a language such as BPEL (Business Process Execution Language).

Orchestration works by reducing the number of dependencies between a system S and new components $\{C_i\}$, and eliminating altogether the explicit dependencies among the components $\{C_i\}$, by centralizing those dependencies at the orchestration mechanism. It may also reduce syntactic and data semantic distance if the orchestration mechanism is used in conjunction with tactics such as adherence to standards.

Manage Resources

A resource manager is a specific form of intermediary that governs access to computing resources; it is similar to the restrict communication paths tactic. With this tactic, software components are not allowed to directly access some computing resources (e.g., threads or blocks of memory), but instead request those resources from a resource manager. Resource managers are typically responsible for allocating resource access across multiple components in a way that preserves some invariants (e.g., avoiding resource exhaustion or concurrent use), enforces some fair access policy, or both. Examples of resource managers include operating systems, transaction mechanisms in databases, use of thread pools in enterprise systems, and use of the ARINC 653 standard for space and time partitioning in safety-critical systems.

The manage resource tactic works by reducing the resource distance between a system S and a component C , by clearly exposing the resource requirements and managing their common use.

7.4 Tactics-Based Questionnaire for Integrability

Based on the tactics described in Section 7.3, we can create a set of integrability tactics-inspired questions, as presented in Table 7.2. To gain an overview of the architectural choices made to support integrability, the analyst asks each question and records the answers in the table. The answers to these questions can then be made the focus of further activities: investigation of documentation, analysis of code or other artifacts, reverse engineering of code, and so forth.

TABLE 7.2 Tactics-Based Questionnaire for Integrability

Tactics Group	Tactics Question	Supported? (Y/N)	Risk	Design Decisions and Location	Rationale and Assumptions
Limit Dependencies	Does the system encapsulate functionality of each element by introducing explicit interfaces and requiring that all access to the elements passes through these interfaces? Does the system broadly use intermediaries for breaking dependencies between components—for example, removing a data producer's knowledge of its consumers? Does the system abstract common services, providing a general, abstract interface for similar services? Does the system provide a means to restrict communication paths between components? Does the system adhere to standards in terms of how components interact and share information with each other?				
Adapt	Does the system provide the ability to statically (i.e., at compile time) tailor interfaces—that is, the ability to add or hide capabilities of a component's interface without changing its API or implementation? Does the system provide a discovery service, cataloguing and disseminating information about services? Does the system provide a means to configure the behavior of components at build, initialization, or runtime?				

continues

TABLE 7.2 Tactics-Based Questionnaire for Integrability *continued*

Tactics Group	Tactics Question	Supported? (Y/N)	Risk	Design Decisions and Location	Rationale and Assumptions
Coordinate	Does the system include an orchestration mechanism that coordinates and manages the invocation of components so they can remain unaware of each other? Does the system provide a resource manager that governs access to computing resources?				

7.5 Patterns

The first three patterns are all centered on the tailor interface tactic, and are described here as a group:

- *Wrappers.* A wrapper is a form of encapsulation whereby some component is encased within an alternative abstraction. A wrapper is the only element allowed to use that component; every other piece of software uses the component's services by going through the wrapper. The wrapper transforms the data or control information for the component it wraps. For example, a component may expect input using Imperial measures but find itself in a system in which all of the other components produce metric measures. Wrappers can:
 - Translate an element of a component interface into an alternative element
 - Hide an element of a component interface
 - Preserve an element of a component's base interface without change
- *Bridges.* A bridge translates some “requires” assumptions of one arbitrary component to some “provides” assumptions of another component. The key difference between a bridge and a wrapper is that a bridge is independent of any particular component. Also, the bridge must be explicitly invoked by some external agent—possibly but not necessarily by one of the components the bridge spans. This last point should convey the idea that bridges are usually transient and that the specific translation is defined at the time of bridge construction (e.g., bridge compile time). The significance of both of these distinctions will be made clear in the discussion of mediators.

Bridges typically focus on a narrower range of interface translations than do wrappers because bridges address specific assumptions. The more assumptions a bridge tries to address, the fewer components to which it applies.

- *Mediators.* Mediators exhibit properties of both bridges and wrappers. The major distinction between bridges and mediators, is that mediators incorporate a planning function that results in runtime determination of the translation, whereas bridges establish this translation at bridge construction time.

A mediator is also similar to a wrapper insofar as it becomes an explicit component in the system architecture. That is, semantically primitive, often transient bridges can be thought of as incidental repair mechanisms whose role in a design can remain implicit. In contrast, mediators have sufficient semantic complexity and runtime autonomy (persistence) to play a first-class role in a software architecture.

Benefits:

- All three patterns allow access to an element without forcing a change to the element or its interface.

Tradeoffs:

- Creating any of the patterns requires up-front development work.
- All of the patterns will introduce some performance overhead while accessing the element, although typically this overhead is small.

Service-Oriented Architecture Pattern

The service-oriented architecture (SOA) pattern describes a collection of distributed components that provide and/or consume services. In an SOA, *service provider* components and *service consumer* components can use different implementation languages and platforms. Services are largely standalone entities: Service providers and service consumers are usually deployed independently, and often belong to different systems or even different organizations. Components have interfaces that describe the services they request from other components and the services they provide. A service's quality attributes can be specified and guaranteed with a service level agreement (SLA), which may sometimes be legally binding. Components perform their computations by requesting services from one another. Communication among the services is typically performed by using web services standards such as WSDL (Web Services Description Language) or SOAP (Simple Object Access Protocol).

The SOA pattern is related to the microservice architecture pattern (see Chapter 5). Microservice architectures are assumed to compose a single system and be managed by a single organization, however, whereas SOAs provide reusable components that are assumed to be heterogeneous and managed by distinct organizations.

Benefits:

- Services are designed to be used by a variety of clients, leading them to be more generic. Many commercial organizations will provide and market their service with the goal of broad adoption.
- Services are independent. The only method for accessing a service is through its interface and through messages over a network. Consequently, a service and the rest of the system do not interact, except through their interfaces.

- Services can be implemented heterogeneously, using whatever languages and technologies are most appropriate.

Tradeoffs:

- SOAs, because of their heterogeneity and distinct ownership, come with a great many interoperability features such as WSDL and SOAP. This adds complexity and overhead.

Dynamic Discovery

Dynamic discovery applies the discovery tactic to enable the discovery of service providers at runtime. Consequently, a runtime binding can occur between a service consumer and a concrete service.

Use of a dynamic discovery capability sets the expectation that the system will clearly advertise both the services available for integration with future components and the minimal information that will be available for each service. The specific information available will vary, but typically comprises data that can be mechanically searched during discovery and runtime integration (e.g., identifying a specific version of an interface standard by string match).

Benefits:

- This pattern allows for flexibility in binding services together into a cooperating whole. For example, services may be chosen at startup or runtime based on their pricing or availability.

Tradeoffs:

- Dynamic discovery registration and de-registration must be automated, and tools for this purpose must be acquired or generated.

7.6 For Further Reading

Much of the material for this chapter was inspired by and drawn from [Kazman 20a].

An in-depth discussion of the quality attribute of integrability can be found in [Henttonen 07].

[MacCormack 06] and [Mo 16] define and provide empirical evidence for architecture-level coupling metrics, which can be useful in measuring designs for integrability.

The book *Design Patterns: Elements of Reusable Object-Oriented Software* [Gamma 94] defines and distinguishes the bridge, wrapper, and adapter patterns.

7.7 Discussion Questions

1. Think about an integration that you have done in the past—perhaps integrating a library or a framework into your code. Identify the various “distances” that you had to deal with, as discussed in Section 7.1. Which of these required the greatest effort to resolve?
2. Write a concrete integrability scenario for a system that you are working on (perhaps an exploratory scenario for some component that you are considering integrating).
3. Which of the integrability tactics do you think would be the easiest to implement in practice, and why? Which would be the most difficult, and why?
4. Many of the integrability tactics are similar to the modifiability tactics. If you make your system highly modifiable, does that automatically mean that it will be easy to integrate into another context?
5. A standard use of SOA is to add a shopping cart feature to an e-commerce site. Which commercially available SOA platforms provide different shopping cart services? What are the attributes of the shopping carts? Can these attributes be discovered at runtime?
6. Write a program that accesses the Google Play Store, via its API, and returns a list of weather forecasting applications and their attributes.
7. Sketch a design for a dynamic discovery service. Which types of distances does this service help to mitigate?

This page intentionally left blank

8



Modifiability

It is not the strongest of the species that survive, nor the most intelligent, but the one most responsive to change.

—Charles Darwin

Change happens.

Study after study shows that most of the cost of the typical software system occurs *after* it has been initially released. If change is the only constant in the universe, then software change is not only constant but ubiquitous. Changes happen to add new features, to alter or even retire old ones. Changes happen to fix defects, tighten security, or improve performance. Changes happen to enhance the user's experience. Changes happen to embrace new technology, new platforms, new protocols, new standards. Changes happen to make systems work together, even if they were never designed to do so.

Modifiability is about change, and our interest in it is to lower the cost and risk of making changes. To plan for modifiability, an architect has to consider four questions:

- *What can change?* A change can occur to any aspect of a system: the functions that the system computes, the platform (the hardware, operating system, middleware), the environment in which the system operates (the systems with which it must interoperate, the protocols it uses to communicate with the rest of the world), the qualities the system exhibits (its performance, its reliability, and even its future modifications), and its capacity (number of users supported, number of simultaneous operations).
- *What is the likelihood of the change?* One cannot plan a system for all potential changes—the system would never be done or if it was done it would be far too expensive and would likely suffer quality attribute problems in other dimensions. Although anything *might* change, the architect has to make the tough decisions about which changes are likely, and hence which changes will be supported and which will not.
- *When is the change made and who makes it?* Most commonly in the past, a change was made to source code. That is, a developer had to make the change, which was tested and then deployed in a new release. Now, however, the question of when a change is made is intertwined with the question of who makes it. An end user changing the screen saver is clearly making a change to one aspect of the system. Equally clear, it is not in the

same category as changing the system so that it uses a different database management system. Changes can be made to the implementation (by modifying the source code), during compilation (using compile-time switches), during the build (by choice of libraries), during configuration setup (by a range of techniques, including parameter setting), or during execution (by parameter settings, plug-ins, allocation to hardware, and so forth). A change can also be made by a developer, an end user, or a system administrator. Systems that learn and adapt supply a whole different answer to the question of when a change is made and “who” makes it—it is the system itself that is the agent for change.

- *What is the cost of the change?* Making a system more modifiable involves two types of costs:
 - The cost of introducing the mechanism(s) to make the system more modifiable
 - The cost of making the modification using the mechanism(s)

For example, the simplest mechanism for making a change is to wait for a change request to come in, then change the source code to accommodate the request. In such a case, the cost of introducing the mechanism is zero (since there is no special mechanism); the cost of exercising it is the cost of changing the source code and revalidating the system.

Toward the other end of the spectrum is an application generator, such as a user interface builder. The builder takes as input a description of the designed UI produced through direct manipulation techniques and which may then produce source code. The cost of introducing the mechanism is the cost of acquiring the UI builder, which may be substantial. The cost of using the mechanism is the cost of producing the input to feed the builder (this cost can be either substantial or negligible), the cost of running the builder (close to zero), and finally the cost of whatever testing is performed on the result (usually much less than for hand-coding).

Still further along the spectrum are software systems that discover their environments, learn, and modify themselves to accommodate any changes. For those systems, the cost of making the modification is zero, but that ability was purchased along with implementing and testing the learning mechanisms, which may have been quite costly.

For N similar modifications, a simplified justification for a change mechanism is that

$$N * \text{Cost of making change without the mechanism} \leq \\ \text{Cost of creating the mechanism} + (N * \text{cost of making the change using the mechanism})$$

Here, N is the anticipated number of modifications that will use the modifiability mechanism—but it is also a prediction. If fewer changes than expected come in, then an expensive modification mechanism may not be warranted. In addition, the cost of creating the modifiability mechanism could be applied elsewhere (opportunity cost)—in adding new functionality, in improving the performance, or even in non-software investments such as hiring or training. Also, the equation does not take time into account. It might be cheaper in the long run to build a sophisticated change-handling mechanism, but you might not be able to wait for its completion. However, if your code is modified frequently, not introducing some architectural mechanism and simply piling change on top of change typically leads to substantial technical debt. We address the topic of architectural debt in Chapter 23.

Change is so prevalent in the life of software systems that special names have been given to specific flavors of modifiability. Some of the common ones are highlighted here:

- *Scalability* is about accommodating more of something. In terms of performance, scalability means adding more resources. Two kinds of performance scalability are horizontal scalability and vertical scalability. Horizontal scalability (scaling out) refers to adding more resources to logical units, such as adding another server to a cluster of servers. Vertical scalability (scaling-up) refers to adding more resources to a physical unit, such as adding more memory to a single computer. The problem that arises with either type of scaling is how to effectively utilize the additional resources. Being *effective* means that the additional resources result in a measurable improvement of some system quality, did not require undue effort to add, and did not unduly disrupt operations. In cloud-based environments, horizontal scalability is called *elasticity*. Elasticity is a property that enables a customer to add or remove virtual machines from the resource pool (see Chapter 17 for further discussion of such environments).
- *Variability* refers to the ability of a system and its supporting artifacts, such as code, requirements, test plans, and documentation, to support the production of a set of variants that differ from each other in a preplanned fashion. Variability is an especially important quality attribute in a product line, which is a family of systems that are similar but vary in features and functions. If the engineering assets associated with these systems can be shared among members of the family, then the overall cost of the product line plummets. This is achieved by introducing mechanisms that allow the artifacts to be selected and/or adapt to usages in the different product contexts that are within the product line's scope. The goal of variability in a software product line is to make it easy to build and maintain products in that family over a period of time.
- *Portability* refers to the ease with which software that was built to run on one platform can be changed to run on a different platform. Portability is achieved by minimizing platform dependencies in the software, isolating dependencies to well-identified locations, and writing the software to run on a “virtual machine” (for example, a Java Virtual Machine) that encapsulates all the platform dependencies. Scenarios describing portability deal with moving software to a new platform by expending no more than a certain level of effort or by counting the number of places in the software that would have to change. Architectural approaches to dealing with portability are intertwined with those for *deployability*, a topic addressed in Chapter 5.
- *Location independence* refers to the case where two pieces of distributed software interact and the location of one or both of the pieces is not known prior to runtime. Alternatively, the location of these pieces may change during runtime. In distributed systems, services are often deployed to arbitrary locations, and clients of those services must discover their location dynamically. In addition, services in a distributed system must often make their location discoverable once they have been deployed to a location. Designing the system for location independence means that the location will be easy to modify with minimal impact on the rest of the system.

8.1 Modifiability General Scenario

From these considerations, we can construct the general scenario for modifiability. Table 8.1 summarizes this scenario.

TABLE 8.1 General Scenario for Modifiability

Portion of Scenario	Description	Possible Values
Source	The agent that causes a change to be made. Most are human actors, but the system might be one that learns or self-modifies, in which case the source is the system itself.	End user, developer, system administrator, product line owner, the system itself
Stimulus	The change that the system needs to accommodate. (For this categorization, we regard fixing a defect as a change, to something that presumably wasn't working correctly.)	A directive to add/delete/modify functionality, or change a quality attribute, capacity, platform, or technology; a directive to add a new product to a product line; a directive to change the location of a service to another location
Artifacts	The artifacts that are modified. Specific components or modules, the system's platform, its user interface, its environment, or another system with which it interoperates.	Code, data, interfaces, components, resources, test cases, configurations, documentation
Environment	The time or stage at which the change is made.	Runtime, compile time, build time, initiation time, design time
Response	Make the change and incorporate it into the system.	One or more of the following: ▪ Make modification ▪ Test modification ▪ Deploy modification ▪ Self-modify
Response measure	The resources that were expended to make the change.	Cost in terms of: ▪ Number, size, complexity of affected artifacts ▪ Effort ▪ Elapsed time ▪ Money (direct outlay or opportunity cost) ▪ Extent to which this modification affects other functions or quality attributes ▪ New defects introduced ▪ How long it took the system to adapt

Figure 8.1 illustrates a concrete modifiability scenario: *A developer wishes to change the user interface. This change will be made to the code at design time, it will take less than three hours to make and test the change, and no side effects will occur.*

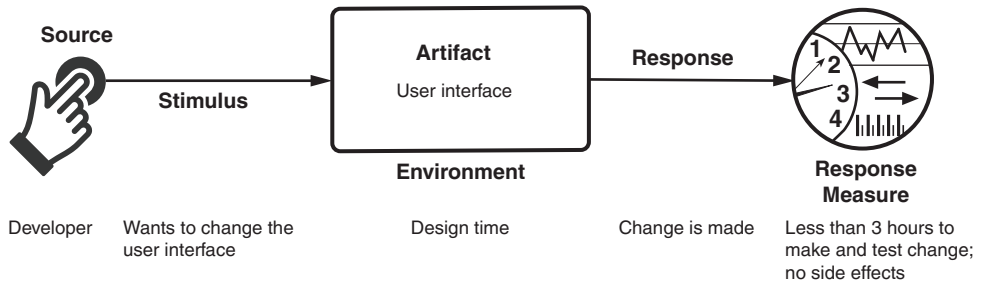


FIGURE 8.1 Sample concrete modifiability scenario

8.2 Tactics for Modifiability

Tactics to control modifiability have as their goal controlling the complexity of making changes, as well as the time and cost to make changes. Figure 8.2 shows this relationship.

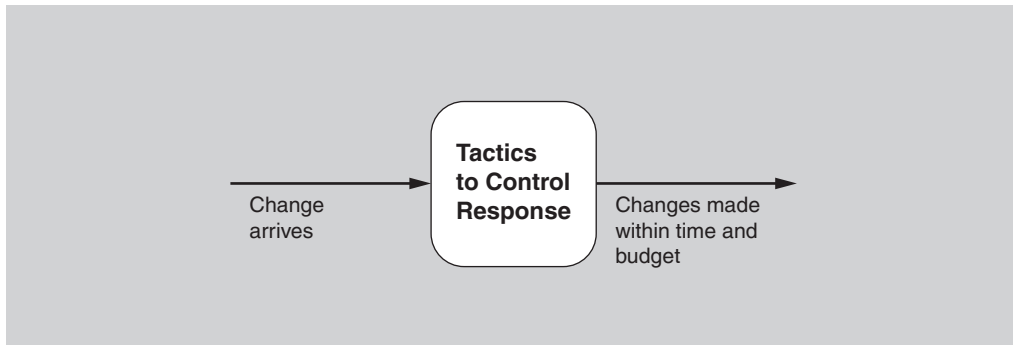


FIGURE 8.2 Goal of modifiability tactics

To understand modifiability, we begin with some of the earliest and most fundamental complexity measures of software design—coupling and cohesion—which were first described in the 1960s.

Generally, a change that affects one module is easier and less expensive than a change that affects more than one module. However, if two modules' responsibilities overlap in some way,

then a single change may well affect them both. We can quantify this overlap by measuring the probability that a modification to one module will propagate to the other. This relationship is called *coupling*, and high coupling is an enemy of modifiability. Reducing the coupling between two modules will decrease the expected cost of any modification that affects either one. Tactics that reduce coupling are those that place intermediaries of various sorts between the two otherwise highly coupled modules.

Cohesion measures how strongly the responsibilities of a module are related. Informally, it measures the module's "unity of purpose." Unity of purpose can be measured by the change scenarios that affect a module. The cohesion of a module is the probability that a change scenario that affects a responsibility will also affect other (different) responsibilities. The higher the cohesion, the lower the probability that a given change will affect multiple modules. High cohesion is good for modifiability; low cohesion is bad for it. If module A has a low cohesion, then cohesion can be improved by removing responsibilities unaffected by anticipated changes.

A third characteristic that affects the cost and complexity of a change is the *size of a module*. All other things being equal, larger modules are more difficult and more costly to change, and are more prone to have bugs.

Finally, we need to be concerned with the point in the software development life cycle where a change occurs. If we ignore the cost of preparing the architecture for the modification, we prefer that a change is bound as late as possible. Changes can be successfully made (i.e., quickly and at low cost) late in the life cycle only if the architecture is suitably prepared to accommodate them. Thus the fourth and final parameter in a model of modifiability is *binding time of modification*. An architecture that is suitably equipped to accommodate modifications late in the life cycle will, on average, cost less than an architecture that forces the same modification to be made earlier. The preparedness of the system means that some costs will be zero, or very low, for modifications that occur late in the life cycle.

Now we can understand tactics and their consequences as affecting one or more of these parameters: reducing size, increasing cohesion, reducing coupling, and deferring binding time. These tactics are shown in Figure 8.3.

Increase Cohesion

Several tactics involve redistributing responsibilities among modules. This step is taken to reduce the likelihood that a single change will affect multiple modules.

- *Split module*. If the module being modified includes responsibilities that are not cohesive, the modification costs will likely be high. Refactoring the module into several more cohesive modules should reduce the average cost of future changes. Splitting a module should not simply consist of placing half of the lines of code into each submodule; instead, it should sensibly and appropriately result in a series of submodules that are cohesive on their own.
- *Redistribute responsibilities*. If responsibilities A, A', and A'' (all similar responsibilities) are sprinkled across several distinct modules, they should be placed together. This

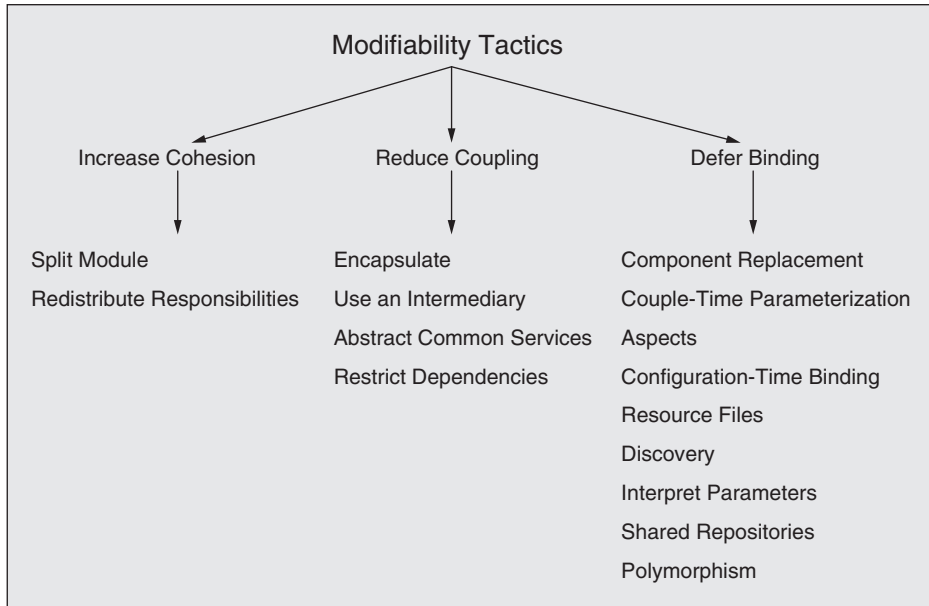


FIGURE 8.3 Modifiability tactics

refactoring may involve creating a new module, or it may involve moving responsibilities to existing modules. One method for identifying responsibilities to be moved is to hypothesize a set of likely changes as scenarios. If the scenarios consistently affect just one part of a module, then perhaps the other parts have separate responsibilities and should be moved. Alternatively, if some scenarios require modifications to multiple modules, then perhaps the responsibilities affected should be grouped together into a new module.

Reduce Coupling

We now turn to tactics that reduce the coupling between modules. These tactics overlap with the integrability tactics described in Chapter 7, because reducing dependencies among independent components (for integrability) is similar to reducing coupling among modules (for modifiability).

- *Encapsulate*. See the discussion in Chapter 7.
- *Use an intermediary*. See the discussion in Chapter 7.
- *Abstract common services*. See the discussion in Chapter 7.

- *Restrict dependencies.* This tactic restricts which modules a given module interacts with or depends on. In practice, this tactic is implemented by restricting a module's visibility (when developers cannot see an interface, they cannot employ it) and by authorization (restricting access to only authorized modules). The restrict dependencies tactic is seen in layered architectures, in which a layer is allowed to use only lower layers (sometimes only the next lower layer), and with the use of wrappers, where external entities can see (and hence depend on) only the wrapper, and not the internal functionality that it wraps.

Defer Binding

Because the work of people is almost always more expensive error-prone than the work of computers, letting computers handle a change as much as possible will almost always reduce the cost of making that change. If we design artifacts with built-in flexibility, then exercising that flexibility is usually cheaper than hand-coding a specific change.

Parameters are perhaps the best-known mechanism for introducing flexibility, and their use is reminiscent of the abstract common services tactic. A parameterized function $f(a, b)$ is more general than the similar function $f(a)$ that assumes $b = 0$. When we bind the value of some parameters at a different phase in the life cycle than the one in which we defined the parameters, we are deferring binding.

In general, the later in the life cycle we can bind values, the better. However, putting the mechanisms in place to facilitate that late binding tends to be more expensive—a well-known tradeoff. And so the equation given earlier in the chapter comes into play. We want to bind as late as possible, as long as the mechanism that allows it is cost-effective.

The following tactics can be used to bind values at compile time or build time:

- Component replacement (for example, in a build script or makefile)
- Compile-time parameterization
- Aspects

The following tactics are available to bind values at deployment, startup time, or initialization time:

- Configuration-time binding
- Resource files

Tactics to bind values at runtime include the following:

- Discovery (see Chapter 7)
- Interpret parameters
- Shared repositories
- Polymorphism

Separating the building of a mechanism for modifiability from the use of that mechanism to make a modification admits the possibility of different stakeholders being involved—one stakeholder (usually a developer) to provide the mechanism and another stakeholder (an administrator or installer) to exercise it later, possibly in a completely different life-cycle

phase. Installing a mechanism so that someone else can make a change to the system without having to change any code is sometimes called *externalizing* the change.

8.3 Tactics-Based Questionnaire for Modifiability

Based on the tactics described in Section 8.2, we can create a set of tactics-inspired questions, as presented in Table 8.2. To gain an overview of the architectural choices made to support modifiability, the analyst asks each question and records the answers in the table. The answers to these questions can then be made the focus of further activities: investigation of documentation, analysis of code or other artifacts, reverse engineering of code, and so forth.

TABLE 8.2 Tactics-Based Questionnaire for Modifiability

Tactics Group	Tactics Question	Supported? (Y/N)	Risk?	Design Decisions and Location	Rationale and Assumptions
Increase Cohesion	Do you make modules more cohesive by splitting the module ? For example, if you have a large, complex module, can you split it into two (or more) more cohesive modules? Do you make modules more cohesive by redistributing responsibilities ? For example, if responsibilities in a module do not serve the same purpose, they should be placed in other modules.				
Reduce Coupling	Do you consistently encapsulate functionality? This typically involves isolating the functionality under scrutiny and introducing an explicit interface to it. Do you consistently use an intermediary to keep modules from being too tightly coupled? For example, if A calls concrete functionality C, you might introduce an abstraction B that mediates between A and C. Do you restrict dependencies between modules in a systematic way? Or is any system module free to interact with any other module?				

continues

TABLE 8.2 Tactics-Based Questionnaire for Modifiability *continued*

Tactics Group	Tactics Question	Supported? (Y/N)	Risk?	Design Decisions and Location	Rationale and Assumptions
Reduce Coupling	Do you abstract common services , in cases where you are providing several similar services? For example, this technique is often used when you want your system to be portable across operating systems, hardware, or other environmental variations.				
Defer Binding	Does the system regularly defer binding of important functionality so that it can be replaced later in the life cycle? For example, are there plug-ins, add-ons, resource files, or configuration files that can extend the functionality of the system?				

8.4 Patterns

Patterns for modifiability divide the system into modules in such a way that the modules can be developed and evolved separately with little interaction among them, thereby supporting portability, modifiability, and reuse. There are probably more patterns designed to support modifiability than for any other quality attribute. We present a few that are among the most commonly used here.

Client-Server Pattern

The client-server pattern consists of a server providing services simultaneously to multiple distributed clients. The most common example is a web server providing information to multiple simultaneous users of a website.

The interactions between a server and its clients follow this sequence:

- *Discovery*:
 - Communication is initiated by a client, which uses a discovery service to determine the location of the server.
 - The server responds to the client using an agreed-upon protocol.

- *Interaction:*

- The client sends requests to the server.
- The server processes the requests and responds.

Several points about this sequence are worth noting:

- The server may have multiple instances if the number of clients grows beyond the capacity of a single instance.
- If the server is stateless with respect to the clients, each request from a client is treated independently.
- If the server maintains state with respect to the clients, then:
 - Each request must identify the client in some fashion.
 - The client should send an “end of session” message so that the server can remove resources associated with that particular client.
 - The server may time out if the client has not sent a request in a specified time so that resources associated with the client can be removed.

Benefits:

- The connection between a server and its clients is established dynamically. The server has no a priori knowledge of its clients—that is, there is low coupling between the server and its clients.
- There is no coupling among the clients.
- The number of clients can easily scale and is constrained only by the capacity of the server. The server functionality can also scale if its capacity is exceeded.
- Clients and servers can evolve independently.
- Common services can be shared among multiple clients.
- The interaction with a user is isolated to the client. This factor has resulted in the development of specialized languages and tools for managing the user interface.

Tradeoffs:

- This pattern is implemented such that communication occurs over a network, perhaps even the Internet. Thus messages may be delayed by network congestion, leading to degradation (or at least unpredictability) of performance.
- For clients that communicate with servers over a network shared by other applications, special provisions must be made for achieving security (especially confidentiality) and maintaining integrity.

Plug-in (Microkernel) Pattern

The plug-in pattern has two types of elements—elements that provide a core set of functionality and specialized variants (called plug-ins) that add functionality to the core via a fixed set of interfaces. The two types are typically bound together at build time or later.

Examples of usage include the following cases:

- The core functionality may be a stripped-down operating system (the microkernel) that provides the mechanisms needed to implement operating system services, such as low-level address space management, thread management, and interprocess communication (IPC). The plug-ins provide the actual operating system functionality, such as device drivers, task management, and I/O request management.
- The core functionality is a product providing services to its users. The plug-ins provide portability, such as operating system compatibility or supporting library compatibility. The plug-ins can also provide additional functionality not included in the core product. In addition, they can act as adapters to enable integration with external systems (see Chapter 7).

Benefits:

- Plug-ins provide a controlled mechanism to extend a core product and make it useful in a variety of contexts.
- The plug-ins can be developed by different teams or organizations than the developers of the microkernel. This allows for the development of two different markets: for the core product and for the plug-ins.
- The plug-ins can evolve independently from the microkernel. Since they interact through fixed interfaces, as long as the interfaces do not change, the two types of elements are not otherwise coupled.

Tradeoffs:

- Because plug-ins can be developed by different organizations, it is easier to introduce security vulnerabilities and privacy threats.

Layers Pattern

The layers pattern divides the system in such a way that the modules can be developed and evolved separately with little interaction among the parts, which supports portability, modifiability, and reuse. To achieve this separation of concerns, the layers pattern divides the software into units called layers. Each layer is a grouping of modules that offers a cohesive set of services. The *allowed-to-use* relationship among the layers is subject to a key constraint: The relations must be unidirectional.

Layers completely partition a set of software, and each partition is exposed through a public interface. The layers are created to interact according to a strict ordering relation. If (A, B) is in this relation, we say that the software assigned to layer A is allowed to use any of the public facilities provided by layer B. (In a vertically arranged representation of layers, which is almost ubiquitous, A will be drawn higher than B.) In some cases, modules in one layer are required to directly use modules in a nonadjacent lower layer, although normally only next-lower-layer uses are allowed. This case of software in a higher layer using modules in a nonadjacent lower layer is called *layer bridging*. Upward usages are not allowed in this pattern.

Benefits:

- Because a layer is constrained to use only lower layers, software in lower layers can be changed (as long as the interface does not change) without affecting the upper layers.
- Lower-level layers may be reused across different applications. For example, suppose a certain layer allows portability across operating systems. This layer would be useful in any system that must run on multiple, different operating systems. The lowest layers are often provided by commercial software—an operating system, for example, or network communications software.
- Because the *allowed-to-use* relations are constrained, the number of interfaces that any team must understand is reduced.

Tradeoffs:

- If the layering is not designed correctly, it may actually get in the way, by not providing the lower-level abstractions that programmers at the higher levels need.
- Layering often adds a performance penalty to a system. If a call is made from a function in the top-most layer, it may have to traverse many lower layers before being executed by the hardware.
- If many instances of layer bridging occur, the system may not meet its portability and modifiability goals, which strict layering helps to achieve.

Publish-Subscribe Pattern

Publish-subscribe is an architectural pattern in which components communicate primarily through asynchronous messages, sometimes referred to as “events” or “topics.” The publishers have no knowledge of the subscribers, and subscribers are only aware of message types. Systems using the publish-subscribe pattern rely on implicit invocation; that is, the component publishing a message does not directly invoke any other component. Components publish messages on one or more events or topics, and other components register an interest in the publication. At runtime, when a message is published, the publish–subscribe (or event) bus notifies all of the elements that registered an interest in the event or topic. In this way, the message publication causes an implicit invocation of (methods in) other components. The result is loose coupling between the publishers and the subscribers.

The publish-subscribe pattern has three types of elements:

- *Publisher component.* Sends (publishes) messages.
- *Subscriber component.* Subscribes to and then receives messages.
- *Event bus.* Manages subscriptions and message dispatch as part of the runtime infrastructure.

Benefits:

- Publishers and subscribers are independent and hence loosely coupled. Adding or changing subscribers requires only registering for an event and causes no changes to the publisher.

- System behavior can be easily changed by changing the event or topic of a message being published, and consequently which subscribers might receive and act on this message. This seemingly small change can have large consequences, as features may be turned on or off by adding or suppressing messages.
- Events can be logged easily to allow for record and playback and thereby reproduce error conditions that can be challenging to recreate manually.

Tradeoffs:

- Some implementations of the publish-subscribe pattern can negatively impact performance (latency). Use of a distributed coordination mechanism will ameliorate the performance degradation.
- In some cases, a component cannot be sure how long it will take to receive a published message. In general, system performance and resource management are more difficult to reason about in publish-subscribe systems.
- Use of this pattern can negatively impact the determinism produced by synchronous systems. The order in which methods are invoked, as a result of an event, can vary in some implementations.
- Use of the publish-subscribe pattern can negatively impact testability. Seemingly small changes in the event bus—such as a change in which components are associated with which events—can have a wide impact on system behavior and quality of service.
- Some publish-subscribe implementations limit the mechanisms available to flexibly implement security (integrity). Since publishers do not know the identity of their subscribers, and vice versa, end-to-end encryption is limited. Messages from a publisher to the event bus can be uniquely encrypted, and messages from the event bus to a subscriber can be uniquely encrypted; however, any end-to-end encrypted communication requires all publishers and subscribers involved to share the same key.

8.5 For Further Reading

Serious students of software engineering and its history should read two early papers about designing for modifiability. The first is Edsger Dijkstra's 1968 paper about the T.H.E. operating system, which is the first paper that talks about designing systems to use layers, and the modifiability benefits that this approach brings [Dijkstra 68]. The second is David Parnas's 1972 paper that introduced the concept of information hiding. [Parnas 72] suggested defining modules not by their functionality, but by their ability to internalize the effects of changes.

More patterns for modifiability are given in *Software Systems Architecture: Working With Stakeholders Using Viewpoints and Perspectives* [Woods 11].

The Decoupling Level metric [Mo 16] is an architecture-level coupling metric that can give insights into how globally coupled an architecture is. This information can be used to track coupling over time, as an early warning indicator of technical debt.

A fully automated way of detecting modularity violations—and other kinds of design flaws—has been described in [Mo 19]. The detected violations can be used as a guide to refactoring, so as to increase cohesion and reduce coupling.

Software modules intended for use in a software product line are often imbued with variation mechanisms that allow them to be quickly modified to serve in different applications—that is, in different members of the product line. Lists of variation mechanisms for components in a product line can be found in the works by Bachmann and Clements [Bachmann 05], Jacobson and colleagues [Jacobson 97], and Anastasopoulos and colleagues [Anastasopoulos 00].

The layers pattern comes in many forms and variations—“layers with a sidecar,” for example. Section 2.4 of [DSA2] sorts them all out, and discusses why (surprisingly for an architectural pattern invented more than a half-century ago) most layer diagrams for software that you’ve ever seen are very ambiguous. If you don’t want to spring for the book, then [Bachmann 00a] is a good substitute.

8.6 Discussion Questions

1. Modifiability comes in many flavors and is known by many names; we discussed a few in the opening section of this chapter, but that discussion only scratches the surface. Find one of the IEEE or ISO standards dealing with quality attributes, and compile a list of quality attributes that refer to some form of modifiability. Discuss the differences.
2. In the list you compiled for question 1, which tactics and patterns are especially helpful for each?
3. For each quality attribute that you discovered as a result of question 2, write a modifiability scenario that expresses it.
4. In many laundromats, washing machines and dryers accept coins but do not give change. Instead, separate machines dispense change. In an average laundromat, there are six or eight washers and dryers for every change machine. What modifiability tactics do you see at work in this arrangement? What can you say about availability?
5. For the laundromat in question 4, describe the specific form of modifiability (using a modifiability scenario) that seems to be the aim of arranging the machines as described.
6. A *wrapper*, introduced in Chapter 7, is a common architectural pattern to aid modifiability. Which modifiability tactics does a wrapper embody?
7. Other common architectural patterns that can increase a system’s modifiability include blackboard, broker, peer-to-peer, model-view-controller, and reflection. Discuss each in terms of the modifiability tactics it packages.
8. Once an intermediary has been introduced into an architecture, some modules may attempt to circumvent it, either inadvertently (because they are not aware of the

intermediary) or intentionally (for performance, for convenience, or out of habit). Discuss some architectural means to prevent an undesirable circumvention of an intermediary. Discuss some non-architectural means as well.

9. The abstract common services tactic is intended to reduce coupling but might also reduce cohesion. Discuss.
10. Discuss the proposition that the client-server pattern is the microkernel pattern with runtime binding.

9



Performance

An ounce of performance is worth pounds of promises.
—Mae West

It's about time.

Performance, that is: It's about time and the software system's ability to meet timing requirements. The melancholy fact is that operations on computers take time. Computations take time on the order of thousands of nanoseconds, disk access (whether solid state or rotating) takes time on the order of tens of milliseconds, and network access takes time ranging from hundreds of microseconds within the same data center to upward of 100 milliseconds for intercontinental messages. Time must be taken into consideration when designing your system for performance.

When events occur—interrupts, messages, requests from users or other systems, or clock events marking the passage of time—the system, or some element of the system, must respond to them in time. Characterizing the events that can occur (and when they can occur) and the system's or element's time-based response to those events is the essence of discussing performance.

Web-based system events come in the form of requests from users (numbering in the tens or tens of millions) via their clients such as web browsers. Services get events from other services. In a control system for an internal combustion engine, events come from the operator's controls and the passage of time; the system must control both the firing of the ignition when a cylinder is in the correct position and the mixture of the fuel to maximize power and efficiency and minimize pollution.

For a web-based system, a database-centric system, or a system processing input signals from its environment, the desired response might be expressed as the number of requests that can be processed in a unit of time. For the engine control system, the response might be the allowable variation in the firing time. In each case, the pattern of events arriving and the pattern of responses can be characterized, and this characterization forms the language with which to construct performance scenarios.

For much of the history of software engineering, which began when computers were slow and expensive and the tasks to perform dwarfed the ability to do them, performance has been the driving factor in architecture. As such, it has frequently compromised the achievement of all other qualities. As the price/performance ratio of hardware continues to plummet and the cost of developing software continues to rise, other qualities have emerged as important competitors to performance.

But performance remains of fundamental importance. There are still (and will likely always be) important problems that we know *how* to solve with computers, but that we can't solve fast enough to be useful.

All systems have performance requirements, even if they are not expressed. For example, a word processing tool may not have any explicit performance requirement, but no doubt you would agree that waiting an hour (or a minute, or a second) before seeing a typed character appear on the screen is unacceptable. Performance continues to be a fundamentally important quality attribute for all software.

Performance is often linked to scalability—that is, increasing your system's capacity for work, while still performing well. They're certainly linked, although technically scalability is making your system easy to change in a particular way, and so is a kind of modifiability, as discussed in Chapter 8. In addition, scalability of services in the cloud is discussed explicitly in Chapter 17.

Often, performance improvement happens after you have constructed a version of your system and found its performance to be inadequate. You can anticipate this by architecting your system with performance in mind. For example, if you have designed the system with a scalable resource pool, and you subsequently determine that this pool is a bottleneck (from your instrumented data), then you can easily increase the size of the pool. If not, your options are limited—and mostly all bad—and they may involve considerable rework.

It is not useful to spend a lot of your time optimizing a portion of the system that is responsible for only a small percentage of the total time. Instrumenting the system by logging timing information will help you determine where the actual time is spent and allow you to focus on improving the performance of critical portions of the system.

9.1 Performance General Scenario

A performance scenario begins with an event arriving at the system. Responding correctly to the event requires resources (including time) to be consumed. While this is happening, the system may be simultaneously servicing other events.

Concurrency

Concurrency is one of the more important concepts that an architect must understand and one of the least-taught topics in computer science courses. Concurrency refers to operations occurring in parallel. For example, suppose there is a thread that executes the statements

```
x = 1;  
x++;
```

and another thread that executes the same statements. What is the value of *x* after both threads have executed those statements? It could be either 2 or 3. I leave it to you to figure out how the value 3 could occur—or should I say I interleave it to you?

Concurrency occurs anytime your system creates a new thread, because threads, by definition, are independent sequences of control. Multitasking on your system is supported by independent threads. Multiple users are simultaneously supported on your system through the use of threads. Concurrency also occurs anytime your system is executing on more than one processor, whether those processors are packaged separately or as multi-core processors. In addition, you must consider concurrency when you use parallel algorithms, parallelizing infrastructures such as map-reduce, or NoSQL databases, or when you use one of a variety of concurrent scheduling algorithms. In other words, concurrency is a tool available to you in many ways.

Concurrency, when you have multiple CPUs or wait states that can exploit it, is a good thing. Allowing operations to occur in parallel improves performance, because delays introduced in one thread allow the processor to progress on another thread. But because of the interleaving phenomenon just described (referred to as a *race condition*), concurrency must also be carefully managed.

As our example shows, race conditions can occur when two threads of control are present and there is shared state. The management of concurrency frequently comes down to managing how state is shared. One technique for preventing race conditions is to use locks to enforce sequential access to state. Another technique is to partition the state based on the thread executing a portion of code. That is, if we have two instances of *x*, *x* is not shared by the two threads and no race condition will occur.

Race conditions are among the hardest types of bugs to discover; the occurrence of the bug is sporadic and depends on (possibly minute) differences in timing. I once had a race condition in an operating system that I could not track down. I put a test in the code so that the next time the race condition occurred, a debugging process was triggered. It took more than a year for the bug to recur so that the cause could be determined.

Do not let the difficulties associated with concurrency dissuade you from utilizing this very important technique. Just use it with the knowledge that you must carefully identify critical sections in your code and ensure (or take actions to ensure) that race conditions will not occur in those sections.

—LB

Table 9.1 summarizes the general scenario for performance.

TABLE 9.1 Performance General Scenario

Portion of Scenario	Description	Possible Values
Source	The stimulus can come from a user (or multiple users), from an external system, or from some portion of the system under consideration.	External: ▪ User request ▪ Request from external system ▪ Data arriving from a sensor or other system Internal: ▪ One component may make a request of another component. ▪ A timer may generate a notification.
Stimulus	The stimulus is the arrival of an event. The event can be a request for service or a notification of some state of either the system under consideration or an external system.	Arrival of a periodic, sporadic, or stochastic event: ▪ A periodic event arrives at a predictable interval. ▪ A stochastic event arrives according to some probability distribution. ▪ A sporadic event arrives according to a pattern that is neither periodic nor stochastic.
Artifact	The artifact stimulated may be the whole system or just a portion of the system. For example, a power-on event may stimulate the whole system. A user request may arrive at (stimulate) the user interface.	▪ Whole system ▪ Component within the system
Environment	The state of the system or component when the stimulus arrives. Unusual modes—error mode, overloaded mode—will affect the response. For example, three unsuccessful login attempts are allowed before a device is locked out.	Runtime. The system or component can be operating in: ▪ Normal mode ▪ Emergency mode ▪ Error correction mode ▪ Peak load ▪ Overload mode ▪ Degraded operation mode ▪ Some other defined mode of the system
Response	The system will process the stimulus. Processing the stimulus will take time. This time may be required for computation, or it may be required because processing is blocked by contention for shared resources. Requests can fail to be satisfied because the system is overloaded or because of a failure somewhere in the processing chain.	▪ System returns a response ▪ System returns an error ▪ System generates no response ▪ System ignores the request if overloaded ▪ System changes the mode or level of service ▪ System services a higher-priority event ▪ System consumes resources

Portion of Scenario	Description	Possible Values
Response measure	Timing measures can include latency or throughput. Systems with timing deadlines can also measure jitter of response and ability to meet the deadlines. Measuring how many of the requests go unsatisfied is also a type of measure, as is how much of a computing resource (e.g., a CPU, memory, thread pool, buffer) is utilized.	▪ The (maximum, minimum, mean, median) time the response takes (latency) ▪ The number or percentage of satisfied requests over some time interval (throughput) or set of events received ▪ The number or percentage of requests that go unsatisfied ▪ The variation in response time (jitter) ▪ Usage level of a computing resource

Figure 9.1 gives an example concrete performance scenario: *Five hundred users initiate 2,000 requests in a 30-second interval, under normal operations. The system processes all of the requests with an average latency of two seconds.*

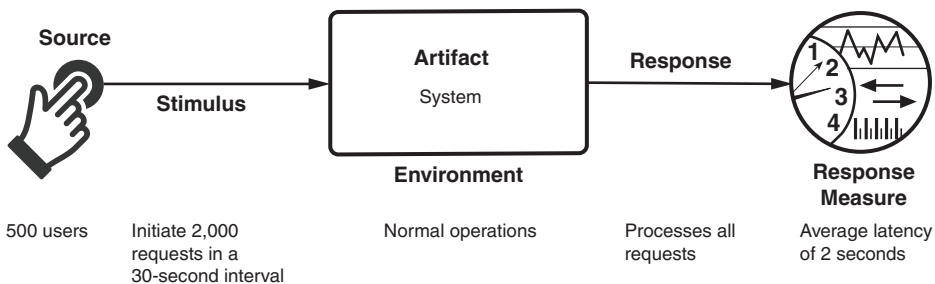
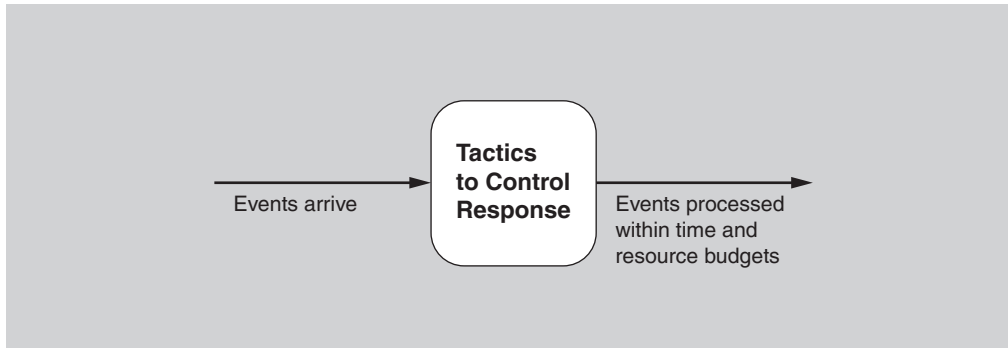


FIGURE 9.1 Sample performance scenario

9.2 Tactics for Performance

The goal of performance tactics is to generate a response to events arriving at the system under some time-based or resource-based constraint. The event can be a single event or a stream, and is the trigger to perform computation. Performance tactics control the time or resources used to generate a response, as illustrated in Figure 9.2.

**FIGURE 9.2** The goal of performance tactics

At any instant during the period after an event arrives but before the system's response to it is complete, either the system is working to respond to that event or the processing is blocked for some reason. This leads to the two basic contributors to the response time and resource usage: processing time (when the system is working to respond and actively consuming resources) and blocked time (when the system is unable to respond).

- *Processing time and resource usage.* Processing consumes resources, which takes time. Events are handled by the execution of one or more components, whose time expended is a resource. Hardware resources include CPU, data stores, network communication bandwidth, and memory. Software resources include entities defined by the system under design. For example, thread pools and buffers must be managed and access to critical sections must be made sequential.

For example, suppose a message is generated by one component. It might be placed on the network, after which it arrives at another component. It is then placed in a buffer; transformed in some fashion; processed according to some algorithm; transformed for output; placed in an output buffer; and sent onward to some component, another system, or some actor. Each of these steps contributes to the overall latency and resource consumption of the processing of that event.

Different resources behave differently as their utilization approaches their capacity—that is, as they become saturated. For example, as a CPU becomes more heavily loaded, performance usually degrades fairly steadily. In contrast, when you start to run out of memory, at some point the page swapping becomes overwhelming and performance crashes suddenly.

- *Blocked time and resource contention.* A computation can be blocked because of contention for some needed resource, because the resource is unavailable, or because the computation depends on the result of other computations that are not yet available:
- *Contention for resources.* Many resources can be used by only a single client at a time. As a consequence, other clients must wait for access to those resources. Figure 9.2

shows events arriving at the system. These events may be in a single stream or in multiple streams. Multiple streams vying for the same resource or different events in the same stream vying for the same resource contribute to latency. The more contention for a resource that occurs, the more latency grows.

- *Availability of resources.* Even in the absence of contention, computation cannot proceed if a resource is unavailable. Unavailability may be caused by the resource being offline or by failure of the component for any reason.
- *Dependency on other computation.* A computation may have to wait because it must synchronize with the results of another computation or because it is waiting for the results of a computation that it initiated. If a component calls another component and must wait for that component to respond, the time can be significant when the called component is at the other end of a network (as opposed to co-located on the same processor), or when the called component is heavily loaded.

Whatever the cause, you must identify places in the architecture where resource limitations might cause a significant contribution to overall latency.

With this background, we turn to our tactic categories. We can either reduce demand for resources (control resource demand) or make the resources we have available handle the demand more effectively (manage resources).

Control Resource Demand

One way to increase performance is to carefully manage the demand for resources. This can be done by reducing the number of events processed or by limiting the rate at which the system responds to events. In addition, a number of techniques can be applied to ensure that the resources that you do have are applied judiciously:

- *Manage work requests.* One way to reduce work is to reduce the number of requests coming into the system to do work. Ways to do that include the following:
 - *Manage event arrival.* A common way to manage event arrivals from an external system is to put in place a service level agreement (SLA) that specifies the maximum event arrival rate that you are willing to support. An SLA is an agreement of the form “The system or component will process X events arriving per unit time with a response time of Y.” This agreement constrains both the system—it must provide that response—and the client—if it makes more than X requests per unit time, the response is not guaranteed. Thus, from the client’s perspective, if it needs more than X requests per unit time to be serviced, it must utilize multiple instances of the element processing the requests. SLAs are one method for managing scalability for Internet-based systems.
 - *Manage sampling rate.* In cases where the system cannot maintain adequate response levels, you can reduce the sampling frequency of the stimuli—for example, the rate at which data is received from a sensor or the number of video frames per second that you process. Of course, the price paid here is the fidelity of the video stream or the information you gather from the sensor data. Nevertheless, this is a viable strategy if the result is “good enough.” Such an approach is commonly used in signal processing

systems where, for example, different codices can be chosen with different sampling rates and data formats. This design choice seeks to maintain predictable levels of latency; you must decide whether having a lower fidelity but consistent stream of data is preferable to having erratic latency. Some systems manage the sampling rate dynamically in response to latency measures or accuracy needs.

- *Limit event response.* When discrete events arrive at the system (or component) too rapidly to be processed, then the events must be queued until they can be processed, or they are simply discarded. You may choose to process events only up to a set maximum rate, thereby ensuring predictable processing for the events that are actually processed. This tactic could be triggered by a queue size or processor utilization exceeding some warning level. Alternatively, it could be triggered by an event rate that violates an SLA. If you adopt this tactic and it is unacceptable to lose any events, then you must ensure that your queues are large enough to handle the worst case. Conversely, if you choose to drop events, then you need to choose a policy: Do you log the dropped events or simply ignore them? Do you notify other systems, users, or administrators?
- *Prioritize events.* If not all events are equally important, you can impose a priority scheme that ranks events according to how important it is to service them. If insufficient resources are available to service them when they arise, low-priority events might be ignored. Ignoring events consumes minimal resources (including time), thereby increasing performance compared to a system that services all events all the time. For example, a building management system may raise a variety of alarms. Life-threatening alarms such as a fire alarm should be given higher priority than informational alarms such as a room being too cold.
- *Reduce computational overhead.* For events that do make it into the system, the following approaches can be implemented to reduce the amount of work involved in handling each event:
 - *Reduce indirection.* The use of intermediaries (so important for modifiability, as we saw in Chapter 8) increases the computational overhead in processing an event stream, so removing them improves latency. This is a classic modifiability/performance tradeoff. Separation of concerns—another linchpin of modifiability—can also increase the processing overhead necessary to service an event if it leads to an event being serviced by a chain of components rather than a single component. You may be able to realize the best of both worlds, however: Clever code optimization can let you *program* using the intermediaries and interfaces that support encapsulation (and thus keep the modifiability) but reduce, or in some cases eliminate, the costly indirection at runtime. Similarly, some brokers allow for direct communication between a client and a server (after initially establishing the relationship via the broker), thereby eliminating the indirection step for all subsequent requests.
 - *Co-locate communicating resources.* Context switching and intercomponent communication costs add up, especially when the components are on different nodes on a network. One strategy for reducing computational overhead is to co-locate resources. Co-location may mean hosting cooperating components on the same processor to avoid

the time delay of network communication; it may mean putting the resources in the same runtime software component to avoid even the expense of a subroutine call; or it may mean placing tiers of a multi-tier architecture on the same rack in the data center.

- *Periodic cleaning.* A special case when reducing computational overhead is to perform a periodic cleanup of resources that have become inefficient. For example, hash tables and virtual memory maps may require recalculation and reinitialization. Many system administrators and even regular computer users do a periodic reboot of their systems for exactly this reason.
- *Bound execution times.* You can place a limit on how much execution time is used to respond to an event. For iterative, data-dependent algorithms, limiting the number of iterations is a method for bounding execution times. The cost, however, is usually a less accurate computation. If you adopt this tactic, you will need to assess its effect on accuracy and see if the result is “good enough.” This resource management tactic is frequently paired with the manage sampling rate tactic.
- *Increase efficiency of resource usage.* Improving the efficiency of algorithms used in critical areas can decrease latency and improve throughput and resource consumption. This is, for some programmers, their *primary* performance tactic. If the system does not perform adequately, they try to “tune up” their processing logic. As you can see, this approach is actually just one of many tactics available.

Manage Resources

Even if the demand for resources is not controllable, the management of these resources can be. Sometimes one resource can be traded for another. For example, intermediate data may be kept in a cache or it may be regenerated depending on which resources are more critical: time, space, or network bandwidth. Here are some resource management tactics:

- *Increase resources.* Faster processors, additional processors, additional memory, and faster networks all have the potential to improve performance. Cost is usually a consideration in the choice of resources, but increasing the resources is, in many cases, the cheapest way to get immediate improvement.
- *Introduce concurrency.* If requests can be processed in parallel, the blocked time can be reduced. Concurrency can be introduced by processing different streams of events on different threads or by creating additional threads to process different sets of activities. (Once concurrency has been introduced, you can choose scheduling policies to achieve the goals you find desirable using the schedule resources tactic.)
- *Maintain multiple copies of computations.* This tactic reduces the contention that would occur if all requests for service were allocated to a single instance. Replicated services in a microservice architecture or replicated web servers in a server pool are examples of replicas of computation. A *load balancer* is a piece of software that assigns new work to one of the available duplicate servers; criteria for assignment vary but can be as simple as a round-robin scheme or assigning the next request to the least busy server. The load balancer pattern is discussed in detail in Section 9.4.

- *Maintain multiple copies of data.* Two common examples of maintaining multiple copies of data are data replication and caching. *Data replication* involves keeping separate copies of the data to reduce the contention from multiple simultaneous accesses. Because the data being replicated is usually a copy of existing data, keeping the copies consistent and synchronized becomes a responsibility that the system must assume. *Caching* also involves keeping copies of data (with one set of data possibly being a subset of the other), but on storage with different access speeds. The different access speeds may be due to memory speed versus secondary storage speed, or the speed of local versus remote communication. Another responsibility with caching is choosing the data to be cached. Some caches operate by merely keeping copies of whatever was recently requested, but it is also possible to predict users' future requests based on patterns of behavior, and to begin the calculations or prefetches necessary to comply with those requests before the user has made them.
- *Bound queue sizes.* This tactic controls the maximum number of queued arrivals and consequently the resources used to process the arrivals. If you adopt this tactic, you need to establish a policy for what happens when the queues overflow and decide if not responding to lost events is acceptable. This tactic is frequently paired with the limit event response tactic.
- *Schedule resources.* Whenever contention for a resource occurs, the resource must be scheduled. Processors are scheduled, buffers are scheduled, and networks are scheduled. Your concern as an architect is to understand the characteristics of each resource's use and choose the scheduling strategy that is compatible with it. (See the "Scheduling Policies" sidebar.)

Figure 9.3 summarizes the tactics for performance.

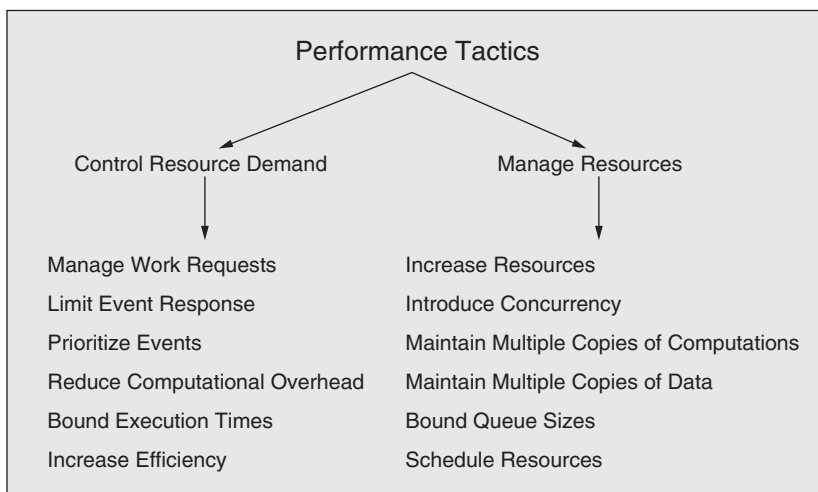


FIGURE 9.3 Performance tactics

Scheduling Policies

A *scheduling policy* conceptually has two parts: a priority assignment and dispatching. All scheduling policies assign priorities. In some cases, the assignment is as simple as first-in/first-out (or FIFO). In other cases, it can be tied to the deadline of the request or its semantic importance. Competing criteria for scheduling include optimal resource usage, request importance, minimizing the number of resources used, minimizing latency, maximizing throughput, preventing starvation to ensure fairness, and so forth. You need to be aware of these possibly conflicting criteria and the effect that the chosen scheduling policy has on the system's ability to meet them.

A high-priority event stream can be dispatched—assigned to a resource—only if that resource is available. Sometimes this depends on preempting the current user of the resource. Possible preemption options are as follows: can occur anytime, can occur only at specific preemption points, or executing processes cannot be preempted. Some common scheduling policies are these:

- *First-in/first-out.* FIFO queues treat all requests for resources as equals and satisfy them in turn. One possibility with a FIFO queue is that one request will be stuck behind another one that takes a long time to generate a response. As long as all of the requests are truly equal, this is not a problem—but if some requests are of higher priority than others, it creates a challenge.
- *Fixed-priority scheduling.* Fixed-priority scheduling assigns each source of resource requests a particular priority and assigns the resources in that priority order. This strategy ensures better service for higher-priority requests. However, it also admits the possibility that a lower-priority, but still important request might take an arbitrarily long time to be serviced, because it is stuck behind a series of higher-priority requests. Three common prioritization strategies are these:
 - *Semantic importance.* Semantic importance assigns a priority statically according to some domain characteristic of the task that generates it.
 - *Deadline monotonic.* Deadline monotonic is a static priority assignment that assigns a higher priority to streams with shorter deadlines. This scheduling policy is used when scheduling streams of different priorities with real-time deadlines.
 - *Rate monotonic.* Rate monotonic is a static priority assignment for periodic streams that assigns a higher priority to streams with shorter periods. This scheduling policy is a special case of deadline monotonic, but is better known and more likely to be supported by the operating system.
- *Dynamic priority scheduling.* Strategies include these:
 - *Round-robin.* The round-robin scheduling strategy orders the requests and then, at every assignment possibility, assigns the resource to the next request in that order. A special form of round-robin is a cyclic executive, where possible assignment times are designated at fixed time intervals.
 - *Earliest-deadline-first.* Earliest-deadline-first assigns priorities based on the pending requests with the earliest deadline.
 - *Least-slack-first.* This strategy assigns the highest priority to the job having the least “slack time,” which is the difference between the execution time remaining and the time to the job's deadline.

For a single processor and processes that are preemptible, both the earliest-deadline-first and least-slack-first scheduling strategies are optimal choices. That is, if the set of processes can be scheduled so that all deadlines are met, then these strategies will be able to schedule that set successfully.

- *Static scheduling.* A cyclic executive schedule is a scheduling strategy in which the preemption points and the sequence of assignment to the resource are determined offline. The runtime overhead of a scheduler is thereby obviated.

Performance Tactics on the Road

Tactics are generic design principles. To exercise this point, think about the design of the systems of roads and highways where you live. Traffic engineers employ a bunch of design “tricks” to optimize the performance of these complex systems, where performance has a number of measures, such as throughput (how many cars per hour get from the suburbs to the football stadium), average-case latency (how long it takes, on average, to get from your house to downtown), and worst-case latency (how long does it take an emergency vehicle to get you to the hospital). What are these tricks? None other than our good old buddies, tactics.

Let’s consider some examples:

- *Manage event rate.* Lights on highway entrance ramps let cars onto the highway only at set intervals, and cars must wait (queue) on the ramp for their turn.
- *Prioritize events.* Ambulances and police, with their lights and sirens going, have higher priority than ordinary citizens; some highways have high-occupancy vehicle (HOV) lanes, giving priority to vehicles with two or more occupants.
- *Maintain multiple copies.* Add traffic lanes to existing roads or build parallel routes.

In addition, users of the system can employ their own tricks:

- *Increase resources.* Buy a Ferrari, for example. All other things being equal, being the fastest car with a competent driver on an open road will get you to your destination more quickly.
- *Increase efficiency.* Find a new route that is quicker and/or shorter than your current route.
- *Reduce computational overhead.* Drive closer to the car in front of you, or load more people into the same vehicle (i.e., carpooling).

What is the point of this discussion? To paraphrase Gertrude Stein: Performance is performance is performance. Engineers have been analyzing and optimizing complex systems for centuries, trying to improve their performance, and they have been employing the same design strategies to do so. So you should feel some comfort in knowing that when you try to improve the performance of your computer-based system, you are applying tactics that have been thoroughly “road tested.”

—RK

9.3 Tactics-Based Questionnaire for Performance

Based on the tactics described in Section 9.2, we can create a set of tactics-inspired questions, as presented in Table 9.2. To gain an overview of the architectural choices made to support performance, the analyst asks each question and records the answers in the table. The answers to these questions can then be made the focus of further activities: investigation of documentation, analysis of code or other artifacts, reverse engineering of code, and so forth.

TABLE 9.2 Tactics-Based Questionnaire for Performance

Tactics Group	Tactics Question	Supported? (Y/N)	Risk	Design Decisions and Location	Rationale and Assumptions
Control Resource Demand	Do you have in place a service level agreement (SLA) that specifies the maximum event arrival rate that you are willing to support? Can you manage the rate at which you sample events arriving at the system? How will the system limit the response (amount of processing) for an event? Have you defined different categories of requests and defined priorities for each category? Can you reduce computational overhead by, for example, co-location, cleaning up resources, or reducing indirection? Can you bound the execution time of your algorithms? Can you increase computational efficiency through your choice of algorithms?				
Manage Resources	Can you allocate more resources to the system or its components? Are you employing concurrency? If requests can be processed in parallel, the blocked time can be reduced. Can computations be replicated on different processors?				

continues

TABLE 9.2 Tactics-Based Questionnaire for Performance *continued*

Tactics Group	Tactics Question	Supported? (Y/N)	Risk	Design Decisions and Location	Rationale and Assumptions
Manage Resources	Can data be cached (to maintain a local copy that can be quickly accessed) or replicated (to reduce contention)? Can queue sizes be bounded to place an upper bound on the resources needed to process stimuli? Have you ensured that the scheduling strategies you are using are appropriate for your performance concerns?				

9.4 Patterns for Performance

Performance concerns have plagued software engineers for decades, so it comes as no surprise that a rich set of patterns have been developed for managing various aspects of performance. In this section, we sample just a few of them. Note that some patterns serve multiple purposes. For example, we saw the circuit breaker pattern in Chapter 4, where it was identified as an availability pattern, but it also has a benefit for performance—since it reduces the time that you wait around for nonresponsive services.

The patterns we will introduce here are service mesh, load balancer, throttling, and map-reduce.

Service Mesh

The service mesh pattern is used in microservice architectures. The main feature of the mesh is a sidecar—a kind of proxy that accompanies each microservice, and which provides broadly useful capabilities to address application-independent concerns such as interservice communications, monitoring, and security. A sidecar executes alongside each microservice and handles all interservice communication and coordination. (As we will describe in Chapter 16, these elements are often packaged into *pods*.) They are deployed together, which cuts down on the latency due to networking, thereby boosting performance.

This approach allows developers to separate the functionality—the core business logic—of the microservice from the implementation, management, and maintenance of cross-cutting concerns, such as authentication and authorization, service discovery, load balancing, encryption, and observability.

Benefits:

- Software to manage cross-cutting concerns can be purchased off the shelf or implemented and maintained by a specialist team that does nothing else, allowing developers of the business logic to focus on only that concern.
- A service mesh enforces the deployment of utility functions onto the same processor as the services that use those utility functions. This cuts down on communication time between the service and its utilities since the communication does not need to use network messages.
- The service mesh can be configured to make communication dependent on context, thus simplifying functions such as the canary and A/B testing described in Chapter 3.

Tradeoffs:

- The sidecars introduce more executing processes, and each of these will consume some processing power, adding to the system's overhead.
- A sidecar typically includes multiple functions, and not all of these will be needed in every service or every invocation of a service.

Load Balancer

A load balancer is a kind of intermediary that handles messages originating from some set of clients and determines which instance of a service should respond to those messages. The key to this pattern is that the load balancer serves as a single point of contact for incoming messages—for example, a single IP address—but it then farms out requests to a pool of providers (servers or services) that can respond to the request. In this way, the load can be balanced across the pool of providers. The load balancer implements some form of the schedule resources tactic. The scheduling algorithm may be very simple, such as round-robin, or it may take into account the load on each provider, or the number of requests awaiting service at each provider.

Benefits:

- Any failure of a server is invisible to clients (assuming there are still some remaining processing resources).
- By sharing the load among several providers, latency can be kept lower and more predictable for clients.
- It is relatively simple to add more resources (more servers, faster servers) to the pool available to the load balancer, and no client needs to be aware of this.

Tradeoffs:

- The load balancing algorithm must be very fast; otherwise, it may itself contribute to performance problems.
- The load balancer is a potential bottleneck or single point of failure, so it is itself often replicated (and even load balanced).

Load balancers are discussed in much more detail in Chapter 17.

Throttling

The throttling pattern is a packaging of the manage work requests tactic. It is used to limit access to some important resource or service. In this pattern, there is typically an intermediary—a throttler—that monitors (requests to) the service and determines whether an incoming request can be serviced.

Benefits:

- By throttling incoming requests, you can gracefully handle variations in demand. In doing so, services never become overloaded; they can be kept in a performance “sweet spot” where they handle requests efficiently.

Tradeoffs:

- The throttling logic must be very fast; otherwise, it may itself contribute to performance problems.
- If client demand regularly exceeds capacity, buffers will need to be very large, or there is a risk of losing requests.
- This pattern can be difficult to add to an existing system where clients and servers are tightly coupled.

Map-Reduce

The map-reduce pattern efficiently performs a distributed and parallel sort of a large data set and provides a simple means for the programmer to specify the analysis to be done. Unlike our other patterns for performance, which are independent of any application, the map-reduce pattern is specifically designed to bring high performance to a specific kind of recurring problem: sort and analyze a large data set. This problem is experienced by any organization dealing with massive data—think Google, Facebook, Yahoo, and Netflix—and all of these organizations do in fact use map-reduce.

The map-reduce pattern has three parts:

- First is a specialized infrastructure that takes care of allocating software to the hardware nodes in a massively parallel computing environment and handles sorting the data as needed. A node may be a virtual machine, a standalone processor, or a core in a multi-core chip.
- Second and third are two programmer-coded functions called, predictably enough, *map* and *reduce*.
 - The *map* function takes as input a key and a data set. It uses the key to hash the data into a set of buckets. For example, if our data set consisted of playing cards, the key could be the suit. The map function is also used to filter the data—that is, determine whether a data record is to be involved in further processing or discarded. Continuing our card example, we might choose to discard jokers or letter cards (A, K, Q, J), keeping only numeric cards, and we could then map each card into a bucket, based on its suit. The performance of the map phase of the map-reduce pattern is enhanced by having multiple map instances, each of which processes a different portion of the data

set. An input file is divided into portions, and a number of map instances are created to process each portion. Continuing our example, let's consider that we have 1 billion playing cards, not just a single deck. Since each card can be examined in isolation, the map process can be carried out by tens or hundreds of thousands of instances in parallel, with no need for communication among them. Once all of the input data has been mapped, these buckets are shuffled by the map-reduce infrastructure, and then assigned to new processing nodes (possibly reusing the nodes used in the map phase) for the reduce phase. For example, all of the clubs could be assigned to one cluster of instances, all of the diamonds to another cluster, and so forth.

- All of the heavy analysis takes place in the *reduce* function. The number of reduce instances corresponds to the number of buckets output by the map function. The reduce phase does some programmer-specified analysis and then emits the results of that analysis. For example, we could count the number of clubs, diamonds, hearts, and spades, or we could sum the numeric values of all of the cards in each bucket. The output set is almost always much smaller than the input sets—hence the name “reduce.”

The map instances are stateless and do not communicate with each other. The only communication between the map instances and the reduce instances is the data emitted from the map instances as <key, value> pairs.

Benefits:

- Extremely large, unsorted data sets can be efficiently analyzed through the exploitation of parallelism.
- A failure of any instance has only a small impact on the processing, since map-reduce typically breaks large input datasets into many smaller ones for processing, allocating each to its own instance.

Tradeoffs:

- If you do not have large data sets, the overhead incurred by the map-reduce pattern is not justified.
- If you cannot divide your data set into similarly sized subsets, the advantages of parallelism are lost.
- Operations that require multiple reduces are complex to orchestrate.

9.5 For Further Reading

Performance is the subject of a rich body of literature. Here are some books we recommend as general overviews of performance:

- *Foundations of Software and System Performance Engineering: Process, Performance Modeling, Requirements, Testing, Scalability, and Practice* [Bondi 14]. This book provides a comprehensive overview of performance engineering, ranging from technical practices to organizational ones.

- *Software Performance and Scalability: A Quantitative Approach* [Liu 09]. This book covers performance geared toward enterprise applications, with an emphasis on queueing theory and measurement.
- *Performance Solutions: A Practical Guide to Creating Responsive, Scalable Software* [Smith 01]. This book covers designing with performance in mind, with emphasis on building (and populating with real data) practical predictive performance models.

To get an overview of some of the many patterns for performance, see *Real-Time Design Patterns: Robust Scalable Architecture for Real-Time Systems* [Douglass 99] and *Pattern-Oriented Software Architecture Volume 3: Patterns for Resource Management* [Kircher 03]. In addition, Microsoft has published a catalog of performance and scalability patterns for cloud-based applications: <https://docs.microsoft.com/en-us/azure/architecture/patterns/category/performance-scalability>.

9.6 Discussion Questions

1. “Every system has real-time performance constraints.” Discuss. Can you provide a counterexample?
2. Write a concrete performance scenario that describes the average on-time flight arrival performance for an airline.
3. Write several performance scenarios for an online auction site. Think about whether your major concern is worst-case latency, average-case latency, throughput, or some other response measure. Which tactics would you use to satisfy your scenarios?
4. Web-based systems often use *proxy servers*, which are the first element of the system to receive a request from a client (such as your browser). Proxy servers are able to serve up often-requested web pages, such as a company’s home page, without bothering the real application servers that carry out transactions. A system may include many proxy servers, and they are often located geographically close to large user communities, to decrease response time for routine requests. What performance tactics do you see at work here?
5. A fundamental difference between interaction mechanisms is whether interaction is synchronous or asynchronous. Discuss the advantages and disadvantages of each with respect to each of these performance responses: latency, deadline, throughput, jitter, miss rate, data loss, or any other required performance-related response you may be used to.
6. Find physical-world (that is, non-software) examples of applying each of the manage resources tactics. For example, suppose you were managing a brick-and-mortar big-box retail store. How would you get people through the checkout lines faster using these tactics?
7. User interface frameworks typically are single-threaded. Why is this? What are the performance implications? (Hint: Think about race conditions.)

10



Safety

Giles: Well, for god's sake, be careful. . . . If you should be hurt or killed, shall take it amiss.

Willow: Well, we try not to get killed. That's part of our whole mission statement: Don't get killed.

Giles: Good.

—*Buffy the Vampire Slayer*, Season 3, episode “Anne”

“Don’t kill anyone” should be a part of every software architect’s mission statement.

The thought that software could kill people or cause injury or damage used to belong solidly in the realm of computers-run-amok science fiction; think of HAL politely declining to open the pod bay doors in the now-aged but still-classic movie *2001: A Space Odyssey*, leaving Dave stranded in space.

Sadly, it didn’t stay there. As software has come to control more and more of the devices in our lives, software safety has become a critical concern.

The thought that software (strings of 0s and 1s) can kill or maim or destroy is still an unnatural notion. To be fair, it’s not the 0s and 1s that wreak havoc—at least, not directly. It’s what they’re connected to. Software, and the computer in which it runs, has to be connected to the outside world in some way before it can do damage. That’s the good news. The bad news is that the good news isn’t all that good. Software *is* connected to the outside world, always. If your program has no effect whatsoever that is observable outside of itself, it probably serves no purpose.

In 2009, an employee of the Shushenskaya hydroelectric power station used a cybernetwork to remotely—and accidentally—activate an unused turbine with a few errant keystrokes. The offline turbine created a “water hammer” that flooded and then destroyed the plant and killed dozens of workers.

There are many other equally notorious examples. The Therac 25 fatal radiation overdose, the Ariane 5 explosion, and a hundred lesser-known accidents all caused harm because the computer was connected to the environment: a turbine, an X-ray emitter, and a rocket’s steering controls, in the examples just cited. The infamous Stuxnet virus was created to intentionally cause damage and destruction. In these cases, software commanded some hardware in its environment to take a disastrous action, and the hardware obeyed. Actuators are devices

that connect hardware to software; they are the bridge between the world of 0s and 1s and the world of motion and control. Send a digital value to an actuator (or write a bit string in the hardware register corresponding to the actuator) and that value is translated to some mechanical action, for better or worse.

But connecting to the outside world doesn't have to mean robot arms or uranium centrifuges or missile launchers: Connecting to a simple display screen is enough. Sometimes all the computer has to do is send erroneous information to its human operators. In September 1983, a Soviet satellite sent data to its ground system computer, which interpreted that data as a missile launched from the United States aimed at Moscow. Seconds later, the computer reported a second missile in flight. Soon, a third, then a fourth, and then a fifth appeared. Soviet Strategic Rocket Forces Lieutenant Colonel Stanislav Yevgrafovich Petrov made the astonishing decision to ignore the computers, believing them to be in error. He thought it extremely unlikely that the United States would have fired just a few missiles, thereby inviting mass retaliatory destruction. He decided to wait it out, to see if the missiles were real—that is, to see if his country's capital city was going to be incinerated. As we know, it wasn't. The Soviet system had mistaken a rare sunlight condition for missiles in flight. You and/or your parents may well owe your life to Lieutenant Colonel Petrov.

Of course, the humans don't always get it right when the computers get it wrong. On the stormy night of June 1, 2009, Air France flight 447 from Rio de Janeiro to Paris plummeted into the Atlantic Ocean, killing all 228 people on board, despite the aircraft's engines and flight controls working perfectly. The Airbus A-330's flight recorders, which were not recovered until May 2011, showed that the pilots never knew that the aircraft had entered a high-altitude stall. The sensors that measure airspeed had become clogged with ice and therefore unreliable; the autopilot disengaged as a result. The human pilots thought the aircraft was going too fast (and in danger of structural failure) when in fact it was going too slow (and falling). During the entire 3-minute-plus plunge from 35,000 feet, the pilots kept trying to pull the nose up and throttle back to lower the speed, when all they needed to do was lower the nose to increase the speed and resume normal flying. Very probably adding to the confusion was the way the A-330's stall warning system worked. When the system detects a stall, it emits a loud audible alarm. The software deactivates the stall warning when it "thinks" that the angle of attack measurements are invalid. This can occur when the airspeed readings are very low. That is what happened with AF447: Its forward speed dropped below 60 knots, and the angle of attack was extremely high. As a consequence of this flight control software rule, the stall warning stopped and started several times. Worse, it came on whenever the pilot pushed forward on the stick (increasing the airspeed and taking the readings into the "valid" range, but still in stall) and then stopped when he pulled back. That is, doing the right thing resulted in exactly the wrong feedback, and vice versa. Was this an unsafe system, or a safe system operated unsafely? Ultimately questions like this are decided in the courts.

As this edition was going to publication, Boeing was still reeling from the grounding of its 737 MAX aircraft after two crashes that appear to have been caused at least partly by a piece of software called MCAS, which pushed the aircraft's nose down at the wrong time. Faulty sensors seem to be involved here, too, as well as a baffling design decision that caused the software to rely on only one sensor to determine its behavior, instead of the two available on the aircraft. It also appears that Boeing never tested the software in question under the

conditions of a sensor failure. The company did provide a way to disable the system in flight, although remembering how to do that when your airplane is doing its best to kill you may be asking a lot of a flight crew—especially when they were never made aware of the existence of the MCAS in the first place. In total, 346 people died in the two crashes of the 737 MAX.

Okay, enough scary stories. Let's talk about the principles behind them as they affect software and architectures.

Safety is concerned with a system's ability to avoid straying into states that cause or lead to damage, injury, or loss of life to actors in its environment. These unsafe states can be caused by a variety of factors:

- *Omissions* (the failure of an event to occur).
- *Commission* (the spurious occurrence of an undesirable event). The event could be acceptable in some system states but undesirable in others.
- *Timing*. Early (the occurrence of an event before the time required) or late (the occurrence of an event after the time required) timing can both be potentially problematic.
- *Problems with system values*. These come in two categories: Coarse incorrect values are incorrect but detectable, whereas subtle incorrect values are typically undetectable.
- *Sequence omission and commission*. In a sequence of events, either an event is missing (omission) or an unexpected event is inserted (commission).
- *Out of sequence*. A sequence of events arrive, but not in the prescribed order.

Safety is also concerned with detecting and recovering from these unsafe states to prevent or at least minimize resulting harm.

Any portion of the system can lead to an unsafe state: The software, the hardware portions, or the environment can behave in an unanticipated, unsafe fashion. Once an unsafe state is detected, the potential system responses are similar to those enumerated for availability (in Chapter 4). The unsafe state should be recognized and the system should be made through

- Continuing operations after recovering from the unsafe state or placing the system in a safe mode, or
- Shutting down (fail safe), or
- Transitioning to a state requiring manual operation (e.g., manual steering if the power steering in a car fails).

In addition, the unsafe state should be reported immediately and/or logged.

Architecting for safety begins by identifying the system's safety-critical functions—those functions that could cause harm as just outlined—using techniques such as failure mode and effects analysis (FMEA; also called hazard analysis) and fault tree analysis (FTA). FTA is a top-down deductive approach to identify failures that could result in moving the system into an unsafe state. Once the failures have been identified, the architect needs to design mechanisms to detect and mitigate the fault (and ultimately the hazard).

The techniques outlined in this chapter are intended to discover possible hazards that could result from the system's operation and help in creating strategies to cope with these hazards.

10.1 Safety General Scenario

With this background, we can construct the general scenario for safety, shown in Table 10.1.

TABLE 10.1 Safety General Scenario

Portion of Scenario	Description	Possible Values
Source	A data source (a sensor, a software component that calculates a value, a communication channel), a time source (clock), or a user action	Specific instances of a: ▪ Sensor ▪ Software component ▪ Communication channel ▪ Device (such as a clock)
Stimulus	An omission, commission, or occurrence of incorrect data or timing	A specific instance of an omission: ▪ A value never arrives. ▪ A function is never performed. A specific instance of a commission: ▪ A function is performed incorrectly. ▪ A device produces a spurious event. ▪ A device produces incorrect data. A specific instance of incorrect data: ▪ A sensor reports incorrect data. ▪ A software component produces incorrect results. A timing failure: ▪ Data arrives too late or too early. ▪ A generated event occurs too late or too early or at the wrong rate. ▪ Events occur in the wrong order.
Environment	System operating mode	▪ Normal operation ▪ Degraded operation ▪ Manual operation ▪ Recovery mode
Artifacts	The artifact is some part of the system.	Safety-critical portions of the system
Response	The system does not leave a safe state space, or the system returns to a safe state space, or the system continues to operate in a degraded mode to prevent (further) injury or damage or to minimize injury or damage. Users are advised of the unsafe state or the prevention of entry into the unsafe state. The event is logged.	Recognize the unsafe state and one or more of the following: ▪ Avoid the unsafe state ▪ Recover ▪ Continue in degraded or safe mode ▪ Shut down ▪ Switch to manual operation ▪ Switch to a backup system ▪ Notify appropriate entities (people or systems) ▪ Log the unsafe state (and the response to it)

Portion of Scenario	Description	Possible Values
Response measure	Time to return to safe state space; damage or injury caused	One or more of the following: ▪ Amount or percentage of entries into unsafe states that are avoided ▪ Amount or percentages of unsafe states from which the system can (automatically) recover ▪ Change in risk exposure: size(loss) * prob(loss) ▪ Percentage of time the system can recover ▪ Amount of time the system is in a degraded or safe mode ▪ Amount or percentage of time the system is shut down ▪ Elapsed time to enter and recover (from manual operation, from a safe or degraded mode)

A sample safety scenario is: *A sensor in the patient monitoring system fails to report a life-critical value after 100 ms. The failure is logged, a warning light is illuminated on the console, and a backup (lower-fidelity) sensor is engaged. The system monitors the patient using the backup sensor after no more than 300 ms.* Figure 10.1 illustrates this scenario.

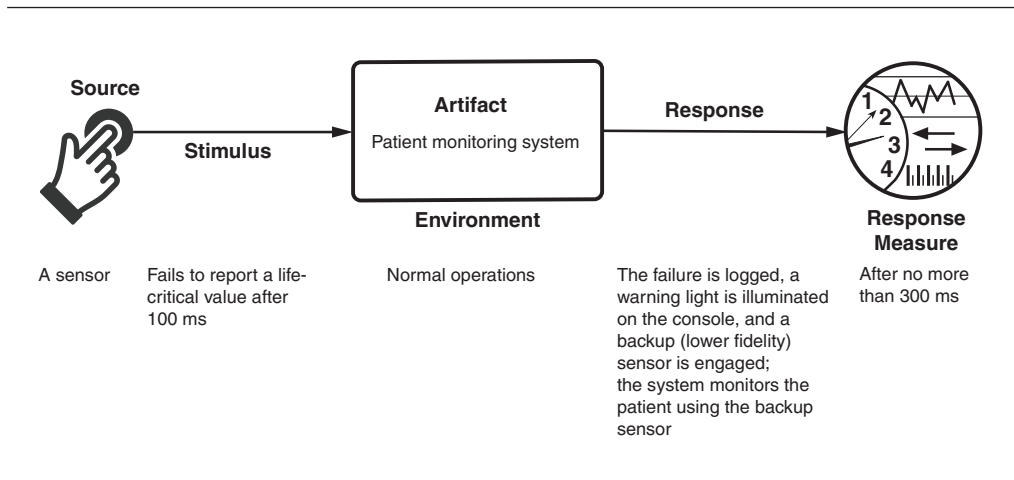


FIGURE 10.1 Sample concrete safety scenario

10.2 Tactics for Safety

Safety tactics may be broadly categorized as unsafe state avoidance, unsafe state detection, or unsafe state remediation. Figure 10.2 shows the goal of the set of safety tactics.

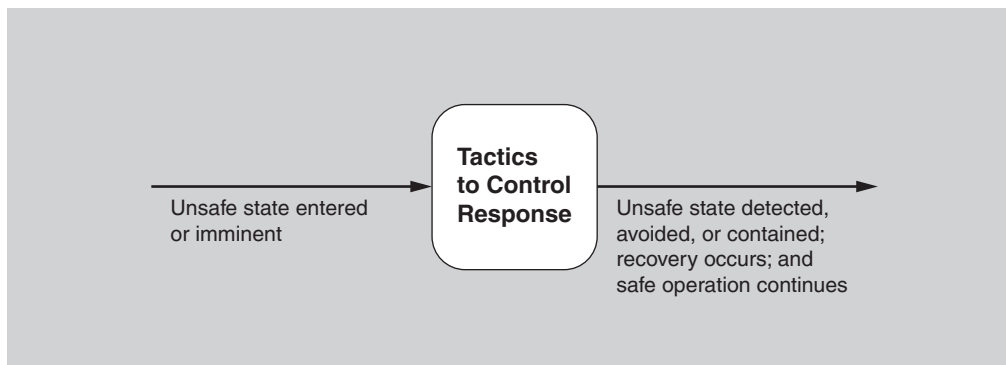


FIGURE 10.2 Goal of safety tactics

A logical precondition to avoid or detect entry into an unsafe state is the ability to recognize what constitutes an unsafe state. The following tactics assume that capability, which means that you should perform your own hazard analysis or FTA once you have your architecture in hand. Your design decisions may themselves have introduced new safety vulnerabilities not accounted for during requirements analysis.

You will note a substantial overlap between the tactics presented here and those presented in Chapter 4 on availability. This overlap occurs because availability problems may often lead to safety problems, and because many of the design solutions for repairing these problems are shared between the qualities.

Figure 10.3 summarizes the architectural tactics to achieve safety.

Unsafe State Avoidance

Substitution

This tactic employs protection mechanisms—often hardware-based—for potentially dangerous software design features. For example, hardware protection devices such as watchdogs, monitors, and interlocks can be used in lieu of software versions. Software versions of these mechanisms can be starved of resources, whereas a separate hardware device provides and controls its own resources. Substitution is typically beneficial only when the function being replaced is relatively simple.

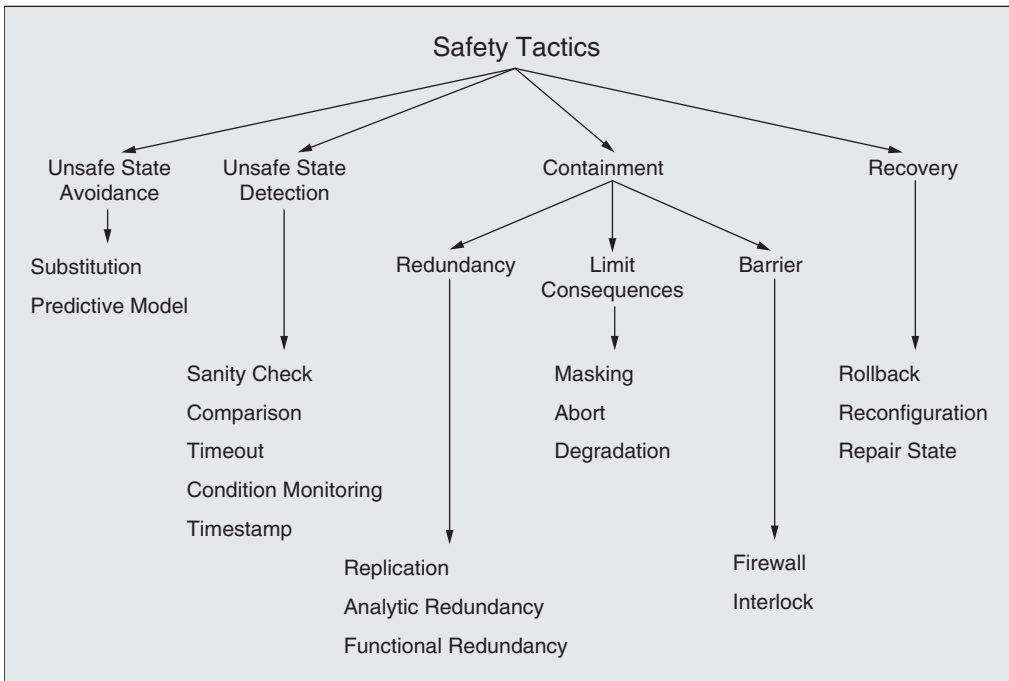


FIGURE 10.3 Safety tactics

Predictive Model

The predictive model tactic, as introduced in Chapter 4, predicts the state of health of system processes, resources, or other properties (based on monitoring the state), not only to ensure that the system is operating within its nominal operating parameters but also to provide early warning of a potential problem. For example, some automotive cruise control systems calculate the closing rate between the vehicle and an obstacle (or another vehicle) ahead and warn the driver before the distance and time become too small to avoid a collision. A predictive model is typically combined with condition monitoring, which we discuss later.

Unsafe State Detection

Timeout

The timeout tactic is used to determine whether the operation of a component is meeting its timing constraints. This might be realized in the form of an exception being raised, to indicate the failure of a component if its timing constraints are not met. Thus this tactic can detect late timing and omission failures. Timeout is a particularly common tactic in real-time or

embedded systems and distributed systems. It is related to the availability tactics of system monitor, heartbeat, and ping-echo.

Timestamp

As described in Chapter 4, the timestamp tactic is used to detect incorrect sequences of events, primarily in distributed message-passing systems. A timestamp of an event can be established by assigning the state of a local clock to the event immediately after the event occurs. Sequence numbers can also be used for this purpose, since timestamps in a distributed system may be inconsistent across different processors.

Condition Monitoring

This tactic involves checking conditions in a process or device, or validating assumptions made during the design, perhaps by using assertions. Condition monitoring identifies system states that may lead to hazardous behavior. However, the monitor should be simple (and, ideally, provable) to ensure that it does not introduce new software errors or contribute significantly to overall workload. Condition monitoring provides the input to a predictive model and to sanity checking.

Sanity Checking

The sanity checking tactic checks the validity or reasonableness of specific operation results, or inputs or outputs of a component. This tactic is typically based on a knowledge of the internal design, the state of the system, or the nature of the information under scrutiny. It is most often employed at interfaces, to examine a specific information flow.

Comparison

The comparison tactic allows the system to detect unsafe states by comparing the outputs produced by a number of synchronized or replicated elements. Thus the comparison tactic works together with a redundancy tactic, typically the active redundancy tactic presented in the discussion of availability. When the number of replicants is three or greater, the comparison tactic can not only detect an unsafe state but also indicate which component has led to it. Comparison is related to the voting tactic used in availability. However, a comparison may not always lead to a vote; another option is to simply shut down if outputs differ.

Containment

Containment tactics seek to limit the harm associated with an unsafe state that has been entered. This category includes three subcategories: redundancy, limit consequences, and barrier.

Redundancy

The redundancy tactics, at first glance, appear to be similar to the various sparing/redundancy tactics presented in the discussion of availability. Clearly, these tactics overlap, but since the goals of safety and availability are different, the use of backup components differs. In

the realm of safety, redundancy enables the system to continue operation in the case where a total shutdown or further degradation would be undesirable.

Replication is the simplest redundancy tactic, as it just involves having clones of a component. Having multiple copies of identical components can be effective in protecting against random failures of hardware, but it cannot protect against design or implementation errors in hardware or software since there is no form of diversity embedded in this tactic.

Functional redundancy, by contrast, is intended to address the issue of common-mode failures (where replicas exhibit the same fault at the same time because they share the same implementation) in hardware or software components, by implementing design diversity. This tactic attempts to deal with the systematic nature of design faults by adding diversity to redundancy. The outputs of functionally redundant components should be the same given the same input. The functional redundancy tactic is still vulnerable to specification errors, however, and of course, functional replicas will be more expensive to develop and verify.

Finally, the analytic redundancy tactic permits not only diversity of components, but also a higher-level diversity that is visible at the input and output level. As a consequence, it can tolerate specification errors by using separate requirement specifications. Analytic redundancy often involves partitioning the system into high assurance and high performance (low assurance) portions. The high assurance portion is designed to be simple and reliable, whereas the high performance portion is typically designed to be more complex and more accurate, but less stable: It changes more rapidly, and may not be as reliable as the high assurance portion. (Hence, here we do not mean high performance in the sense of latency or throughput; rather, this portion “performs” its task better than the high assurance portion.)

Limit Consequences

The second subcategory of containment tactics is called limit consequences. These tactics are all intended to limit the bad effects that may result from the system entering an unsafe state.

The abort tactic is conceptually the simplest. If an operation is determined to be unsafe, it is aborted before it can cause damage. This technique is widely employed to ensure that systems fail safely.

The degradation tactic maintains the most critical system functions in the presence of component failures, dropping or replacing functionality in a controlled way. This approach allows individual component failures to gracefully reduce system functionality in a planned, deliberate, and safe way, rather than causing a complete system failure. For example, a car navigation system may continue to operate using a (less accurate) dead reckoning algorithm in a long tunnel where it has lost its GPS satellite signal.

The masking tactic masks a fault by comparing the results of several redundant components and employing a voting procedure in case one or more of the components differ. For this tactic to work as intended, the voter must be simple and highly reliable.

Barrier

The barrier tactics contain problems by keeping them from propagating.

The firewall tactic is a specific realization of the limit access tactic, which is described in Chapter 11. A firewall limits access to specified resources, typically processors, memory, and network connections.

The interlock tactic protects against failures arising from incorrect sequencing of events. Realizations of this tactic provide elaborate protection schemes by controlling all access to protected components, including controlling the correct sequencing of events affecting those components.

Recovery

The final category of safety tactics is recovery, which acts to place the system in a safe state. It encompasses three tactics: rollback, repair state, and reconfiguration.

The rollback tactic permits the system to revert to a saved copy of a previous known good state—the rollback line—upon the detection of a failure. This tactic is often combined with checkpointing and transactions, to ensure that the rollback is complete and consistent. Once the good state is reached, then execution can continue, potentially employing other tactics such as retry or degradation to ensure that the failure does not reoccur.

The repair state tactic repairs an erroneous state—effectively increasing the set of states that a component can handle competently (i.e., without failure)—and then continues execution. For example, a vehicle’s lane keep assist feature will monitor whether a driver is staying within their lane and actively return the vehicle to a position between the lines—a safe state—if it drifts out. This tactic is inappropriate as a means of recovery from unanticipated faults.

Reconfiguration attempts to recover from component failures by remapping the logical architecture onto the (potentially limited) resources left functioning. Ideally, this remapping allows full functionality to be maintained. When this is not possible, the system may be able to maintain partial functionality in combination with the degradation tactic.

10.3 Tactics-Based Questionnaire for Safety

Based on the tactics described in Section 10.2, we can create a set of tactics-inspired questions, as presented in Table 10.2. To gain an overview of the architectural choices made to support safety, the analyst asks each question and records the answers in the table. The answers to these questions can then be made the focus of further activities: investigation of documentation, analysis of code or other artifacts, reverse engineering of code, and so forth.

Prior to beginning the tactics-based questionnaire for safety, you should assess whether the project under review has performed a hazard analysis or FTA to identify what constitutes an unsafe state (to be detected, avoided, contained, or recovered from) in your system. Without this analysis, designing for safety is likely to be less effective.

TABLE 10.2 Tactics-Based Questionnaire for Safety

Tactics Group	Tactics Question	Supported? (Y/N)	Risk	Design Decisions and Location	Rationale and Assumptions
Unsafe State Avoidance	Do you employ substitution —that is, safer, often hardware-based protection mechanisms for potentially dangerous software design features? Do you use a predictive model to predict the state of health of system processes, resources, or other properties—based on monitored information—not only to ensure that the system is operating within its nominal operating parameters, but also to provide early warning of a potential problem?				
Unsafe State Detection	Do you use timeouts to determine whether the operation of a component meets its timing constraints? Do you use timestamps to detect incorrect sequences of events? Do you employ condition monitoring to check conditions in a process or device, particularly to validate assumptions made during design? Is sanity checking employed to check the validity or reasonableness of specific operation results, or inputs or outputs of a component? Does the system employ comparison to detect unsafe states, by comparing the outputs produced based on the number of synchronized or replicated elements?				
Containment: Redundancy	Do you use replication —clones of a component—to protect against random failures of hardware? Do you use functional redundancy to address the common-mode failures by implementing diversely designed components?				

continues

TABLE 10.2 Tactics-Based Questionnaire for Safety *continued*

Tactics Group	Tactics Question	Supported? (Y/N)	Risk	Design Decisions and Location	Rationale and Assumptions
Containment: Redundancy	Do you use analytic redundancy —functional “replicas” that include high assurance/high performance and low assurance/low performance alternatives—to be able to tolerate specification errors?				
Containment: Limit Consequences	Can the system abort an operation that is determined to be unsafe before it can cause damage? Does the system provide controlled degradation , where the most critical system functions are maintained in the presence of component failures, while less critical functions are dropped or degraded? Does the system mask a fault by comparing the results of several redundant components and employ a voting procedure in case one or more of the components differ?				
Containment: Barrier	Does the system support limiting access to critical resources (e.g., processors, memory, and network connections) through a firewall ? Does the system control access to protected components and protect against failures arising from incorrect sequencing of events through interlocks ?				
Recovery	Is the system able to roll back —that is, to revert to a previous known good state—upon the detection of a failure? Can the system repair a state determined to be erroneous, without failure, and then continue execution? Can the system reconfigure resources, in the event of failures, by remapping the logical architecture onto the resources left functioning?				

10.4 Patterns for Safety

A system that unexpectedly stops operating, or starts operating incorrectly, or falls into a degraded mode of operation is likely to affect safety negatively, if not catastrophically. Hence, the first place to look for safety patterns is in patterns for availability, such as the ones described in Chapter 4. They all apply here.

- *Redundant sensors.* If the data produced by a sensor is important to determine whether a state is safe or unsafe, that sensor should be replicated. This protects against the failure of any single sensor. Also, independent software should monitor each sensor—in essence, the redundant spare tactic from Chapter 4 applied to safety-critical hardware.

Benefits:

- This form of redundancy, which is applied to sensors, guards against the failure of a single sensor.

Tradeoffs:

- Redundant sensors add cost to the system, and processing the inputs from multiple sensors is more complicated than processing the input from a single sensor.
- *Monitor-actuator.* This pattern focuses on two software elements—a monitor and an actuator controller—that are employed before sending a command to a physical actuator. The actuator controller performs the calculations necessary to determine the values to send to the physical actuator. The monitor checks these values for reasonableness before sending them. This separates the computation of the value from the testing of the value.

Benefits:

- In this form of redundancy applied to actuator control, the monitor acts as a redundant check on the actuator controller computations.

Tradeoffs:

- The development and maintenance of the monitor take time and resources.
- Because of the separation this pattern achieves between actuator control and monitoring, this particular tradeoff is easy to manipulate by making the monitor as simple (easy to produce but may miss errors) or as sophisticated (more complex but catches more errors) as required.
- *Separated safety.* Safety-critical systems must frequently be certified as safe by some authority. Certifying a large system is expensive, but dividing a system into safety-critical portions and non-safety-critical portions can reduce those costs. The safety-critical portion must still be certified. Likewise, the division into safety-critical and non-critical portions must be certified to ensure that there is no influence on the safety-critical portion from the non-safety-critical portion.

Benefits:

- The cost of certifying the system is reduced because you need to certify only a (usually small) portion of the total system.
- Cost and safety benefits accrue because the effort focuses on just those portions of the system that are germane to safety.

Tradeoffs:

- The work involved in performing the separation can be expensive, such as installing two different networks in a system to partition safety-critical and non-safety-critical messages. However, this approach limits the risk and consequences of bugs in the non-safety-critical portion from affecting the safety-critical portion.
- Separating the system and convincing the certification agency that the separation was performed correctly and that there are no influences from the non-safety-critical portion on the safety-critical portion is difficult, but is far easier than the alternative: having the agency certify everything to the same rigid level.

Design Assurance Levels

The separated safety pattern emphasizes dividing the software system into safety-critical portions and non-safety-critical portions. In avionics, the distinction is finer-grained. DO-178C, “Software Considerations in Airborne Systems and Equipment Certification,” is the primary document by which certification authorities such as Federal Aviation Administration (FAA), European Union Aviation Safety Agency (EASA), and Transport Canada approve all commercial software-based aerospace systems. It defines a ranking called Design Assurance Level (DAL) for each software function. The DAL is determined from the safety assessment process and hazard analysis by examining the effects of a failure condition in the system. The failure conditions are categorized by their effects on the aircraft, crew, and passengers:

- *A: Catastrophic.* Failure may cause deaths, usually with loss of the airplane.
- *B: Hazardous.* Failure has a large negative impact on safety or performance, or reduces the crew’s ability to operate the aircraft due to physical distress or a higher workload, or causes serious or fatal injuries among the passengers.
- *C: Major.* Failure significantly reduces the safety margin or significantly increases crew workload, and may result in passenger discomfort (or even minor injuries).
- *D: Minor.* Failure slightly reduces the safety margin or slightly increases crew workload. Examples might include causing passenger inconvenience or a routine flight plan change.
- *E: No effect.* Failure has no impact on safety, aircraft operation, or crew workload.

Software validation and testing is a terrifically expensive task, undertaken with very finite budgets. DALs help you decide where to put your limited testing resources. The next time you’re on a commercial airline flight, if you see a glitch in the entertainment system or your reading light keeps blinking off, take comfort by thinking of all the validation money spent on making sure the flight control system works just fine.

—PC

10.5 For Further Reading

To gain an appreciation for the importance of software safety, we suggest reading some of the disaster stories that arise when software fails. A venerable source is the ACM Risks Forum, available at risks.org. This has been moderated by Peter Neumann since 1985 and is still going strong.

Two prominent standard safety *processes* are described in ARP-4761, “Guidelines and Methods for Conducting the Safety Assessment Process on Civil Airborne Systems and Equipment,” developed by SAE International, and MIL STD 882E, “Standard Practice: System Safety,” developed by the U.S. Department of Defense.

Wu and Kelly [Wu 04] published a set of safety tactics in 2004, based on a survey of existing architectural approaches, which inspired much of the thinking in this chapter.

Nancy Leveson is a thought leader in the area of software and safety. If you’re working in safety-critical systems, you should become familiar with her work. You can start small with a paper like [Leveson 04], which discusses a number of software-related factors that have contributed to spacecraft accidents. Or you can start at the top with [Leveson 11], a book that treats safety in the context of today’s complex, socio-technical, software-intensive systems.

The Federal Aviation Administration is the U.S. government agency charged with oversight of the U.S. airspace system and is extremely concerned about safety. Its 2019 *System Safety Handbook* is a good practical overview of the topic. Chapter 10 of this handbook deals with software safety. You can download it from faa.gov/regulations_policies/handbooks_manuals/aviation/risk_management/ss_handbook/.

Phil Koopman is well known in the automotive safety field. He has several tutorials available online that deal with safety-critical patterns. See, for example, youtube.com/watch?v=JA5wdyOjoXg and youtube.com/watch?v=4Tdh3jq6W4Y. Koopman’s book, *Better Embedded System Software*, gives much more detail about safety patterns [Koopman 10].

Fault tree analysis dates from the early 1960s, but the granddaddy of resources for it is the U.S. Nuclear Regulatory Commission’s *Fault Tree Handbook*, published in 1981. NASA’s 2002 *Fault Tree Handbook with Aerospace Applications* is an updated comprehensive primer of the NRC handbook. Both are available online as downloadable PDF files.

Similar to Design Assurance Levels, Safety Integrity Levels (SILs) provide definitions of how safety-critical various functions are. These definitions create a common understanding among the architects involved in designing the system, but also assist with safety evaluation. The IEC 61508 Standard titled “Functional Safety of Electrical/Electronic/Programmable Electronic Safety-related Systems” defines four SILs, with SIL 4 being the most dependable and SIL 1 being the least dependable. This standard is instantiated through domain-specific standards such as IEC 62279 for the railway industry, titled “Railway Applications: Communication, Signaling and Processing Systems: Software for Railway Control and Protection Systems.”

In a world where semi-autonomous and autonomous vehicles are the subject of much research and development, functional safety is becoming increasingly more prominent. For a long time, ISO 26026 has been the standard in functional safety of road vehicles. There is

also a wave of new norms such as ANSI/UL 4600, “Standard for Safety for the Evaluation of Autonomous Vehicles and Other Products,” which tackle the challenges that emerge when software takes the wheel, figuratively and literally.

10.6 Discussion Questions

1. List 10 computer-controlled devices that are part of your everyday life right now, and hypothesize ways that a malicious or malfunctioning system could use them to hurt you.
2. Write a safety scenario that is designed to prevent a stationary robotic device (such as an assembly arm on a manufacturing line) from injuring someone, and discuss tactics to achieve it.
3. The U.S. Navy’s F/A-18 Hornet fighter aircraft was one of the early applications of fly-by-wire technology, in which onboard computers send digital commands to the control surfaces (ailerons, rudder, etc.) based on the pilot’s input to the control stick and rudder pedals. The flight control software was programmed to prevent the pilot from commanding certain violent maneuvers that might cause the aircraft to enter an unsafe flight regime. During early flight testing, which often involves pushing the aircraft to (and beyond) its utmost limits, an aircraft entered an unsafe state and “violent maneuvers” were exactly what were needed to save it—but the computers dutifully prevented them. The aircraft crashed into the ocean because of software designed to keep it safe. Write a safety scenario to address this situation, and discuss the tactics that would have prevented this outcome.
4. According to slate.com and other sources, a teenage girl in Germany “went into hiding after she forgot to set her Facebook birthday invitation to private and accidentally invited the entire Internet. After 15,000 people confirmed they were coming, the girl’s parents canceled the party, notified police, and hired private security to guard their home.” Fifteen hundred people showed up anyway, resulting in several minor injuries and untold mayhem. Is Facebook unsafe? Discuss.
5. Write a safety scenario to protect the unfortunate girl in Germany from Facebook.
6. On February 25, 1991, during the Gulf War, a U.S. Patriot missile battery failed to intercept an incoming Scud missile, which struck a barracks, killing 28 soldiers and injuring dozens. The cause of the failure was an inaccurate calculation of the time since boot due to arithmetic errors in the software that accumulated over time. Write a safety scenario that addresses the Patriot failure and discuss tactics that might have prevented it.
7. Author James Gleick (“A Bug and a Crash,” around.com/ariane.html) writes that “It took the European Space Agency 10 years and \$7 billion to produce Ariane 5, a giant rocket capable of hurling a pair of three-ton satellites into orbit with each launch. . . . All it took to explode that rocket less than a minute into its maiden voyage . . . was a small

computer program trying to stuff a 64-bit number into a 16-bit space. One bug, one crash. Of all the careless lines of code recorded in the annals of computer science, this one may stand as the most devastatingly efficient.” Write a safety scenario that addresses the Ariane 5 disaster, and discuss tactics that might have prevented it.

8. Discuss how you think safety tends to “trade off” against the quality attributes of performance, availability, and interoperability.
9. Discuss the relationship between safety and testability.
10. What is the relationship between safety and modifiability?
11. With the Air France flight 447 story in mind, discuss the relationship between safety and usability.
12. Create a list of faults or a fault tree for an automatic teller machine. Include faults dealing with hardware component failure, communications failure, software failure, running out of supplies, user errors, and security attacks. How would you use tactics to accommodate these faults?

This page intentionally left blank

11



Security

*If you reveal your secrets to the wind, you should not blame the wind
for revealing them to the trees.*

—Kahlil Gibran

Security is a measure of the system's ability to protect data and information from unauthorized access while still providing access to people and systems that are authorized. An attack—that is, an action taken against a computer system with the intention of doing harm—can take a number of forms. It may be an unauthorized attempt to access data or services or to modify data, or it may be intended to deny services to legitimate users.

The simplest approach to characterizing security focuses on three characteristics: confidentiality, integrity, and availability (CIA):

- *Confidentiality* is the property that data or services are protected from unauthorized access. For example, a hacker cannot access your income tax returns on a government computer.
- *Integrity* is the property that data or services are not subject to unauthorized manipulation. For example, your grade has not been changed since your instructor assigned it.
- *Availability* is the property that the system will be available for legitimate use. For example, a denial-of-service attack won't prevent you from ordering *this* book from an online bookstore.

We will use these characteristics in our general scenario for security.

One technique that is used in the security domain is threat modeling. An “attack tree,” which is similar to the fault tree discussed in Chapter 4, is used by security engineers to determine possible threats. The root of the tree is a successful attack, and the nodes are possible direct causes of that successful attack. Children nodes decompose the direct causes, and so forth. An attack is an attempt to compromise CIA, with the leaves of attack trees being the stimulus in the scenario. The response to the attack is to preserve CIA or deter attackers through monitoring of their activities.

Privacy

An issue closely related to security is the quality of privacy. Privacy concerns have become more important in recent years and are enshrined into law in the European Union through the General Data Protection Regulation (GDPR). Other jurisdictions have adopted similar regulations.

Achieving privacy is about limiting access to information, which in turn is about which information should be access-limited and to whom access should be allowed. The general term for information that should be kept private is personally identifiable information (PII). The National Institute of Standards and Technology (NIST) defines PII as “any information about an individual maintained by an agency, including (1) any information that can be used to distinguish or trace an individual’s identity, such as name, social security number, date and place of birth, mother’s maiden name, or biometric records; and (2) any other information that is linked or linkable to an individual, such as medical, educational, financial, and employment information.”

The question of who is permitted access to such data is more complicated. Users are routinely asked to review and agree to privacy agreements initiated by organizations. These privacy agreements detail who, outside of the collecting organization, is entitled to see PII. The collecting organization itself should have policies that govern who within that organization can have access to such data. Consider, for example, a tester for a software system. To perform tests, realistic data should be used. Does that data include PII? Generally, PII is obscured for testing purposes.

Frequently the architect, perhaps acting for the project manager, is asked to verify that PII is hidden from members of the development team who do not need to have access to PII.

11.1 Security General Scenario

From these considerations, we can now describe the individual portions of a security general scenario, which is summarized in Table 11.1.

TABLE 11.1 Security General Scenario

Portion of Scenario	Description	Possible Values
Source	The attack may be from outside the organization or from inside the organization. The source of the attack may be either a human or another system. It may have been previously identified (either correctly or incorrectly) or may be currently unknown.	▪ Human ▪ Another system which is: ▪ Inside the organization ▪ Outside the organization ▪ Previously identified ▪ Unknown

Portion of Scenario	Description	Possible Values
Stimulus	The stimulus is an attack.	An unauthorized attempt to: ▪ Display data ▪ Capture data ▪ Change or delete data ▪ Access system services ▪ Change the system's behavior ▪ Reduce availability
Artifact	What is the target of the attack?	▪ System services ▪ Data within the system ▪ A component or resources of the system ▪ Data produced or consumed by the system
Environment	What is the state of the system when the attack occurs?	The system is: ▪ Online or offline ▪ Connected to or disconnected from a network ▪ Behind a firewall or open to a network ▪ Fully operational ▪ Partially operational ▪ Not operational
Response	The system ensures that confidentiality, integrity, and availability are maintained.	Transactions are carried out in a fashion such that ▪ Data or services are protected from unauthorized access ▪ Data or services are not being manipulated without authorization ▪ Parties to a transaction are identified with assurance ▪ The parties to the transaction cannot repudiate their involvements ▪ The data, resources, and system services will be available for legitimate use The system tracks activities within it by ▪ Recording access or modification ▪ Recording attempts to access data, resources, or services ▪ Notifying appropriate entities (people or systems) when an apparent attack is occurring
Response measure	Measures of a system's response are related to the frequency of successful attacks, the time and cost to resist and repair attacks, and the consequential damage of those attacks.	One or more of the following: ▪ How much of a resource is compromised or ensured ▪ Accuracy of attack detection ▪ How much time passed before an attack was detected ▪ How many attacks were resisted ▪ How long it takes to recover from a successful attack ▪ How much data is vulnerable to a particular attack

Figure 11.1 shows a sample concrete scenario derived from the general scenario: *A disgruntled employee at a remote location attempts to improperly modify the pay rate table during normal operations. The unauthorized access is detected, the system maintains an audit trail, and the correct data is restored within one day.*

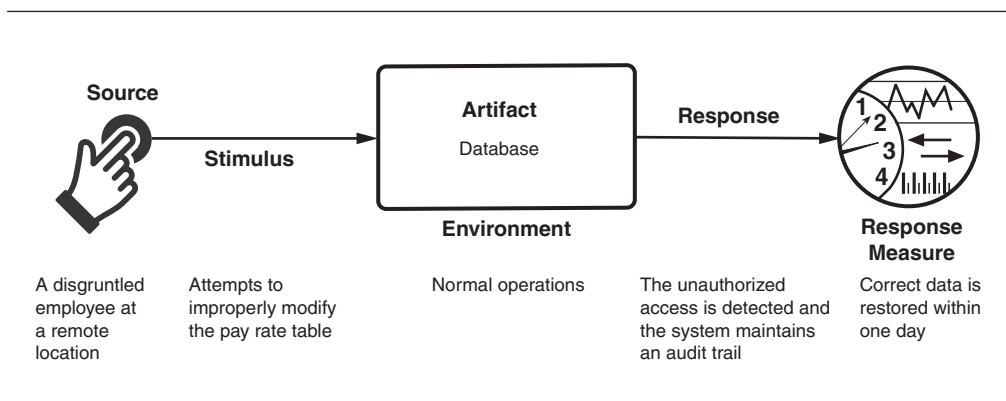


FIGURE 11.1 Sample scenario for security

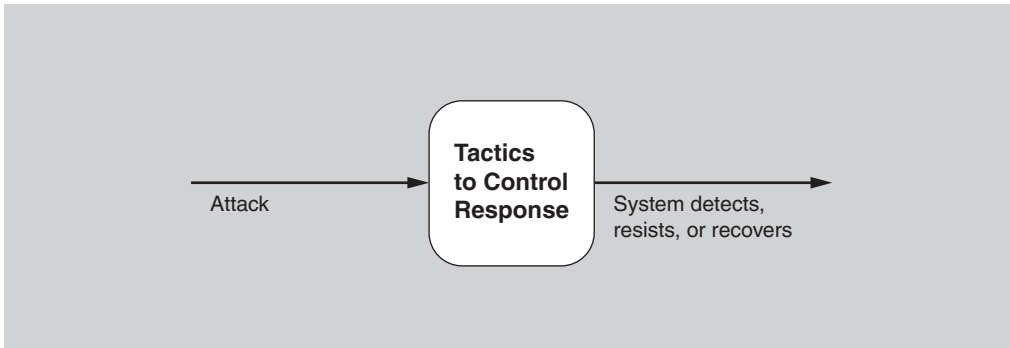
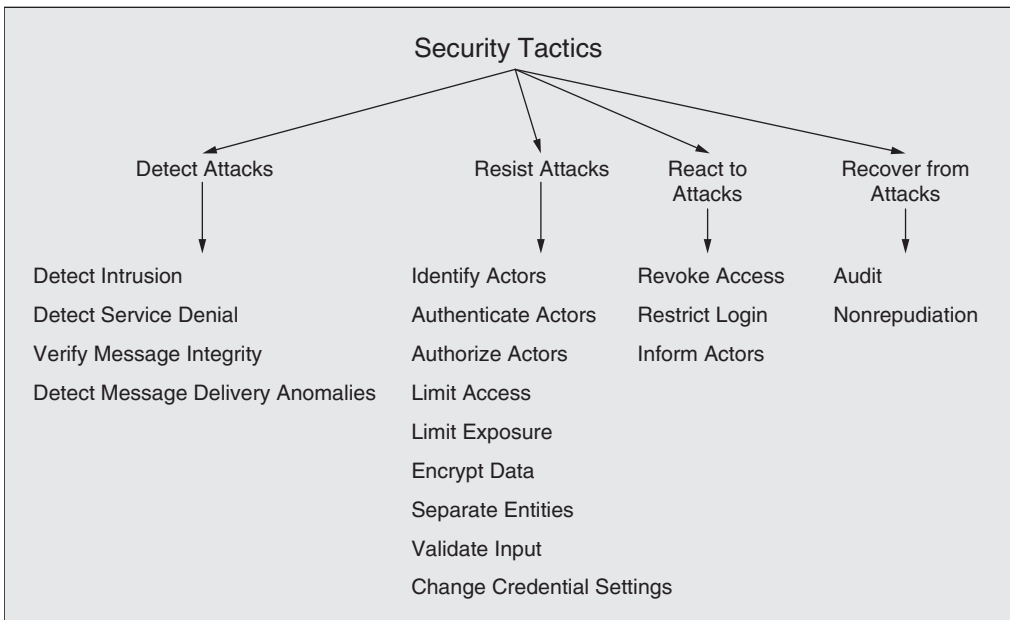
11.2 Tactics for Security

One method for thinking about how to achieve security in a system is to focus on physical security. Secure installations permit only limited access to them (e.g., by using fences and security checkpoints), have means of detecting intruders (e.g., by requiring legitimate visitors to wear badges), have deterrence mechanisms (e.g., by having armed guards), have reaction mechanisms (e.g., automatic locking of doors), and have recovery mechanisms (e.g., off-site backup). These lead to our four categories of tactics: detect, resist, react, and recover. The goal of security tactics is shown in Figure 11.2, and Figure 11.3 outlines these categories of tactics.

Detect Attacks

The detect attacks category consists of four tactics: detect intrusion, detect service denial, verify message integrity, and detect message delay.

- *Detect intrusion.* This tactic compares network traffic or service request patterns within a system to a set of signatures or known patterns of malicious behavior stored in a database. The signatures can be based on protocol characteristics, request characteristics, payload sizes, applications, source or destination address, or port number.
- *Detect service denial.* This tactic compares the pattern or signature of network traffic coming into a system to historical profiles of known denial-of-service (DoS) attacks.

**FIGURE 11.2** Goal of security tactics**FIGURE 11.3** Security tactics

- *Verify message integrity.* This tactic employs techniques such as checksums or hash values to verify the integrity of messages, resource files, deployment files, and configuration files. A checksum is a validation mechanism wherein the system separately maintains redundant information for files and messages, and uses this redundant information to verify the file or message. A hash value is a unique string generated by a hashing function, whose input could be files or messages. Even a slight change in the original files or messages results in a significant change in the hash value.
- *Detect message delivery anomalies.* This tactic seeks to detect potential man-in-the-middle-attacks, in which a malicious party is intercepting (and possibly modifying) messages. If message delivery times are normally stable, then by checking the time that it takes to deliver or receive a message, it becomes possible to detect suspicious timing behavior. Similarly, abnormal numbers of connections and disconnections may indicate such an attack.

Resist Attacks

There are a number of well-known means of resisting an attack:

- *Identify actors.* Identifying actors (users or remote computers) focuses on identifying the source of any external input to the system. Users are typically identified through user IDs. Other systems may be “identified” through access codes, IP addresses, protocols, ports, or some other means.
- *Authenticate actors.* Authentication means ensuring that an actor is actually who or what it purports to be. Passwords, one-time passwords, digital certificates, two-factor authentication, and biometric identification provide a means for authentication. Another example is CAPTCHA (Completely Automated Public Turing test to tell Computers and Humans Apart), a type of challenge–response test that is used to determine whether the user is human. Systems may require periodic reauthentication, such as when your smartphone automatically locks after a period of inactivity.
- *Authorize actors.* Authorization means ensuring that an authenticated actor has the rights to access and modify either data or services. This mechanism is usually enabled by providing some access control mechanisms within a system. Access control can be assigned per actor, per actor class, or per role.
- *Limit access.* This tactic involves limiting access to computer resources. Limiting access might mean restricting the number of access points to the resources, or restricting the type of traffic that can go through the access points. Both kinds of limits minimize the attack surface of a system. For example, a demilitarized zone (DMZ) is used when an organization wants to let external users access certain services but not access other services. The DMZ sits between the Internet and an intranet, and is protected by a pair of firewalls, one on either side. The internal firewall is a single point of access to the intranet; it functions as a limit to the number of access points as well as controls the type of traffic allowed through to the intranet.

- *Limit exposure.* This tactic focuses on minimizing the effects of damage caused by a hostile action. It is a passive defense since it does not proactively prevent attackers from doing harm. Limiting exposure is typically realized by reducing the amount of data or services that can be accessed through a single access point, and hence compromised in a single attack.
- *Encrypt data.* Confidentiality is usually achieved by applying some form of encryption to data and to communication. Encryption provides extra protection to persistently maintained data beyond that available from authorization. Communication links, by comparison, may not have authorization controls. In such cases, encryption is the only protection for passing data over publicly accessible communication links. Encryption can be symmetric (readers and writers use the same key) or asymmetric (with readers and writers use paired public and private keys).
- *Separate entities.* Separating different entities limits the scope of an attack. Separation within the system can be done through physical separation on different servers attached to different networks, the use of virtual machines, or an “air gap”—that is, by having no electronic connection between different portions of a system. Finally, sensitive data is frequently separated from nonsensitive data to reduce the possibility of attack by users who have access to nonsensitive data.
- *Validate input.* Cleaning and checking input as it is received by a system, or portion of a system, is an important early line of defense in resisting attacks. This is often implemented by using a security framework or validation class to perform actions such as filtering, canonicalization, and sanitization of input. Data validation is the main form of defense against attacks such as SQL injection, in which malicious code is inserted into SQL statements, and cross-site scripting (XSS), in which malicious code from a server runs on a client.
- *Change credential settings.* Many systems have default security settings assigned when the system is delivered. Forcing the user to change those settings will prevent attackers from gaining access to the system through settings that may be publicly available. Similarly, many systems require users to choose a new password after some maximum time period.

React to Attacks

Several tactics are intended to respond to a potential attack.

- *Revoke access.* If the system or a system administrator believes that an attack is under way, then access can be severely limited to sensitive resources, even for normally legitimate users and uses. For example, if your desktop has been compromised by a virus, your access to certain resources may be limited until the virus is removed from your system.
- *Restrict login.* Repeated failed login attempts may indicate a potential attack. Many systems limit access from a particular computer if there are repeated failed attempts to access an account from that computer. Of course, legitimate users may make mistakes

in attempting to log in, so the limited access may last for only a certain time period. In some cases, systems double the lockout time period after each unsuccessful login attempt.

- *Inform actors.* Ongoing attacks may require action by operators, other personnel, or cooperating systems. Such personnel or systems—the set of relevant actors—must be notified when the system has detected an attack.

Recover from Attacks

Once a system has detected and attempted to resist an attack, it needs to recover. Part of recovery is restoration of services. For example, additional servers or network connections may be kept in reserve for such a purpose. Since a successful attack can be considered a kind of failure, the set of availability tactics (from Chapter 4) that deal with recovering from a failure can be brought to bear for this aspect of security as well.

In addition to the availability tactics for recovery, the audit and nonrepudiation tactics can be used:

- *Audit.* We audit systems—that is, keep a record of user and system actions and their effects—to help trace the actions of, and to identify, an attacker. We may analyze audit trails to attempt to prosecute attackers or to create better defenses in the future.
- *Nonrepudiation.* This tactic guarantees that the sender of a message cannot later deny having sent the message and that the recipient cannot deny having received the message. For example, you cannot deny ordering something from the Internet, and the merchant cannot disclaim getting your order. This could be achieved with some combination of digital signatures and authentication by trusted third parties.

11.3 Tactics-Based Questionnaire for Security

Based on the tactics described in Section 11.2, we can create a set of security tactics—inspired questions, as presented in Table 11.2. To gain an overview of the architectural choices made to support security, the analyst asks each question and records the answers in the table. The answers to these questions can then be made the focus of further activities: investigation of documentation, analysis of code or other artifacts, reverse engineering of code, and so forth.

TABLE 11.2 Tactics-Based Questionnaire for Security

Tactics Group	Tactics Question	Supported? (Y/N)	Risk	Design Decisions and Location	Rationale and Assumptions
Detecting Attacks	Does the system support the detection of intrusions by, for example, comparing network traffic or service request patterns within a system to a set of signatures or known patterns of malicious behavior stored in a database? Does the system support the detection of denial-of-service attacks by, for example, comparing the pattern or signature of network traffic coming into a system to historical profiles of known DoS attacks? Does the system support the verification of message integrity via techniques such as checksums or hash values? Does the system support the detection of message delays by, for example, checking the time that it takes to deliver a message?				
Resisting Attacks	Does the system support the identification of actors through user IDs, access codes, IP addresses, protocols, ports, etc.? Does the system support the authentication of actors via, for example, passwords, digital certificates, two-factor authentication, or biometrics? Does the system support the authorization of actors, ensuring that an authenticated actor has the rights to access and modify either data or services? Does the system support limiting access to computer resources via restricting the number of access points to the resources, or restricting the type of traffic that can go through the access points? Does the system support limiting exposure by reducing the amount of data or services that can be accessed through a single access point?				

continues

TABLE 11.2 Tactics-Based Questionnaire for Security *continued*

Tactics Group	Tactics Question	Supported? (Y/N)	Risk	Design Decisions and Location	Rationale and Assumptions
Resisting Attacks	Does the system support data encryption, for data in transit or data at rest? Does the system design consider the separation of entities via physical separation on different servers attached to different networks, virtual machines, or an “air gap”? Does the system support changing credential settings, forcing the user to change those settings periodically or at critical events? Does the system validate input in a consistent, system-wide way—for example, using a security framework or validation class to perform actions such as filtering, canonicalization, and sanitization of external input?				
Reacting to Attacks	Does the system support revoking access by limiting access to sensitive resources, even for normally legitimate users and uses if an attack is under way? Does the system support restricting login in instances such as multiple failed login attempts? Does the system support informing actors such as operators, other personnel, or cooperating systems when the system has detected an attack?				
Recovering from Attacks	Does the system support maintaining an audit trail to help trace the actions of, and to identify, an attacker? Does the system guarantee the property of nonrepudiation, which guarantees that the sender of a message cannot later deny having sent the message and that the recipient cannot deny having received the message? Have you checked the “recover from faults” category of tactics from Chapter 4?				

11.4 Patterns for Security

Two of the more well-known patterns for security are intercepting validator and intrusion prevention system.

Intercepting Validator

This pattern inserts a software element—a wrapper—between the source and the destination of messages. This approach assumes greater importance when the source of the messages is outside the system. The most common responsibility of this pattern is to implement the verify message integrity tactic, but it can also incorporate tactics such as detect intrusion and detect service denial (by comparing messages to known intrusion patterns), or detect message delivery anomalies.

Benefits:

- Depending on the specific validator that you create and deploy, this pattern can cover most of the waterfront of the “detect attack” category of tactics, all in one package.

Tradeoffs:

- As always, introducing an intermediary exacts a performance price.
- Intrusion patterns change and evolve over time, so this component must be kept up-to-date so that it maintains its effectiveness. This imposes a maintenance obligation on the organization responsible for the system.

Intrusion Prevention System

An intrusion prevention system (IPS) is a standalone element whose main purpose is to identify and analyze any suspicious activity. If the activity is deemed acceptable, it is allowed. Conversely, if it is suspicious, the activity is prevented and reported. These systems look for suspicious patterns of overall usage, not just anomalous messages.

Benefits:

- These systems can encompass most of the “detect attacks” and “react to attacks” tactics.

Tradeoffs:

- The patterns of activity that an IPS looks for change and evolve over time, so the patterns database must be constantly updated.
- Systems employing an IPS incur a performance cost.
- IPSs are available as commercial off-the-shelf components, which makes them unnecessary to develop but perhaps not entirely suited to a specific application.

Other notable security patterns include compartmentalization and distributed responsibility. Both of these combine the “limit access” and “limit exposure” tactics—the former with respect to information, the latter with respect to activities.

Just as we included (by reference) tactics for availability in our list of security tactics, patterns for availability also apply to security by counteracting attacks that seek to stop the system from operating. Consider the availability patterns discussed in Chapter 4 here as well.

11.5 For Further Reading

The architectural tactics that we have described in this chapter are only one aspect of making a system secure. Other aspects include the following:

- Coding. *Secure Coding in C and C++* [Seacord 13] describes how to code securely.
- Organizational processes. Organizations must have processes that take responsibility for various aspects of security, including ensuring that systems are upgraded to put into place the latest protections. NIST 800-53 provides an enumeration of organizational processes [NIST 09]. Organizational processes must account for insider threats, which account for 15–20 percent of attacks. [Cappelli 12] discusses insider threats.
- Technical processes. Microsoft’s Security Development Lifecycle includes modeling of threats: microsoft.com/download/en/details.aspx?id=16420.

The Common Weakness Enumeration is a list of the most common categories of vulnerabilities discovered in systems, including SQL injection and XSS: <https://cwe.mitre.org/>.

NIST has published several volumes that give definitions of security terms [NIST 04], categories of security controls [NIST 06], and an enumeration of security controls that an organization could employ [NIST 09]. A security control could be a tactic, but it could also be organizational, coding, or technical in nature.

Good books on engineering systems for security include Ross Anderson’s *Security Engineering: A Guide to Building Dependable Distributed Systems*, third edition [Anderson 20], and the series of books by Bruce Schneier.

Different domains have different sets of security practices that are relevant to their domain. The Payment Card Industry (PCI), for example, has established a set of standards intended for those involved in credit card processing (pcisecuritystandards.org).

The Wikipedia page on “Security Patterns” contains brief definitions of a large number of security patterns.

Access control is commonly performed using a standard called OAuth. You can read about OAuth at <https://en.wikipedia.org/wiki/OAuth>.

11.6 Discussion Questions

1. Write a set of concrete scenarios for security for an automobile. Consider in particular how you would specify scenarios regarding control of the vehicle.

2. One of the most sophisticated attacks on record was carried out by a virus known as Stuxnet. Stuxnet first appeared in 2009, but became widely known in 2011 when it was revealed that it had apparently severely damaged or incapacitated the high-speed centrifuges involved in Iran's uranium enrichment program. Read about Stuxnet, and see if you can devise a defense strategy against it, based on the tactics described in this chapter.
3. Security and usability are often seen to be at odds with each other. Security often imposes procedures and processes that seem like needless overhead to the casual user. Nevertheless, some say that security and usability go (or should go) hand in hand, and argue that making the system easy to use securely is the best way to promote security to the users. Discuss.
4. List some examples of critical resources for security, which a DoS attack might target and try to exhaust. Which architectural mechanisms could be employed to prevent this kind of attack?
5. Which of the tactics detailed in this chapter will protect against an insider threat? Can you think of any that should be added?
6. In the United States, Netflix typically accounts for more than 10 percent of all Internet traffic. How would you recognize a DoS attack on Netflix.com? Can you create a scenario to characterize this situation?
7. The public disclosure of vulnerabilities in an organization's production systems is a matter of controversy. Discuss why this is so, and identify the pros and cons of public disclosure of vulnerabilities. How could this issue affect your role as an architect?
8. Similarly, the public disclosure of an organization's security measures and the software to achieve them (via open source software, for example) is a matter of controversy. Discuss why this is so, identify the pros and cons of public disclosure of security measures, and describe how this could affect your role as an architect.

This page intentionally left blank

12



Testability

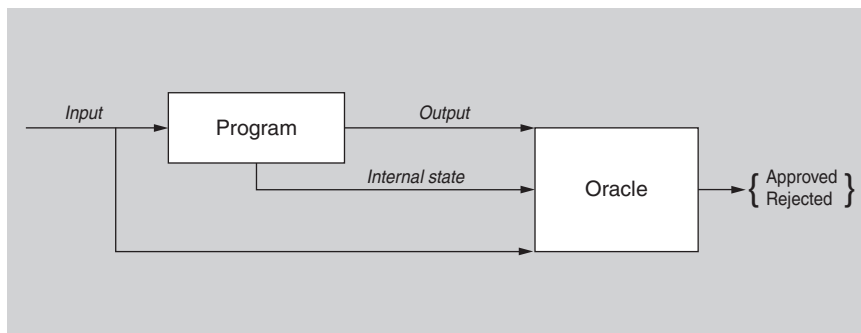
Testing leads to failure, and failure leads to understanding.
—Burt Rutan

A substantial portion of the cost of developing well-engineered systems is taken up by testing. If a carefully thought-out software architecture can reduce this cost, the payoff is large.

Software testability refers to the ease with which software can be made to demonstrate its faults through (typically execution-based) testing. Specifically, testability refers to the probability, assuming that the software has at least one fault, that it will fail on its next test execution. Intuitively, a system is testable if it “reveals” its faults easily. If a fault is present in a system, then we want it to fail during testing as quickly as possible. Of course, calculating this probability is not easy and—as you will see when we discuss response measures for testability—other measures will be used. In addition, an architecture can enhance testability by making it easier both to replicate a bug and to narrow down the possible root causes of the bug. We do not typically think of these activities as part of testability per se, but in the end just revealing a bug isn’t enough: You also need to find and fix the bug!

Figure 12.1 shows a simple model of testing in which a program processes input and produces output. An oracle is an agent (human or computational) that decides whether the output is correct by comparing the output to the expected result. Output is not just the functionally produced value, but can also include derived measures of quality attributes such as how long it took to produce the output. Figure 12.1 also indicates that the program’s internal state can be shown to the oracle, and an oracle can decide whether that state is correct—that is, it can detect whether the program has entered an erroneous state and render a judgment as to the correctness of the program. Setting and examining a program’s internal state is an aspect of testing that will figure prominently in our tactics for testability.

For a system to be properly testable, it must be possible to control each component’s inputs (and possibly manipulate its internal state) and then to observe its outputs (and possibly its internal state, either after or on the way to computing the outputs). Frequently, control and observation are accomplished through the use of a *test harness*, a set of specialized software (or in some cases, hardware) designed to exercise the software under test. Test harnesses come in various forms, and may include capabilities such as a record-and-playback capability

**FIGURE 12.1** A model of testing

for data sent across interfaces, or a simulator for an external environment in which a piece of embedded software is tested, or even distinct software that runs during production (see the sidebar “Netflix’s Simian Army”). The test harness can provide assistance in executing the test procedures and recording the output. A test harness and its accompanying infrastructure can be substantial pieces of software in their own right, with their own architecture, stakeholders, and quality attribute requirements.

Netflix’s Simian Army

Netflix distributes movies and television shows via both DVD and streaming video. Its streaming video service has been extremely successful. In fact, in 2018, Netflix’s streaming video accounted for 15 percent of the global Internet traffic. Naturally, high availability is important to Netflix.

Netflix hosts its computer services in the Amazon EC2 cloud, and the company utilizes a set of services that were originally called the “Simian Army” as a portion of its testing process. Netflix began with a Chaos Monkey, which randomly killed processes in the running system. This allows the monitoring of the effect of failed processes and gives the ability to ensure that the system will not fail or suffer serious degradation as a result of a process failure.

The Chaos Monkey acquired some friends to assist in the testing. The Netflix Simian Army included these, in addition to the Chaos Monkey:

- The Latency Monkey induced artificial delays in network communication to simulate service degradation and measured whether upstream services responded appropriately.
- The Conformity Monkey identified instances that did not adhere to best practices and shut them down. For example, if an instance did not belong to an auto-scaling group, it would not appropriately scale when demand went up.

- The Doctor Monkey tapped into health checks that ran on each instance as well as monitoring other external signs of health (e.g., CPU load) to detect unhealthy instances.
- The Janitor Monkey ensured that the Netflix cloud environment was running free of clutter and waste. It searched for unused resources and disposed of them.
- The Security Monkey was an extension of Conformity Monkey. It found security violations or vulnerabilities, such as improperly configured security groups, and terminated the offending instances. It also ensured that all SSL and digital rights management (DRM) certificates were valid and not coming up for renewal.
- The 10-18 Monkey (localization-internationalization) detected configuration and run-time problems in instances serving customers in multiple geographic regions, using different languages and character sets. The name 10-18 comes from *L10n-i18n*, a sort of shorthand for the words “localization” and “internationalization.”

Some members of the Simian Army used fault injection to place faults into the running system in a controlled and monitored fashion. Other members monitored various specialized aspects of the system and its environment. Both of these techniques have broader applicability than just for Netflix.

Given that not all faults are equal in terms of severity, more emphasis should be placed on finding the most severe faults than on finding other faults. The Simian Army reflected a determination by Netflix that the targeted faults were the most serious in terms of their impacts.

Netflix’s strategy illustrates that some systems are too complex and adaptive to be tested fully, because some of their behaviors are emergent. One aspect of testing in that arena is logging of operational data produced by the system, so that when failures occur, the logged data can be analyzed in the lab to try to reproduce the faults.

—LB

Testing is carried out by various developers, users, or quality assurance personnel. Either portions of the system or the entire system may be tested. The response measures for testability deal with how effective the tests are in discovering faults and how long it takes to perform the tests to some desired level of coverage. Test cases can be written by the developers, the testing group, or the customer. In some cases, testing actually drives development, as is the case with test-driven development.

Testing of code is a special case of validation, which entails making sure that an engineered artifact meets its stakeholders’ needs or is suitable for use. In Chapter 21, we will discuss architectural design reviews—another kind of validation, in which the artifact being tested is the architecture.

12.1 Testability General Scenario

Table 12.1 enumerates the elements of the general scenario that characterize testability.

TABLE 12.1 Testability General Scenario

Portion of Scenario	Description	Possible Values
Source	The test cases can be executed by a human or an automated test tool.	One or more of the following: ▪ Unit testers ▪ Integration testers ▪ System testers ▪ Acceptance testers ▪ End users Either run tests manually or use automated testing tools
Stimulus	A test or set of tests is initiated.	These tests serve to: ▪ Validate system functions ▪ Validate qualities ▪ Discover emerging threats to quality
Environment	Testing occurs at various events or life-cycle milestones.	The set of tests is executed due to: ▪ The completion of a coding increment such as a class, layer, or service ▪ The completed integration of a subsystem ▪ The complete implementation of the whole system ▪ The deployment of the system into a production environment ▪ The delivery of the system to a customer ▪ A testing schedule
Artifacts	The artifact is the portion of the system being tested and any required test infrastructure.	The portion being tested: ▪ A unit of code (corresponding to a module in the architecture) ▪ Components ▪ Services ▪ Subsystems ▪ The entire system ▪ The test infrastructure
Response	The system and its test infrastructure can be controlled to perform the desired tests, and the results from the test can be observed.	One or more of the following: ▪ Execute test suite and capture results ▪ Capture activity that resulted in the fault ▪ Control and monitor the state of the system

Portion of Scenario	Description	Possible Values
Response measure	Response measures are aimed at representing how easily a system under test “gives up” its faults or defects.	One or more of the following: ▪ Effort to find a fault or class of faults ▪ Effort to achieve a given percentage of state space coverage ▪ Probability of a fault being revealed by the next test ▪ Time to perform tests ▪ Effort to detect faults ▪ Length of time to prepare test infrastructure ▪ Effort required to bring the system into a specific state ▪ Reduction in risk exposure: $\text{size}(\text{loss}) \times \text{probability}(\text{loss})$

Figure 12.2 shows a concrete scenario for testability: *The developer completes a code unit during development and performs a test sequence whose results are captured and that gives 85 percent path coverage within 30 minutes.*

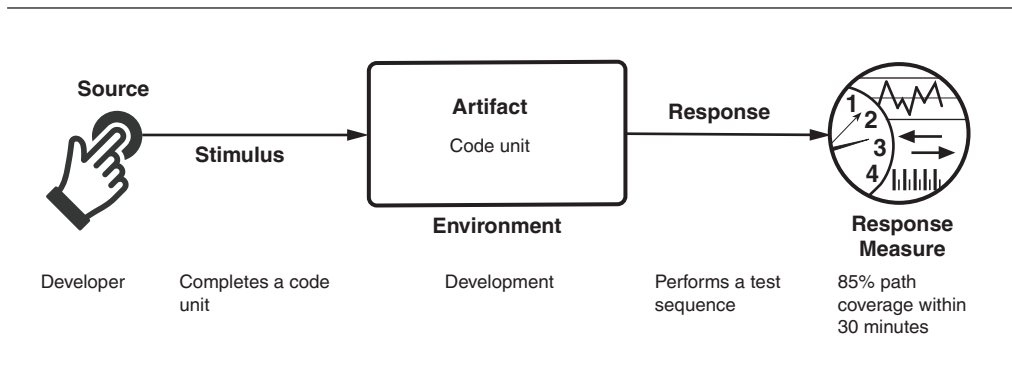


FIGURE 12.2 Sample testability scenario

12.2 Tactics for Testability

Tactics for testability are intended to promote easier, more efficient, and more capable testing. Figure 12.3 illustrates the goal of the testability tactics. Architectural techniques for enhancing the software testability have not received as much attention as other quality attribute disciplines such as modifiability, performance, and availability, but as we stated earlier, anything the architect can do to reduce the high cost of testing will yield a significant benefit.

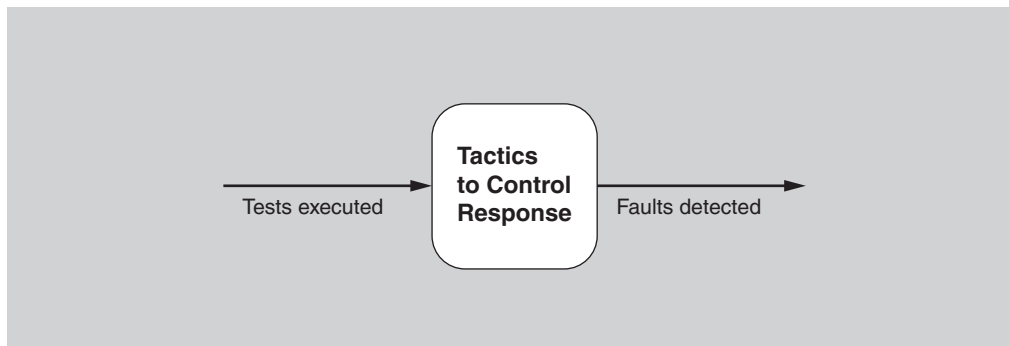


FIGURE 12.3 The goal of testability tactics

There are two categories of tactics for testability. The first category deals with adding controllability and observability to the system. The second deals with limiting complexity in the system's design.

Control and Observe System State

Control and observation are so central to testability that some authors define testability in those terms. The two go hand in hand; it makes no sense to control something if you can't observe what happens when you do. The simplest form of control and observation is to provide a software component with a set of inputs, let it do its work, and then observe its outputs. However, the control-and-observe category of testability tactics provides insights into software that go beyond its inputs and outputs. These tactics cause a component to maintain some sort of state information, allow testers to assign a value to that state information, and make that information accessible to testers on demand. The state information might be an operating state, the value of some key variable, performance load, intermediate process steps, or anything else useful to re-creating component behavior. Specific tactics include the following:

- *Specialized interfaces.* Having specialized testing interfaces allows you to control or capture variable values for a component either through the application of a test harness or through normal execution. Examples of specialized test routines, some of which might otherwise not be available except for testing purposes, include these:
 - A *set* and *get* method for important variables, modes, or attributes
 - A *report* method that returns the full state of the object
 - A *reset* method to set the internal state (e.g., all the attributes of a class) to a specified internal state
 - A method to turn on verbose output, various levels of event logging, performance instrumentation, or resource monitoring

Specialized testing interfaces and methods should be clearly identified or kept separate from the access methods and interfaces for required functionality, so that they can be removed if needed. Note, however, that in performance-critical and some safety-critical systems, it is problematic to field different code than that which was tested. If you remove the test code, how will you know the code released has the same behavior, particularly the same timing behavior, as the code you tested? Thus this strategy is more effective for other kinds of systems.

- *Record/playback.* The state that caused a fault is often difficult to re-create. Recording the state when it crosses an interface allows that state to be used to “play the system back” and to re-create the fault. *Record* refers to capturing information crossing an interface and *playback* refers to using it as input for further testing.
- *Localize state storage.* To start a system, subsystem, or component in an arbitrary state for a test, it is most convenient if that state is stored in a single place. By contrast, if the state is buried or distributed, this approach becomes difficult, if not impossible. The state can be fine-grained, even bit-level, or coarse-grained to represent broad abstractions or overall operational modes. The choice of granularity depends on how the states will be used in testing. A convenient way to “externalize” state storage (i.e., to make it amenable to manipulation through interface features) is to use a state machine (or state machine object) as the mechanism to track and report current state.
- *Abstract data sources.* Similar to the case when controlling a program’s state, the ability to control its input data makes it easier to test. Abstracting the interfaces lets you substitute test data more easily. For example, if you have a database of customer transactions, you could design your architecture so that you can readily point your test system at other test databases, or possibly even to files of test data instead, without having to change your functional code.
- *Sandbox.* “Sandboxing” refers to isolating an instance of the system from the real world to enable experimentation that is unconstrained by any worries about having to undo the consequences of the experiment. Testing is facilitated by the ability to operate the system in such a way that it has no permanent consequences, or so that any consequences can be rolled back. The sandbox tactic can be used for scenario analysis, training, and simulation. Simulation, in particular, is a commonly employed strategy for testing and training in contexts where failure in the real world might lead to severe consequences.

One common form of sandboxing is to virtualize resources. Testing a system often involves interacting with resources whose behavior is outside the system’s control. Using a sandbox, you can build a version of the resource whose behavior is under your control. For example, the system clock’s behavior is typically not under our control—it increments one second each second. Thus, if we want to make the system think it’s midnight on the day when all of the data structures are supposed to overflow, we need a way to do that, because waiting around is a poor choice. When we can abstract system time from clock time, we can allow the system (or components) to run at faster than wall-clock time, and test the system (or components) at critical time boundaries such as the next transition to or from Daylight Savings Time. Similar virtualizations could be done for other resources, such as the memory, battery, network, and so on. Stubs, mocks, and dependency injection are simple but effective forms of virtualization.

- *Executable assertions.* With this tactic, assertions are (usually) hand-coded and placed at desired locations to indicate when and where a program is in a faulty state. The assertions are often designed to check that data values satisfy specified constraints. Assertions are defined in terms of specific data declarations, and they must be placed where the data values are referenced or modified. Assertions can be expressed as pre- and post-conditions for each method and also as class-level invariants. This increases the system's observability, as an assertion can be flagged as having failed. Assertions systematically inserted where data values change can be seen as a manual way to produce an "extended" type. Essentially, the user is annotating a type with additional checking code. Anytime an object of that type is modified, the checking code executes automatically, with warnings being generated if any conditions are violated. To the extent that the assertions cover the test cases, they effectively embed the test oracle in the code—assuming the assertions are correct and correctly coded.

All of these tactics add some capability or abstraction to the software that (were we not interested in testing) otherwise would not be there. They can be seen as augmenting bare-bones, get-the-job-done software with more elaborate software that has some special capabilities designed to enhance the efficiency and effectiveness of testing.

In addition to the testability tactics, a number of techniques are available for replacing one component with a different version of itself that facilitates testing:

- *Component replacement* simply swaps the implementation of a component with a different implementation that (in the case of testability) has features that facilitate testing. Component replacement is often accomplished in a system's build scripts.
- *Preprocessor macros*, when activated, can expand to state-reporting code or activate probe statements that return or display information, or return control to a testing console.
- *Aspects* (in aspect-oriented programs) can handle the cross-cutting concern of how the state is reported.

Limit Complexity

Complex software is much harder to test. Its operating state space is large, and (all else being equal) it is more difficult to re-create an exact state in a large state space than to do so in a small state space. Because testing is not just about making the software fail, but also about finding the fault that caused the failure so that it can be removed, we are often concerned with making behavior repeatable. This category includes two tactics:

- *Limit structural complexity.* This tactic includes avoiding or resolving cyclic dependencies between components, isolating and encapsulating dependencies on the external environment, and reducing dependencies between components in general (typically realized by lowering the coupling between components). For example, in object-oriented systems you can simplify the inheritance hierarchy:
 - Limit the number of classes from which a class is derived, or the number of classes derived from a class.
 - Limit the depth of the inheritance tree, and the number of children of a class.
 - Limit polymorphism and dynamic calls.

One structural metric that has been shown empirically to correlate to testability is the *response* of a class. The response of class C is a count of the number of methods of C plus the number of methods of other classes that are invoked by the methods of C. Keeping this metric low can increase testability. In addition, architecture-level coupling metrics, such as propagation cost and decoupling level, can be used to measure and track the overall level of coupling in a system's architecture.

Ensuring that the system has high cohesion, loose coupling, and separation of concerns—all modifiability tactics (see Chapter 8)—can also help with testability. These characteristics limit the complexity of the architectural elements by giving each element a focused task such that it has limited interactions with other elements. Separation of concerns can help achieve controllability and observability, as well as reduce the size of the overall program's state space.

Finally, some architectural patterns lend themselves to testability. In a layered pattern, you can test lower layers first, then test higher layers with confidence in the lower layers.

- *Limit nondeterminism.* The counterpart to limiting structural complexity is limiting behavioral complexity. When it comes to testing, nondeterminism is a pernicious form of complex behavior, and nondeterministic systems are more difficult to test than deterministic systems. This tactic involves finding all the sources of nondeterminism, such as unconstrained parallelism, and weeding them out to the extent possible. Some sources of nondeterminism are unavoidable—for instance, in multi-threaded systems that respond to unpredictable events—but for such systems, other tactics (such as record/playback) are available to help manage this complexity.

Figure 12.4 summarizes the tactics used for testability.

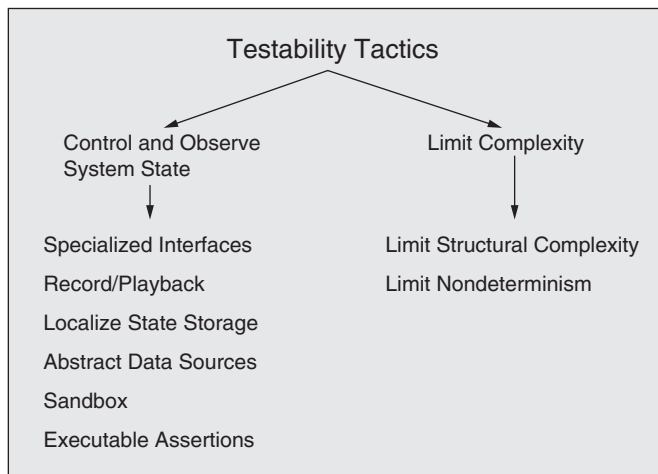


FIGURE 12.4 Testability tactics

12.3 Tactics-Based Questionnaire for Testability

Based on the tactics described in Section 12.2, we can create a set of tactics-inspired questions, as presented in Table 12.2. To gain an overview of the architectural choices made to support testability, the analyst asks each question and records the answers in the table. The answers to these questions can then be made the focus of further activities: investigation of documentation, analysis of code or other artifacts, reverse engineering of code, and so forth.

TABLE 12.2 Tactics-Based Questionnaire for Testability

Tactics Group	Tactics Question	Supported? (Y/N)	Risk	Design Decisions and Location	Rationale and Assumptions
Control and Observe System State	Does your system have specialized interfaces for getting and setting values?				
	Does your system have a record/playback mechanism?				
	Is your system's state storage localized ?				
	Does your system abstract its data sources ?				
	Can some or all of your system operate in a sandbox ?				
Limit Complexity	Is there a role for executable assertions in your system?				
	Does your system limit structural complexity in a systematic way?				
	Is there nondeterminism in your system, and is there a way to control or limit this nondeterminism ?				

12.4 Patterns for Testability

Patterns for testability all make it easier to decouple test-specific code from the actual functionality of a system. We discuss three patterns here: dependency injection, strategy, and intercepting filter.

Dependency Injection Pattern

In the dependency injection pattern, a client's dependencies are separated from its behavior. This pattern makes use of inversion of control. Unlike in traditional declarative programming, where control and dependencies reside explicitly in the code, inversion of control dependencies means that control and dependencies are provided from, and injected into the code, by some external source.

In this pattern, there are four roles:

- A service (that you want to make broadly available)
- A client of the service
- An interface (used by the client, implemented by the service)
- An injector (that creates an instance of the service and injects it into the client)

When an interface creates the service and injects it into the client, a client is written with no knowledge of a concrete implementation. In other words, all of the implementation specifics are injected, typically at runtime.

Benefits:

- Test instances can be injected (rather than production instances), and these test instances can manage and monitor the state of the service. Thus the client can be written with no knowledge of how it is to be tested. This is, in fact, how many modern testing frameworks are implemented.

Tradeoffs:

- Dependency injection makes runtime performance less predictable, because it might change the behavior being tested.
- Adding this pattern adds a small amount of up-front complexity and may require retraining of developers to think in terms of inversion of control.

Strategy Pattern

In the strategy pattern, a class's behavior can be changed at runtime. This pattern is often employed when multiple algorithms can be employed to perform a given task, and the specific algorithm to be used can be chosen dynamically. The class simply contains an abstract method for the desired functionality, with the concrete version of this method being selected based on contextual factors. This pattern is often used to replace non-test versions of some functionality with test versions that provide additional outputs, additional internal sanity checking, and so forth.

Benefits:

- This pattern makes classes simpler, by not combining multiple concerns (such as different algorithms for the same function) into a single class.

Tradeoffs:

- The strategy pattern, like all design patterns, adds a small amount of up-front complexity. If the class is simple or if there are few runtime choices, this added complexity is likely wasted.
- For small classes, the strategy pattern can make code slightly less readable. However, as complexity grows, breaking up the class in this way can enhance readability.

Intercepting Filter Pattern

The intercepting filter pattern is used to inject pre- and post-processing to a request or a response between a client and a service. Any number of filters can be defined and applied, in an arbitrary order, to the request before passing the request to the eventual service. For example, logging and authentication services are filters that are often useful to implement once and apply universally. Testing filters can be inserted in this way, without disturbing any of the other processing in the system.

Benefits:

- This pattern, like the strategy pattern, makes classes simpler, by not placing all of the pre- and post-processing logic in the class.
- Using an intercepting filter can be a strong motivator for reuse and can dramatically reduce the size of the code base.

Tradeoffs:

- If a large amount of data is being passed to the service, this pattern can be highly inefficient and can add a nontrivial amount of latency, as each filter makes a complete pass over the entire input.

12.5 For Further Reading

The literature on software testing would sink a battleship, but the writing about how to make your system more testable from an architectural standpoint is less voluminous. For a good overview of testing, see [Binder 00]. Jeff Voas's foundational work on testability and the relationship between testability and reliability is worth investigating, too. There are several papers to choose from, but [Voas 95] is a good start that will point you to others.

Bertolino and Strigini [Bertolino 96a, 96b] are the developers of the model of testing shown in Figure 12.1.

“Uncle Bob” Martin has written extensively on test-driven development and the relationship between architecture and testing. The best book on this is Robert C. Martin’s *Clean Architecture: A Craftsman’s Guide to Software Structure and Design* [Martin 17]. An early and authoritative reference for test-driven development was written by Kent Beck: *Test-Driven Development by Example* [Beck 02].

The propagation cost coupling metric was first described in [MacCormack 06]. The decoupling level metric was described in [Mo 16].

Model checking is a technique that symbolically executes all possible code paths. The size of a system that can be validated using model checking is limited, but device drivers and microkernels have successfully been model checked. See https://en.wikipedia.org/wiki/Model_checking for a list of model checking tools.

12.6 Discussion Questions

1. A testable system is one that gives up its faults easily. That is, if a system contains a fault, then it doesn't take long or much effort to make that fault show up. In contrast, fault tolerance is all about designing systems that jealously hide their faults; there, the whole idea is to make it very difficult for a system to reveal its faults. Is it possible to design a system that is both highly testable *and* highly fault tolerant, or are these two design goals inherently incompatible? Discuss.
2. What other quality attributes do you think testability is most in conflict with? What other quality attributes do you think testability is most compatible with?
3. Many of the tactics for testability are also useful for achieving modifiability. Why do you think that is?
4. Write some concrete testability scenarios for a GPS-based navigation app. What tactics would you employ in a design to respond to these scenarios?
5. One of our tactics is to limit nondeterminism, and one method is to use locking to enforce synchronization. What impact does the use of locks have on other quality attributes?
6. Suppose you're building the next great social networking system. You anticipate that within a month of your debut, you will have half a million users. You can't pay half a million people to test your system, yet it has to be robust and easy to use when all half a million are banging away at it. What should you do? What tactics will help you? Write a testability scenario for this social network system.
7. Suppose you use executable assertions to improve testability. Make a case for, and then a case against, allowing the assertions to run in the production system as opposed to removing them after testing.

This page intentionally left blank

13



Usability

People ignore design that ignores people.
—Frank Chimero

Usability is concerned with how easy it is for the user to accomplish a desired task and the kind of user support that the system provides. Over the years, a focus on usability has shown itself to be one of the cheapest and easiest ways to improve a system's quality (or more precisely, the user's perception of quality) and hence end-user satisfaction.

Usability comprises the following areas:

- *Learning system features.* If the user is unfamiliar with a particular system or a particular aspect of it, what can the system do to make the task of learning easier? This might include providing help features.
- *Using a system efficiently.* What can the system do to make the user more efficient in its operation? This might include enabling the user to redirect the system after issuing a command. For example, the user may wish to suspend one task, perform several operations, and then resume that task.
- *Minimizing the impact of user errors.* What can the system do to ensure that a user error has minimal impact? For example, the user may wish to cancel a command issued incorrectly or undo its effects.
- *Adapting the system to user needs.* How can the user (or the system itself) adapt to make the user's task easier? For example, the system may automatically fill in URLs based on a user's past entries.
- *Increasing confidence and satisfaction.* What does the system do to give the user confidence that the correct action is being taken? For example, providing feedback that indicates that the system is performing a long-running task, along with the completion percentage so far, will increase the user's confidence in the system.

Researchers focusing on human–computer interactions have used the terms *user initiative*, *system initiative*, and *mixed initiative* to describe which of the human–computer pair takes the initiative in performing certain actions and how the interaction proceeds. Usability scenarios can combine initiatives from both perspectives. For example, when canceling a command, the user issues a cancel (user initiative) and the system responds. During the cancel,

however, the system may display a progress indicator (system initiative). Thus the cancel operation may comprise a mixed initiative. In this chapter, we will use this distinction between user initiative and system initiative to discuss the tactics that the architect uses to achieve the various scenarios.

There is a strong connection between the achievement of usability and modifiability. The user interface design process consists of generating and then testing a user interface design. It is highly unlikely that you will get this right the first time, so you should plan to iterate this process—and hence you should design your architecture to make that iteration less painful. This is why usability is strongly connected to modifiability. As you iterate, deficiencies in the design are—one hopes—corrected and the process repeats.

This connection has resulted in standard patterns to support user interface design. Indeed, one of the most helpful things you can do to achieve usability is to modify your system, over and over, to make it better as you learn from your users and discover improvements to be made.

13.1 Usability General Scenario

Table 13.1 enumerates the elements of the general scenario that characterize usability.

TABLE 13.1 Usability General Scenario

Portion of Scenario	Description	Possible Values
Source	Where does the stimulus come from?	The end user (who may be in a specialized role, such as a system or network administrator) is the primary source of the stimulus for usability. An external event arriving at a system (to which the user may react) may also be a stimulus source.
Stimulus	What does the end user want?	End user wants to: ▪ Use a system efficiently ▪ Learn to use the system ▪ Minimize the impact of errors ▪ Adapt the system ▪ Configure the system
Environment	When does the stimulus reach the system?	The user actions with which usability is concerned always occur at runtime or at system configuration time.
Artifacts	What portion of the system is being stimulated?	Common examples include: ▪ A GUI ▪ A command-line interface ▪ A voice interface ▪ A touch screen

Portion of Scenario	Description	Possible Values
Response	How should the system respond?	The system should: ▪ Provide the user with the features needed ▪ Anticipate the user's needs ▪ Provide appropriate feedback to the user
Response measure	How is the response measured?	One or more of the following: ▪ Task time ▪ Number of errors ▪ Learning time ▪ Ratio of learning time to task time ▪ Number of tasks accomplished ▪ User satisfaction ▪ Gain of user knowledge ▪ Ratio of successful operations to total operations ▪ Amount of time or data lost when an error occurs

Figure 13.1 gives an example of a concrete usability scenario that you could generate using Table 13.1: *The user downloads a new application and is using it productively after 2 minutes of experimentation.*

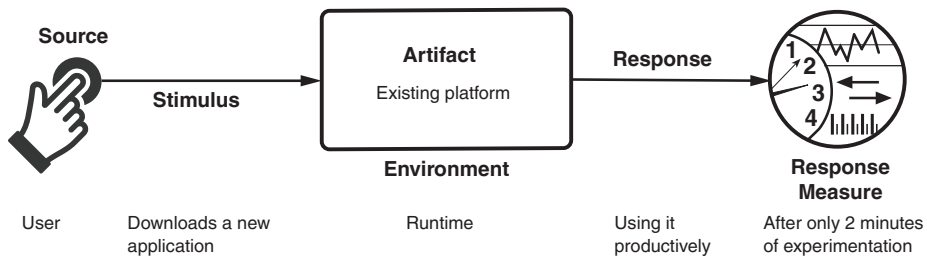


FIGURE 13.1 Sample usability scenario

13.2 Tactics for Usability

Figure 13.2 shows the goal of the set of usability tactics.

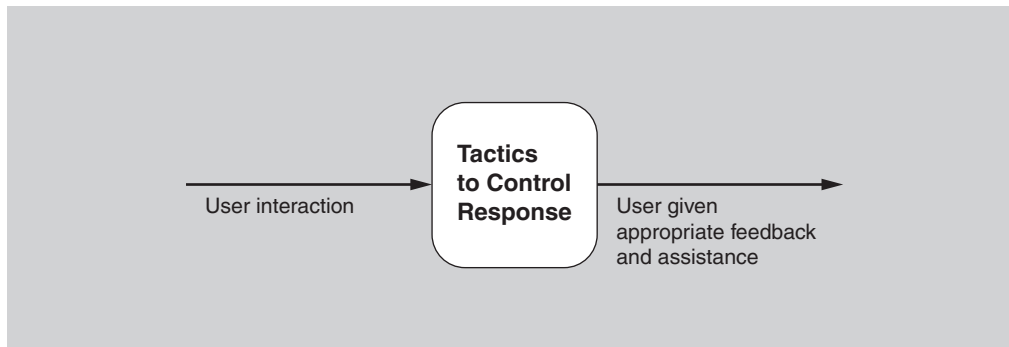


FIGURE 13.2 The goal of usability tactics

Support User Initiative

Once a system is executing, usability is enhanced by giving the user feedback about what the system is doing and by allowing the user to make appropriate responses. For example, the tactics described next—*cancel*, *undo*, *pause/resume*, and *aggregate*—support the user in either correcting errors or being more efficient.

The architect designs a response for user initiative by enumerating and allocating the responsibilities of the system to respond to the user command. Here are some common examples of tactics to support user initiative:

- *Cancel*. When the user issues a cancel command, the system must be listening for it (thus there is the responsibility to have a constant listener that is not blocked by the actions of whatever is being canceled); the activity being canceled must be terminated; any resources being used by the canceled activity must be freed; and components that are collaborating with the canceled activity must be informed so that they can also take appropriate action.
- *Undo*. To support the ability to undo, the system must maintain a sufficient amount of information about system state so that an earlier state may be restored, at the user's request. Such a record may take the form of state "snapshots"—for example, checkpoints—or a set of reversible operations. Not all operations can be easily reversed. For example, changing all occurrences of the letter "a" to the letter "b" in a document cannot be reversed by changing all instances of "b" to "a", because some of those instances of "b" may have existed prior to the original change. In such a case, the system must maintain a

more elaborate record of the change. Of course, some operations cannot be undone at all: You can't unship a package or unfire a missile, for example.

Undo comes in flavors. Some systems allow a single undo (where invoking undo again reverts you to the state in which you commanded the first undo, essentially undoing the undo). In other systems, commanding multiple undo operations steps you back through many previous states, either up to some limit or all the way back to the time when the application was last opened.

- *Pause/resume.* When a user has initiated a long-running operation—say, downloading a large file or a set of files from a server—it is often useful to provide the ability to pause and resume the operation. Pausing a long-running operation may be done to temporarily free resources so that they may be reallocated to other tasks.
- *Aggregate.* When a user is performing repetitive operations, or operations that affect a large number of objects in the same way, it is useful to provide the ability to aggregate the lower-level objects into a single group, so that the operation may be applied to the group, thus freeing the user from the drudgery, and potential for mistakes, of doing the same operation repeatedly. An example is aggregating all of the objects in a slide and changing the text to 14-point font.

Support System Initiative

When the system takes the initiative, it must rely on a model of the user, a model of the task being undertaken by the user, or a model of the system state. Each model requires various types of input to accomplish its initiative. The support system initiative tactics identify the models the system uses to predict either its own behavior or the user's intention. Encapsulating this information will make it easier to tailor or modify it. Tailoring and modification can either be dynamically based on past user behavior or happen offline during development. The relevant tactics are described here:

- *Maintain task model.* The task model is used to determine context so the system can have some idea of what the user is attempting to do and provide assistance. For example, many search engines provide predictive type-ahead capabilities, and many mail clients provide spell-correction. Both of these functions are based on task models.
- *Maintain user model.* This model explicitly represents the user's knowledge of the system, the user's behavior in terms of expected response time, and other aspects specific to a user or a class of users. For example, language-learning apps are constantly monitoring areas where a user makes mistakes and then providing additional exercises to correct those behaviors. A special case of this tactic is commonly found in user interface *customization*, wherein a user can explicitly modify the system's user model.
- *Maintain system model.* The system maintains an explicit model of itself. This is used to determine expected system behavior so that appropriate feedback can be given to the user. A common manifestation of a system model is a progress bar that predicts the time needed to complete the current activity.

Figure 13.3 summarizes the tactics to achieve usability.

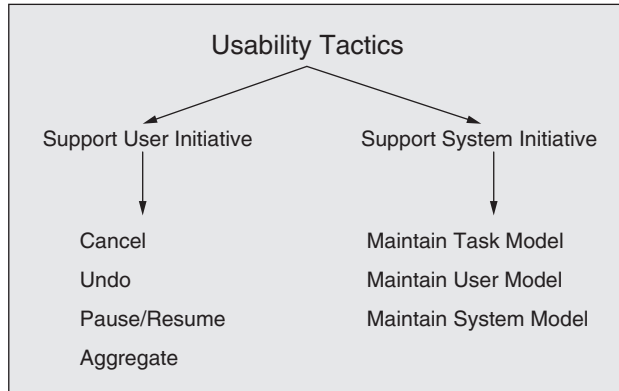


FIGURE 13.3 Usability tactics

13.3 Tactics-Based Questionnaire for Usability

Based on the tactics described in Section 13.2, we can create a set of usability tactics–inspired questions, as presented in Table 13.2. To gain an overview of the architectural choices made to support usability, the analyst asks each question and records the answers in the table. The answers to these questions can then be made the focus of further activities: investigation of documentation, analysis of code or other artifacts, reverse engineering of code, and so forth.

TABLE 13.2 Tactics-Based Questionnaire for Usability

Tactics Group	Tactics Question	Supported? (Y/N)	Risk	Design Decisions and Location	Rationale and Assumptions
Support User Initiative	Is the system able to listen to and respond to a cancel command?				
	Is it possible to undo the last command, or the last several commands?				
	Is it possible to pause and then resume long-running operations?				
	Is it possible to aggregate UI objects into a group and apply operations on the group?				

Tactics Group	Tactics Question	Supported? (Y/N)	Risk	Design Decisions and Location	Rationale and Assumptions
Support System Initiative	Does the system maintain a model of the task ?				
	Does the system maintain a model of the user ?				
	Does the system maintain a model of itself ?				

13.4 Patterns for Usability

We will briefly discuss three usability patterns: model-view-controller (MVC) and its variants, observer, and memento. These patterns primarily promote usability by promoting separation of concerns, which in turn makes it easy to iterate the design of a user interface. Other kinds of patterns are also possible—including patterns used in the design of the user interface itself, such as breadcrumbs, shopping cart, or progressive disclosure—but we will not discuss them here.

Model-View-Controller

MVC is likely the most widely known pattern for usability. It comes in many variants, such as MVP (model-view-presenter), MVVM (model-view-view-model), MVA (model-view-adapter), and so forth. Essentially all of these patterns are focused on separating the model—the underlying “business” logic of the system—from its realization in one or more UI views. In the original MVC model, the model would send updates to a view, which a user would see and interact with. User interactions—key presses, button clicks, mouse motions, and so forth—are transmitted to the controller, which interprets them as operations on the model and then sends those operations to the model, which changes its state in response. The reverse path was also a portion of the original MVC pattern. That is, the model might be changed and the controller would send updates to the view.

The sending of updates depends on whether the MVC is in one process or is distributed across processes (and potentially across the network). If the MVC is in one process, then the updates are sent using the observer pattern (discussed in the next subsection). If the MVC is distributed across processes, then the publish-subscribe pattern is often used to send updates (see Chapter 8).

Benefits:

- Because MVC promotes clear separation of concerns, changes to one aspect of the system, such as the layout of the UI (the view), often have no consequences for the model or the controller.

- Additionally, because MVC promotes separation of concerns, developers can be working on all aspects of the pattern—model, view, and controller—relatively independently and in parallel. These separate aspects can also be tested in parallel.
- A model can be used in systems with different views, or a view might be used in systems with different models.

Tradeoffs:

- MVC can become burdensome for complex UIs, as information is often sprinkled throughout several components. For example, if there are multiple views of the same model, a change to the model may require changes to several otherwise unrelated components.
- For simple UIs, MVC adds up-front complexity that may not pay off in downstream savings.
- MVC adds a small amount of latency to user interactions. While this is generally acceptable, it might be problematic for applications that require very low latency.

Observer

The observer pattern is a way to link some functionality with one or more views. This pattern has a *subject*—the entity being observed—and one or more *observers* of that subject. Observers need to register themselves with the subject; then, when the state of the subject changes, the observers are notified. This pattern is often used to implement MVC (and its variants)—for example, as a way to notify the various views of changes to the model.

Benefits:

- This pattern separates some underlying functionality from the concern of how, and how many times, this functionality is presented.
- The observer pattern makes it easy to change the bindings between the subject and the observers at runtime.

Tradeoffs:

- The observer pattern is overkill if multiple views of the subject are not required.
- The observer pattern requires that all observers register and de-register with the subject. If observers neglect to de-register, then their memory is never freed, which effectively results in a memory leak. In addition, this can negatively affect performance, since obsolete observers will continue to be invoked.
- Observers may need to do considerable work to determine if and how to reflect a state update, and this work may be repeated for each observer. For example, suppose the subject is changing its state at a fine granularity, such as a temperature sensor that reports 1/100th degree fluctuations, but the view updates changes only in full degrees. In such cases where there is an “impedance mismatch,” substantial processing resources may be wasted.

Memento

The memento pattern is a common way to implement the undo tactic. This pattern features three major components: the *originator*, the *caretaker*, and the *memento*. The originator is processing some stream of events that change its state (originating from user interaction). The caretaker is sending events to the originator that cause it to change its state. When the caretaker is about to change the state of the originator, it can request a memento—a snapshot of the existing state—and can use this artifact to restore that existing state if needed, by simply passing the memento back to the originator. In this way, the caretaker knows nothing about how state is managed; the memento is simply an abstraction that the caretaker employs.

Benefits:

- The obvious benefit of this pattern is that you delegate the complicated process of implementing undo, and figuring out what state to preserve, to the class that is actually creating and managing that state. In consequence, the originator's abstraction is preserved and the rest of the system does not need to know the details.

Tradeoffs:

- Depending on the nature of the state being preserved, the memento can consume arbitrarily large amounts of memory, which can affect performance. In a very large document, try cutting and pasting many large sections, and then undoing all of that. This is likely to result in your text processor noticeably slowing down.
- In some programming languages, it is difficult to enforce the memento as an opaque abstraction.

13.5 For Further Reading

Claire Marie Karat has investigated the relation between usability and business advantage [Karat 94].

Jakob Nielsen has also written extensively on this topic, including a calculation of the ROI of usability [Nielsen 08].

Bonnie John and Len Bass have investigated the relation between usability and software architecture. They have enumerated approximately two dozen usability scenarios that have architectural impact and given associated patterns for these scenarios [Bass 03].

Greg Hartman has defined attentiveness as the system's ability to support user initiative and allow cancel or pause/resume [Hartman 10].

13.6 Discussion Questions

1. Write a concrete usability scenario for your automobile that specifies how long it takes you to set your favorite radio stations. Now consider another part of the driver experience

and create scenarios that test other aspects of the response measures from the general scenario table (Table 13.1).

2. How might usability trade off against security? How might it trade off against performance?
3. Pick a few of your favorite websites that do similar things, such as social networking or online shopping. Now pick one or two appropriate responses from the usability general scenario (such as “anticipate the user’s need”) and an appropriate corresponding response measure. Using the response and response measure you chose, compare the websites’ usability.
4. Why is it that in so many systems, the cancel button in a dialog box appears to be unresponsive? Which architectural principles do you think were ignored in these systems?
5. Why do you think that progress bars frequently behave erratically, moving from 10 to 90 percent in one step and then getting stuck on 90 percent?
6. Research the crash of Air France flight 296 into the forest at Habsheim, France, in 1988. The pilots said they were unable to read the digital display of the radio altimeter or hear its audible readout. In this context, discuss the relationship between usability and safety.

14



Working with Other Quality Attributes

Quality is not what happens when what you do matches your intentions. It is what happens when what you do matches your customers' expectations.
—Guaspari

Chapters 4–13 each dealt with a particular quality attribute (QA) that is important to software systems. Each of those chapters discussed how its particular QA is defined, gave a general scenario for that QA, and showed how to write specific scenarios to express precise shades of meaning concerning that QA. In addition, each provided a collection of techniques to achieve that QA in an architecture. In short, each chapter presented a kind of portfolio for specifying and designing to achieve a particular QA.

However, as you can no doubt infer, those ten chapters only begin to scratch the surface of the various QAs that you might need in a software system you're working on.

This chapter will show how to build the same kind of specification and design approach for a QA not covered in our “A list.”

14.1 Other Kinds of Quality Attributes

The quality attributes covered so far in Part II of this book all have something in common: They deal with either the system in operation, or the development project that creates and fields the system. Put another way, to measure one of those QAs, either you measure the system while it is running (availability, energy efficiency, performance, security, safety, usability), or you measure the people doing something to the system while it is not (modifiability, deployability, integrability, testability). While these certainly give you an “A list” of important QAs, there are other qualities that could be equally useful.

Quality Attributes of the Architecture

Another category of QAs focuses on measuring the architecture itself. Here are three examples:

- *Buildability*. This QA measures how well the architecture lends itself to rapid and efficient development. It is measured by the cost (typically in money or time) that it takes to turn the architecture into a working product that meets all of its requirements. In that sense, it resembles the other QAs that measure a development project, but it differs in that the knowledge targeted by the measurement relates to the architecture itself.
- *Conceptual integrity*. Conceptual integrity refers to consistency in the design of the architecture, and it contributes to the architecture's understandability and leads to less confusion and more predictability in its implementation and maintenance. Conceptual integrity demands that the same thing is done in the same way through the architecture. In an architecture with conceptual integrity, less is more. For example, there are countless ways that components can send information to each other: messages, data structures, signaling of events, and so forth. An architecture with conceptual integrity would feature a small number of ways, and provide alternatives only if there is a compelling reason to do so. Similarly, components should all report and handle errors in the same way, log events or transactions in the same way, interact with the user in the same way, sanitize data in the same way, and so forth.
- *Marketability*. An architecture's "marketability" is another QA of concern. Some systems are well known for their architectures, and these architectures sometimes carry a meaning all their own, independent of what other QAs they bring to the system. The current emphasis on building cloud-based and micro-service-based systems has taught us that the perception of an architecture can be at least as important as the actual qualities that the architecture brings. Many organizations, for example, have felt compelled to build cloud-based systems (or some other *technologie du jour*) whether or not that was the correct technical choice.

Development Distributability

Development distributability is the quality of designing the software to support distributed software development. Like modifiability, this quality is measured in terms of the activities of a development project. Many systems these days are developed using globally distributed teams. One problem that must be overcome when adopting this approach is coordinating the teams' activities. The system should be designed so that coordination among teams is minimized—that is, the major subsystems should exhibit low coupling. This minimal coordination needs to be achieved both for the code and for the data model. Teams working on modules that communicate with each other may need to negotiate the interfaces of those modules. When a module is used by many other modules, each developed by a different team, communication and negotiation become more complex and burdensome. Thus the architectural structure and the social (and business) structure of the project need to be reasonably aligned. Similar considerations apply for the data model. Scenarios for development distributability

will deal with the compatibility of the communication structures and data model of the system being developed and the coordination mechanisms utilized by the organizations doing the development.

System Quality Attributes

Physical systems, such as aircraft and automobiles and kitchen appliances, that rely on software embedded within them are designed to meet a whole litany of QAs: weight, size, electric consumption, power output, pollution output, weather resistance, battery life, and on and on. Often the software architecture can have a profound effect on the system's QAs. For example, software that makes inefficient use of computing resources might require additional memory, a faster processor, a bigger battery, or even an additional processor (we dealt with the topic of energy efficiency as a QA in Chapter 6). Additional processors will add to a system's power consumption, of course, but also to its weight, its physical profile, and expense.

Conversely, the architecture or implementation of a system can enable or preclude software from meeting its QA requirements. For example:

1. The performance of a piece of software is fundamentally constrained by the performance of the processor that runs it. No matter how well you design the software, you just can't run the latest whole-earth weather forecasting models on Grandpa's laptop and expect to know if it's going to rain tomorrow.
2. Physical security is probably more important and more effective than software security at preventing fraud and theft. If you don't believe this, write your laptop's password on a slip of paper, tape it to your laptop, and leave it in an unlocked car with the windows down. (Actually, please don't do that. Consider this a thought experiment.)

The lesson here is that if you are the architect for software that resides in a physical system, you will need to understand the QAs that are important for the entire system to achieve, and work with the *system* architects and engineers to ensure that your software architecture contributes positively to achieving them.

The scenario techniques we introduced for software QAs work equally well for system QAs. If the system engineers and architects aren't already using them, try to introduce them.

14.2 Using Standard Lists of Quality Attributes—Or Not

Architects have no shortage of QA lists for software systems at their disposal. The standard with the pause-and-take-a-breath title of “ISO/IEC FCD 25010: Systems and Software Engineering: Systems and Software Product Quality Requirements and Evaluation (SQuaRE): System and Software Quality Models” is a good example (Figure 14.1). This standard divides QAs into those supporting a “quality in use” model and those supporting a “product quality”

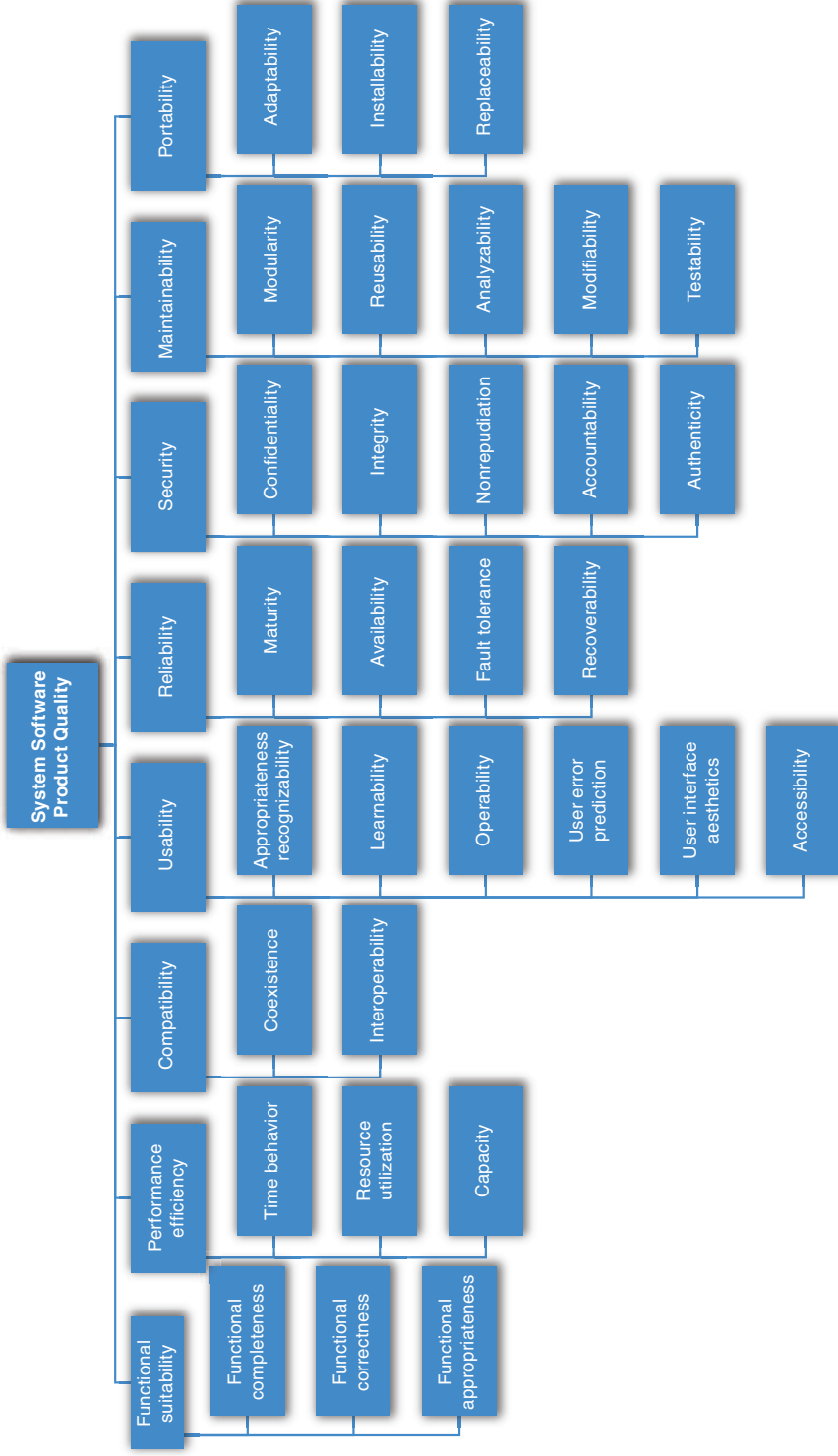


FIGURE 14.1 ISO/IEC FCD 25010 Product Quality Standard

model. That division is a bit of a stretch in some places, but it nevertheless begins a divide-and-conquer march through a breathtaking array of qualities.

ISO 25010 lists the following QAs that deal with product quality:

- *Functional suitability.* Degree to which a product or system provides functions that meet the stated and implied needs when used under the specified conditions.
- *Performance efficiency.* Performance relative to the amount of resources used under the stated conditions.
- *Compatibility.* Degree to which a product, system, or component can exchange information with other products, systems, or components, and/or perform its required functions, while sharing the same hardware or software environment.
- *Usability.* Degree to which a product or system can be used by specified users to achieve specified goals with effectiveness, efficiency, and satisfaction in a specified context of use.
- *Reliability.* Degree to which a system, product, or component performs the specified functions under the specified conditions for a specified period of time.
- *Security.* Degree to which a product or system protects information and data so that persons or other products or systems have the degree of data access appropriate to their types and levels of authorization.
- *Maintainability.* Degree of effectiveness and efficiency with which a product or system can be modified by the intended maintainers.
- *Portability.* Degree of effectiveness and efficiency with which a system, product, or component can be transferred from one hardware, software, or other operational or usage environment to another.

In ISO 25010, these “quality characteristics” are each composed of “quality sub-characteristics” (for example, nonrepudiation is a sub-characteristic of security). The standard slogs through almost five dozen separate descriptions of quality sub-characteristics in this way. It defines for us the qualities of “pleasure and “comfort.” It distinguishes between “functional correctness” and “functional completeness,” and then adds “functional appropriateness” for good measure. To exhibit “compatibility,” systems must either have “interoperability” or just plain “coexistence.” “Usability” is a product quality, not a quality-in-use quality, although it includes “satisfaction,” which *is* a quality-in-use quality. “Modifiability” and “testability” are both part of “maintainability.” So is “modularity,” which is a strategy for achieving a quality rather than a goal in its own right. “Availability” is part of “reliability.” “Interoperability” is part of “compatibility.” And “scalability” isn’t mentioned at all.

Got all that?

Lists like these—and there are many of them floating around—do serve a purpose. They can be helpful checklists to assist requirements gatherers in making sure that no important needs were overlooked. Even more useful than standalone lists, they can serve as the basis for creating your own checklist that contains the QAs of concern in your domain, your industry, your organization, your products. QA lists can also serve as the basis for establishing measures, though the names themselves give little clue as to how to do this. If “fun” turns out to be an important concern in your system, how do you measure it to know if your system is providing enough of it?

General lists like these also have some drawbacks. First, no list will ever be complete. As an architect, you will inevitably be called upon to design a system to meet a stakeholder concern not foreseen by any list-maker. For example, some writers speak of “manageability,” which expresses how easy it is for system administrators to manage the application. This can be achieved by inserting useful instrumentation for monitoring operations and for debugging and performance tuning. We know of an architecture that was designed with the conscious goal of retaining key staff and attracting talented new hires to a quiet region of the American Midwest. That system’s architects spoke of imbuing the system with “Iowability.” They achieved it by bringing in state-of-the-art technology and giving their development teams wide creative latitude. Good luck finding “Iowability” in any standard list of QAs, but that QA was as important to that organization as any other.

Second, lists often generate more controversy than understanding. You might argue persuasively that “functional correctness” should be part of “reliability,” or that “portability” is just a kind of “modifiability,” or that “maintainability” is a kind of “modifiability” (not the other way around). The writers of ISO 25010 apparently spent time and effort deciding to make security its own characteristic, instead of a sub-characteristic of functionality, which it was in a previous version. We strongly believe that effort in making these arguments could be better spent elsewhere.

Third, these lists often purport to be *taxonomies*—that is, lists with the special property that every member can be assigned to exactly one place. But QAs are notoriously squishy in this regard. For example, we discussed denial of service as being part of security, availability, performance, and usability in Chapter 3.

These observations reinforce the lesson introduced in Chapter 3: QA names, by themselves, are largely useless and are at best invitations to begin a conversation. Moreover, spending time worrying about which qualities are subqualities of which other qualities is almost useless. Instead, scenarios provide the best way for us to specify precisely what we mean when we speak of a QA.

Use standard lists of QAs to the extent that they are helpful as checklists, but don’t feel the need to slavishly adhere to their terminology or structure. And don’t fool yourself that such a checklist removes the need for deeper analysis.

14.3 Dealing with “X-Ability”: Bringing a New QA into the Fold

Suppose, as an architect, you had to deal with a QA for which there is no compact body of knowledge, no “portfolio” like Chapters 4–13 provided for those QAs. Suppose you find yourself having to deal with a QA like “development distributability” or “manageability” or even “Iowability”? What do you do?

Capture Scenarios for the New Quality Attribute

The first step is to interview the stakeholders whose concerns have led to the need for this QA. You can work with them, either individually or as a group, to build a set of attribute characterizations that refine what is meant by the QA. For example, you might decompose development distributability into the subattributes of software segmentation, software composition, and team coordination. After that refinement, you can work with the stakeholders to craft a set of specific scenarios that characterize what is meant by that QA. An example of this process can be found in Chapter 22, where we describe building a “utility tree.”

Once you have a set of specific scenarios, then you can work to generalize the collection. Look at the set of stimuli you’ve collected, the set of responses, the set of response measures, and so on. Use those to construct a general scenario by making each part of the general scenario a generalization of the specific instances you collected.

Model the Quality Attribute

If you can build (or even better, find) a conceptual model of the QA, that foundation can be helpful in creating a set of design approaches for it. By “model,” we don’t mean anything more than an understanding of the set of parameters to which the QA is sensitive and the set of architectural characteristics that influence those parameters. For example, a model of modifiability might tell us that modifiability is a function of how many places in a system have to be changed in response to a modification, and the interconnectedness of those places. A model for performance might tell us that throughput is a function of transactional workload, the dependencies among the transactions, and the number of transactions that can be processed in parallel.

Figure 14.2 shows a simple queuing model for performance. Such models are widely used to analyze the latency and throughput of various types of queuing systems, including manufacturing and service environments, as well as computer systems.

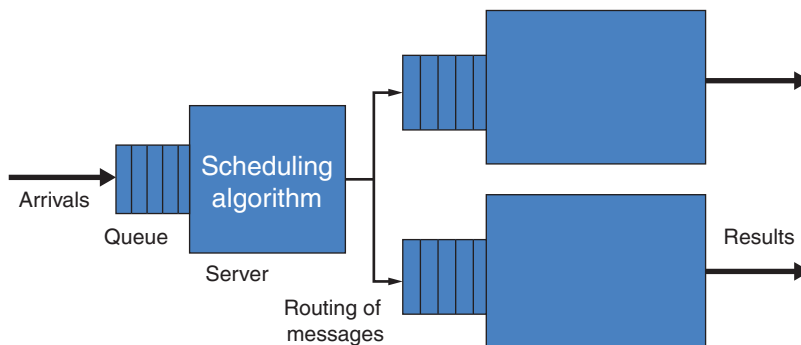


FIGURE 14.2 A generic queuing model

Within this model, seven parameters can affect the latency that the model predicts:

- Arrival rate
- Queuing discipline
- Scheduling algorithm
- Service time
- Topology
- Network bandwidth
- Routing algorithm

These are the *only* parameters that can affect latency within this model. This is what gives the model its power. Furthermore, each of these parameters can be affected by various architectural decisions. This is what makes the model useful for an architect. For example, the routing algorithm can be fixed or it could be a load-balancing algorithm. A scheduling algorithm must be chosen. The topology can be affected by dynamically adding or removing new servers. And so forth.

If you are creating your own model, your set of scenarios will inform your investigation. Its parameters can be derived from the stimuli (and its sources), the responses (and their measures), the artifacts (and their properties), and the environment (and its characteristics).

Assemble Design Approaches for the New Quality Attribute

The process of generating a set of mechanisms based on a model includes the following steps:

- Enumerate the model's parameters.
- For each parameter, enumerate the architectural characteristics (and the mechanisms to achieve those characteristics) that can affect this parameter. You can do this by:
 - Revisiting a body of mechanisms you're familiar with and asking yourself how each one affects the QA parameter.
 - Searching for designs that have successfully dealt with this QA. You can search on the name you've given the QA itself, but you can also search for the terms you chose when you refined the QA into subattributes.
 - Searching for publications and blog posts on this QA and attempting to generalize their observations and findings.
 - Finding experts in this area and interviewing them or simply writing and asking them for advice.

What results is a list of mechanisms to, in the example case, control performance and, in the more general case, to control the QA that the model is concerned with. This makes the design problem much more tractable. This list of mechanisms is finite and reasonably small, because the number of parameters of the model is bounded and for each parameter, the number of architectural decisions to affect the parameter is limited.

14.4 For Further Reading

The mother of all QA lists may be the one on—where else?—Wikipedia. This list can be found, naturally enough, under “List of system quality attributes.” As this book went to publication, you could gorge yourself on definitions of more than 80 distinct QAs. Our favorite is “demonstrability,” which is helpfully defined as the quality of being demonstrable. Who says you can’t believe what you read on the Internet?

See Chapter 8 of [Bass 19] to get a list of qualities of a deployment pipeline. These include traceability, testability (of the deployment pipeline), tooling, and cycle time.

14.5 Discussion Questions

1. The Kingdom of Bhutan measures the happiness of its population, and government policy is formulated to increase Bhutan’s GNH (gross national happiness). Read about how the GNH is measured (try grossnationalhappiness.com) and then sketch a general scenario for the QA of *happiness* that will let you express concrete happiness requirements for a software system.
2. Choose a QA not described in Chapters 4–13. For that QA, assemble a set of specific scenarios that describe what you mean by it. Use that set of scenarios to construct a general scenario for it.
3. For the QA you chose for question 2, assemble a set of design mechanisms (patterns and tactics) that help you achieve it.
4. Repeat questions 2 and 3 for the QA of *development cost*, and then for the QA of *operating cost*.
5. What might cause you to add a tactic or pattern to the sets of QAs already described in Chapters 4–13 (or any other QA, for that matter)?
6. Discuss how you think development distributability tends to trade off against the QAs of performance, availability, modifiability, and integrability.
7. Research some QA lists for things that are not software systems: qualities of a good car, for example, or a good person to be in a relationship with. Add qualities of your own choosing to the list or lists that you find.
8. Development-time tactics have to do with separating and encapsulating responsibilities. Performance tactics have to do with putting things together. That is why they are perpetually in conflict. Must it always be so? Is there a principled way of quantifying the tradeoffs?

9. Is there a taxonomy of tactics? Chemists have the periodic table and laws of molecular interaction, atomic physicists have their catalogs of subatomic particles and laws for what happens when they collide, pharmacologists have their catalogs of chemicals and laws for their interactions with receptors and metabolic systems, and so forth. What is the equivalent for tactics? And are there laws for their interaction?
10. Security is a QA that is especially sensitive to processes that take place in the physical world outside the computer: processes for applying patches, processes for choosing and safeguarding your passwords, processes for physically securing the installations where computers and data live, processes for deciding whether to trust a piece of imported software, processes for deciding whether to trust a human developer or user, and so forth. What are the corresponding processes that are important for performance? Or usability? Are there any? Why is security so process-sensitive? Should processes be a portion of the QA structure or are they orthogonal to it?
11. What is the relationship between each pair of QAs in the following list?
 - Performance and security
 - Security and buildability
 - Energy efficiency and time to market